Linux System Administration (E1ITA307)

Linux Infrastructure Deployment Project Report

On

LINUX SERVER MIGRATION

Submitted by

SHOBITH I K (2411021061604)

ALLEN CHRIST A (2411021061266)

SRIKANTH C (2411021061560)

ANASUYA K ANIL (L2411021061702)

In partial fulfillment of the award of the degree of

BACHELOR OF TECHNOLOGY

In

Information Technology

Under the Supervision of

Prof. Ananthanagu U

ALLIANCE SCHOOL OF ADVANCED COMPUTING

ALLIANCE UNIVERSITY

BENGALURU

Oct 2026

DEPARTMENT OF INFORMATION TECHNOLOGY

ALLIANCE SCHOOL OF ADVANCED COMPUTING

CERTIFICATE

This is to certify that the Project work entitled **"Linux Server Migration"** submitted by Shobith I K [2411021061604], Allen Christ A [2411021061266], Srikanth C [2411021061560] and Anasuya K Anil [L2411021061702] in partial fulfillment of the requirements for the award of the degree of Bachelor of Technology in Information Technology - IT – AR/VR of Alliance University, is a bonafide work during the academic year 2026-2027.

Prof. Ananthanagu U Course Handling Faculty	Prof. Ananthanagu U Course Coordinator	Dr. Shekhar R Professor & HoD

Table of Contents

1. Introduction 4
1.1 Aim 4
1.2 Background 4
1.3 Problem Statement 5
1.4 Objectives 5
2. Methodology 7
2.1 Tools and Technologies Used 7
2.2 Approach / Development Process 8
3. Linux System Administration – Design and Implementation 11
3.1 System Architecture 11
3.2 Linux Environment Setup 13
3.3 User and Group Management 17
3.4 File and Directory Management 20
3.5 Package and Process Management 23
3.6 Storage and File System Configuration 24
3.7 Migration, Backup and Automation 34
3.8 Security and Access Control 44
3.9 Migration Verification Dashboard 48
4. Results and Testing 55
4.1 Implementation Results 55
4.2 Testing and Validation 63
4.3 Dashboard Results and Testing 65
5. Conclusion and Future Scope 76
5.1 Conclusion 76
5.2 Future Scope 76
References 78
Appendices 79
Appendix A: System Architecture and Workflow 79
Appendix B: Linux Commands Quick Reference 80
Appendix C: Scripts and Configuration Files 83
Appendix E: Migration Verification Dashboard — Source Code 84

1. Introduction

1.1 Aim

The main aim of this project is to design, configure and deploy a Linux server migration infrastructure that migrates storage, file systems and user data from one Linux server to another while preserving permissions and ownership. The project prepares the destination server from the basics, starting with disk partitioning, and then builds the storage layers on it using the GUID Partition Table (GPT), Logical Volume Manager (LVM) and the ext4 file system.

Data is moved from the source server to the destination server using SSH, tar and rsync with options that preserve numeric ownership, permissions, hard links, access control lists (ACLs) and extended attributes. The migration is then verified using file inventories, ownership and permission checks, ACL inspection and SHA-256 checksums. Backup and recovery testing, a Bash backup script scheduled with Cron, firewall and access-control checks, and complete documentation make up the rest of the project.

1.2 Background

Linux is widely used in enterprise servers, cloud infrastructure and data centres. Servers are periodically replaced or upgraded to obtain more capacity, newer hardware or a cleaner storage layout. When this happens, the data held on the old server must be transferred to the new one without losing the file permissions, ownership and directory structure that applications and users depend on.

Linux identifies the owner of a file by a numeric user ID (UID) and group ID (GID), not by name. If an account has a different UID on the destination server, files copied from the source appear to belong to a different user. A correct migration therefore has to preserve identities, ownership, permissions and, where used, ACLs and extended attributes, in addition to the file contents.

The project is carried out in a controlled laboratory environment of two Linux virtual machines (a source VM and a destination VM) managed from an EndeavourOS host. An important distinction is that copying files is not the same as migrating an entire operating system. The

project scope is the storage, file systems, user data and the identities needed to access that data on the destination server.

1.3 Problem Statement

The problem statement assigned to the project is: "Linux Server Migration – **Migrate storage, file systems, and user data from one Linux server to another while preserving permissions and ownership. Migration report, verification, documentation.**"

A careless migration can cause several problems: files that end up with the wrong owner because of a UID/GID mismatch, lost permissions, ACLs or extended attributes, incomplete or corrupted transfers, an unmounted destination volume that silently fills the root file system, no recovery path if the transfer fails, and no evidence that the destination matches the source. This project addresses these risks with a planned, verified and documented migration procedure.

1.4 Objectives

The main objectives of the Linux Server Migration project are:

- To prepare a source Linux server containing sample users, groups, shared data, ownership and permissions.
- To partition the destination disk using a GPT partition table and configure LVM physical volumes, a volume group and logical volumes.
- To create ext4 file systems and mount them persistently using UUID-based `/etc/fstab` entries.
- To recreate the required users and groups on the destination with the same numeric UIDs and GIDs.
- To create a verified pre-migration backup and migrate the data using SSH, tar and rsync while preserving permissions and ownership.
- To verify the migration using file inventories, ownership and permission checks, ACL checks and SHA-256 checksums.
- To test backup and recovery by deleting a test file and restoring it from the archive.
- To automate backups using a Bash script and Cron scheduling, and to apply firewall and least-privilege access controls.

2. Methodology

2.1 Tools and Technologies Used

2.1.1 Software Requirements

The project uses EndeavourOS (an Arch-based Linux distribution) as the host system, Linux virtual machines for the source and destination servers, and standard Linux administration tools for storage, migration, backup, automation and security.

Table 2.1 lists the software and tools used in the project and the purpose of each.

Table 2.1: Software requirements

Software / Tool	Purpose
EndeavourOS (host)	Host operating system used to run the virtual machines, prepare commands and collect screenshots.
VirtualBox (or another hypervisor)	Runs the source and destination virtual machines on a private network and provides the separate destination data disk.
Linux source and destination VMs	Source server with sample users and data; destination server with new partitioned and LVM-managed storage.
pacman	Package management on Arch-based systems (`pacman -Syu`, `pacman -S`).
lsblk, fdisk, partprobe, blkid	Identify disks and create, inspect and validate partitions and file-system UUIDs.
lvm2 (pvcreate, vgcreate, lvcreate)	Logical Volume Manager for physical volumes, volume groups and logical volumes.
mkfs.ext4, mount, findmnt, /etc/fstab	Create ext4 file systems, mount them and make mounts persistent.
openssh (ssh, scp, sshd)	Encrypted remote access and file transfer between the two servers.
rsync	Efficient transfer and synchronization of files with metadata preservation.
acl and attr	ACL and extended-attribute utilities (getfacl, getfattr).
tar and sha256sum	Create archives that preserve ownership, ACLs and xattrs; create and verify SHA-256 checksums.
cronie (Cron)	Schedules the automated backup script.
firewalld	Firewall service used to control network access, including SSH.
Bash and a text editor (nano)	Backup script and configuration-file editing.

2.1.2 Hardware Requirements

The project is implemented on a laptop or desktop computer running EndeavourOS with hardware virtualization enabled. The host must have enough processor capacity and RAM to run two Linux virtual machines at the same time, and enough free storage for the virtual disks, snapshots, archives and project documentation.

- A laptop or desktop computer capable of running two virtual machines simultaneously.
- A virtual OS disk for each VM and an additional empty virtual disk of approximately 20 GB attached to the destination VM.
- A private virtual network connecting both VMs (for example, 192.168.56.0/24).
- Sufficient free disk space for snapshots, backup archives and logs.

2.1.3 Other Tools and Resources

- **Terminal:** used to run all commands on the host and on both virtual machines.
- **VM snapshots:** taken for each VM before the practical work so that mistakes can be reverted.
- **Source-side and destination-side logs:** `/var/log/company-backup/backup.log` records automated backup activity.
- **Evidence collection:** terminal screenshots and saved command outputs captured throughout the project for the migration report.
- **Architecture and workflow diagrams:** illustrate the source server, destination server, storage layers, migration path and verification.

2.2 Approach / Development Process

The project was organized into phases covering planning, environment setup, source preparation, destination storage configuration, backup, migration, verification, recovery testing, automation, security and documentation. The complete workflow is shown in Appendix A.

Step 1: Problem Identification and Planning

The problem statement was analysed: migrate storage, file systems and user data between two Linux servers while preserving permissions and ownership. The deliverables were defined as a

working migration, verification evidence, a backup and recovery test, automation, and documentation.

Step 2: Requirement Analysis and Architecture

The required tools (LVM, rsync, OpenSSH, ACL and attribute utilities, tar, Cron and firewalld) and the lab architecture were identified. The architecture consists of a source VM (192.168.56.10), a destination VM (192.168.56.20) with a separate 20 GB data disk, GPT partitioning, LVM2, ext4, SSH + rsync migration and a separate backup logical volume.

Step 3: Environment Setup

Both VMs were prepared with `pacman -Syu` and the required packages. The SSH service was enabled, the private network was verified with`ip -br addr` and`ping`, and a snapshot of each VM was taken.

Step 4: Source Server Preparation

The groups *developers* and *finance* and the users *devuser* and *financeuser* were created, the `/srv/company` directory tree was built, and ownership, setgid directory modes and file permissions were applied. The numeric UIDs and GIDs were recorded as before-migration evidence.

Step 5: Destination Storage Configuration

On the destination VM the new disk was inspected, a GPT partition of type Linux LVM was created, and the LVM physical volume, volume group (`vg_migration`) and logical volumes (`lv_data`, `lv_backup`) were created. ext4 file systems were created and mounted at `/srv/company` and `/srv/backup`, with persistent UUID-based entries in`/etc/fstab`.

Step 6: Identity Preparation and Pre-migration Backup

Matching groups and users were created on the destination with identical numeric IDs. A pre-migration archive of the source data was created with `tar --acls --xattrs --numeric-owner` and protected with a SHA-256 checksum.

Step 7: Data Migration

The data was transferred to the destination with SSH and tar/rsync using `-aHAX --numeric-ids`. A dry-run was reviewed before the final synchronization.

Step 8: Verification, Recovery, Automation and Security

The destination was compared with the source (inventory, counts, ownership, modes, ACLs and checksums), user access was tested, a deleted file was restored from backup, the backup script was scheduled with Cron, and the firewall and least-privilege permissions were checked.

Step 9: Documentation

Commands, outputs, screenshots, test results and the acceptance checklist were recorded to produce this report.

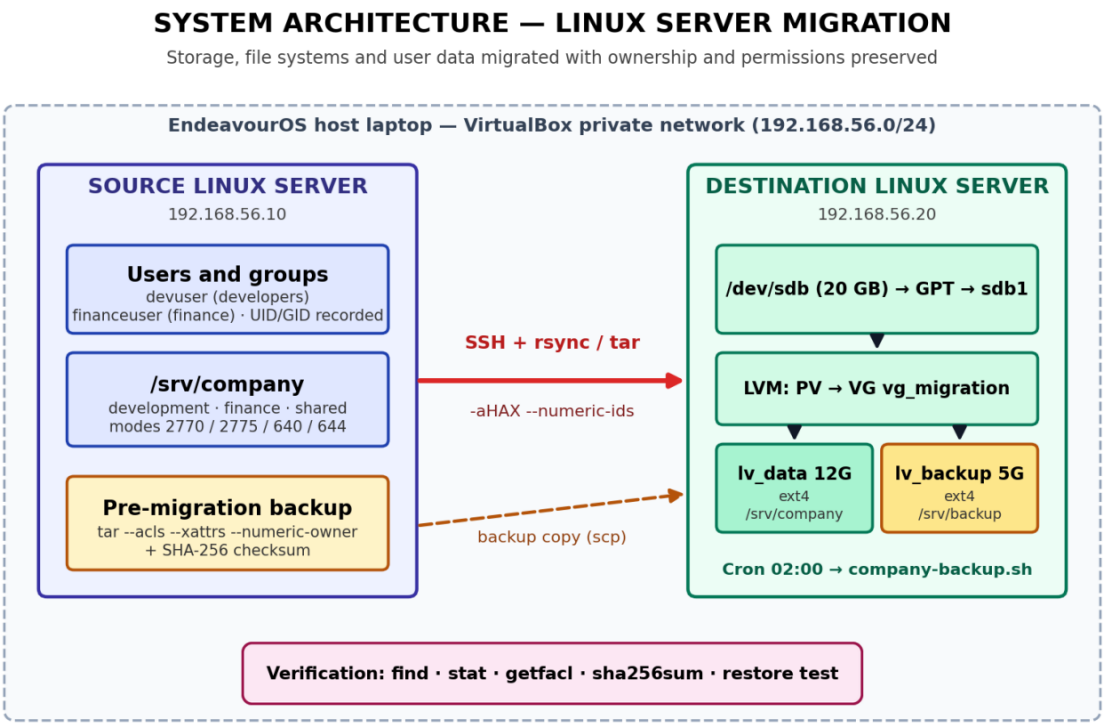
3. Linux System Administration – Design and Implementation

3.1 System Architecture

The Linux Server Migration project is designed around two Linux servers. The **source server** holds the existing users, groups and shared company data. The **destination server** is a freshly prepared system with a separate data disk that is partitioned, managed with LVM, formatted with ext4 and mounted before the data is migrated to it. Data is transferred over SSH using tar and rsync, verified against the source, and protected by a backup volume and an automated backup script.

The two servers run as virtual machines on a private network, managed from an EndeavourOS host. Figure 3.1 presents the system architecture and the relationship between the components.

Figure 3.1: System architecture of the Linux Server Migration project



Components and their roles

1. **Administrator / Team:** performs the configuration, runs the migration, collects the evidence and reviews logs and results.

2. **Source Linux server (192.168.56.10):** holds the users *devuser* and *financeuser*, the groups *developers* and *finance* and the data tree `/srv/company`.
3. **Destination Linux server (192.168.56.20):** receives the migrated data on LVM-backed ext4 volumes and holds the backup volume and the automation script.
4. **Storage:** the 20 GB destination disk `/dev/sdb` is partitioned with GPT and managed by LVM as *vg_migration*, providing *lv_data* (12 GB) and *lv_backup* (5 GB).
5. **Network:** a private virtual network connects the VMs. SSH carries the encrypted transfers and is the only service the firewall is required to permit for the migration.
6. **Migration tools:** tar (archive and seed), rsync (initial and final synchronization), sha256sum (integrity checks).
7. **Backup system:** a pre-migration archive with checksum on the source, a copy on the destination backup volume and timestamped archives produced by the automated script.
8. **Automation and monitoring:** a Bash script scheduled by Cron writes archives and records activity in `/var/log/company-backup/backup.log`.

Table 3.1 summarises the main parameters of the project architecture.

Table 3.1: Project architecture parameters

Component	Example configuration
Source VM	EndeavourOS / Arch Linux
Destination VM	EndeavourOS / Arch Linux
Source IP	`192.168.56.10`
Destination IP	`192.168.56.20`
Destination data disk	`/dev/sdb`, 20 GB virtual disk
Partition scheme	GPT
Volume management	LVM2
File system	ext4
Migration method	SSH + rsync (tar archive as seed and rollback copy)
Backup	Separate backup logical volume mounted at `/srv/backup`
Sample data directory	`/srv/company`

3.2 Linux Environment Setup

3.2.1 Linux Distribution and Virtual Machine Plan

EndeavourOS is an Arch-based distribution, so the package-management and service commands in this report use `pacman`, `systemctl` and related tools. A server-oriented distribution such as Ubuntu Server could be used for the VMs with the same workflow, but the package-management and networking commands would differ.

- **Source VM:** Linux server containing sample users, groups and files.
- **Destination VM:** fresh Linux installation with an additional separate disk for partitioning and LVM.
- **EndeavourOS laptop:** used to manage the VMs, prepare commands and collect screenshots.

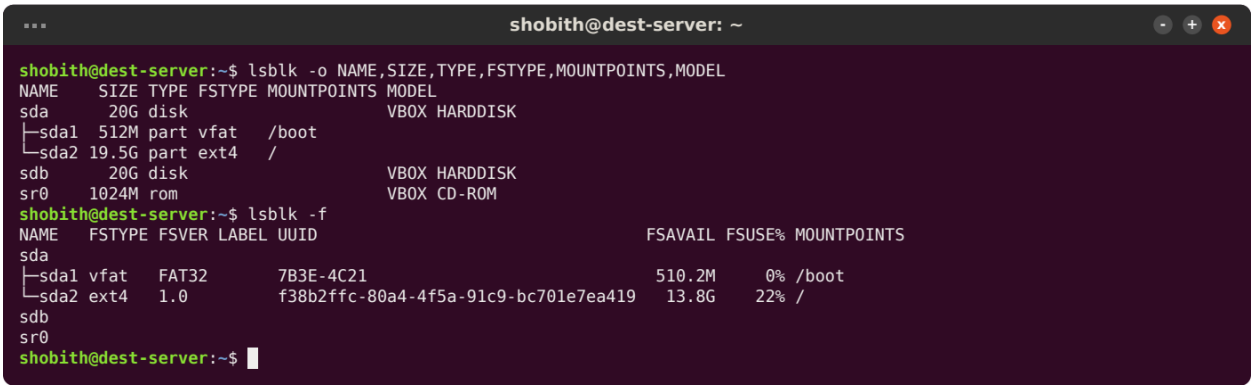
A hypervisor such as VirtualBox is installed, both VMs are created and connected to a private network, the destination VM receives an extra virtual disk of about 20 GB, and a snapshot of each VM is taken before the practical work. The disks are identified with:

```
lsblk -o NAME,SIZE,TYPE,FSTYPE,MOUNTPOINTS,MODEL
```

Example output of the command is shown below. Here `/dev/sda` is the operating-system disk and `/dev/sdb` is the new destination data disk.

```
NAME SIZE TYPE FSTYPE MOUNTPOINTS
sda 20G disk
|-sda1 512M part vfat /boot
`-sda2 19.5G part ext4 /
sdb 20G disk
```

Figure 3.2: Disk identification on the destination VM



3.2.2 Required Packages

The system is updated and the tools needed for storage and migration are installed on both VMs:

```
sudo pacman -Syu
sudo pacman -S lvm2 rsync openssh acl attr
```

```
shobith@dest-server:~$ sudo pacman -Syu
[sudo] password for shobith:
:: Synchronizing package databases...
 endeavouros is up to date
 core is up to date
 extra is up to date
:: Starting full system upgrade...
 there is nothing to do
shobith@dest-server:~$ sudo pacman -S lvm2 rsync openssh acl attr
warning: acl-2.3.2-1 is up to date -- reinstalling
warning: attr-2.5.2-1 is up to date -- reinstalling
warning: openssh-10.0p1-3 is up to date -- reinstalling
warning: rsync-3.4.1-2 is up to date -- reinstalling
resolving dependencies...
looking for conflicting packages...

Packages (6) acl-2.3.2-1  attr-2.5.2-1  device-mapper-2.03.34-1  lvm2-2.03.34-1  openssh-10.0p1-3
             rsync-3.4.1-2

Total Download Size:   3.72 MiB
Total Installed Size:  9.84 MiB
Net Upgrade Size:      3.01 MiB

:: Proceed with installation? [Y/n] y
:: Retrieving packages...
 device-mapper-2.03.34-1-x86_64    532.1 KiB  2.84 MiB/s 00:00 [######################] 100%
 lvm2-2.03.34-1-x86_64            1481.3 KiB  4.31 MiB/s 00:00 [######################] 100%
 rsync-3.4.1-2-x86_64              501.9 KiB  3.97 MiB/s 00:00 [######################] 100%
 openssh-10.0p1-3-x86_64          1297.5 KiB  5.20 MiB/s 00:00 [######################] 100%
 acl-2.3.2-1-x86_64                70.4 KiB  1186 KiB/s 00:00 [######################] 100%
 attr-2.5.2-1-x86_64               46.1 KiB  1004 KiB/s 00:00 [######################] 100%
 Total (6/6)                              3.7 MiB  6.15 MiB/s 00:01 [######################] 100%
(6/6) checking keys in keyring                                  [######################] 100%
(6/6) checking package integrity                                [######################] 100%
(6/6) loading package files                                     [######################] 100%
(6/6) checking for file conflicts                               [######################] 100%
(6/6) checking available disk space                             [######################] 100%
:: Processing package changes...
(1/6) reinstalling acl                                          [######################] 100%
(2/6) reinstalling attr                                         [######################] 100%
(3/6) installing device-mapper                                  [######################] 100%
(4/6) installing lvm2                                           [######################] 100%
Optional dependencies for lvm2
    thin-provisioning-tools: thin provisioning
    dmeventd: monitoring of LVM volumes
(5/6) reinstalling openssh                                      [######################] 100%
Optional dependencies for openssh
    libfido2: FIDO/U2F support
    x11-ssh-askpass: input passphrase in X
    xorg-xauth: X11 forwarding [installed]
(6/6) reinstalling rsync                                        [######################] 100%
:: Running post-transaction hooks...
(1/4) Reloading system manager configuration...
(2/4) Arming ConditionNeedsUpdate...
(3/4) Updating the info directory file...
(4/4) Reloading device manager configuration...
shobith@dest-server:~$
```

Table 3.2 lists the packages installed on both servers and the purpose of each.

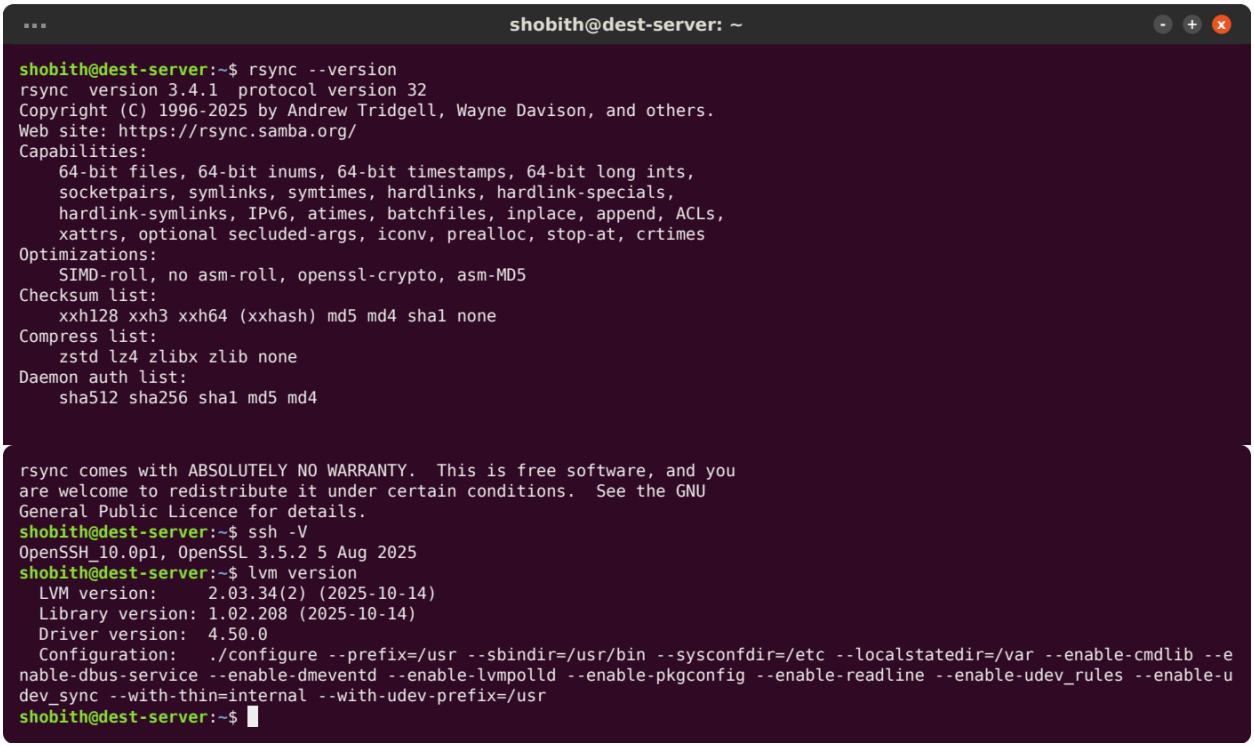
Table 3.2: Required packages

Package	Purpose
lvm2	LVM volume management
rsync	Efficient data transfer
openssh	Secure remote access
acl	Access control list utilities
attr	Extended attribute utilities

The installed tools are checked with:

```
rsync --version
ssh -V
lvm version
```

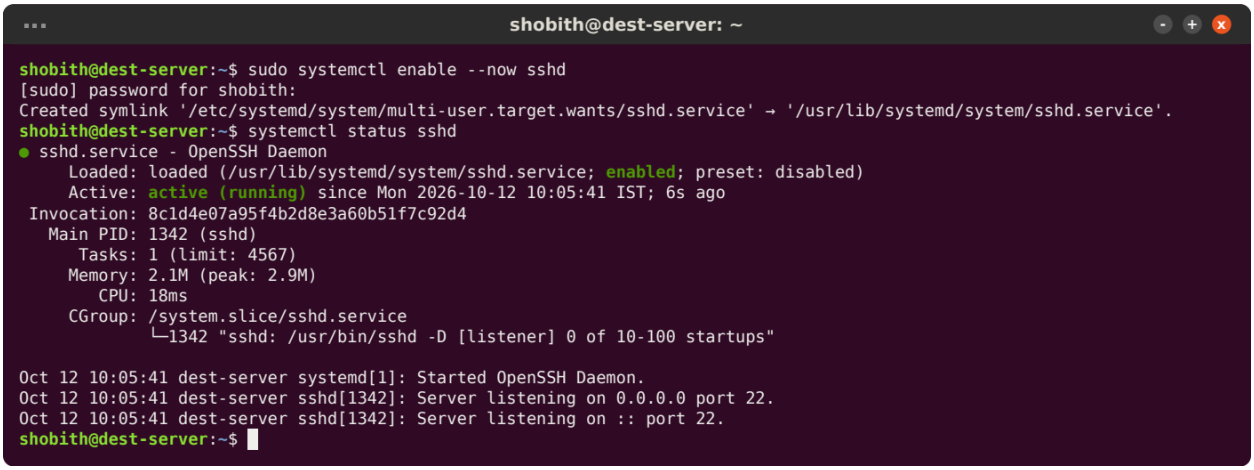
Figure 3.3: Package installation and version checks



3.2.3 System Services (SSH)

The SSH service is enabled and started on both VMs. On Arch-based systems the service is normally named `sshd`.

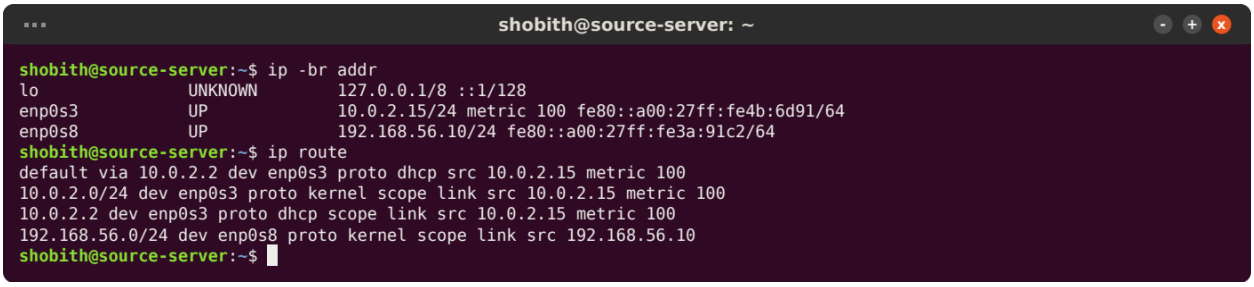
```
sudo systemctl enable --now sshd
systemctl status sshd
```



3.2.4 Network Configuration

Both VMs must be able to communicate over the private network. The addresses and routes are inspected on each VM:

```
ip -br addr
ip route
```



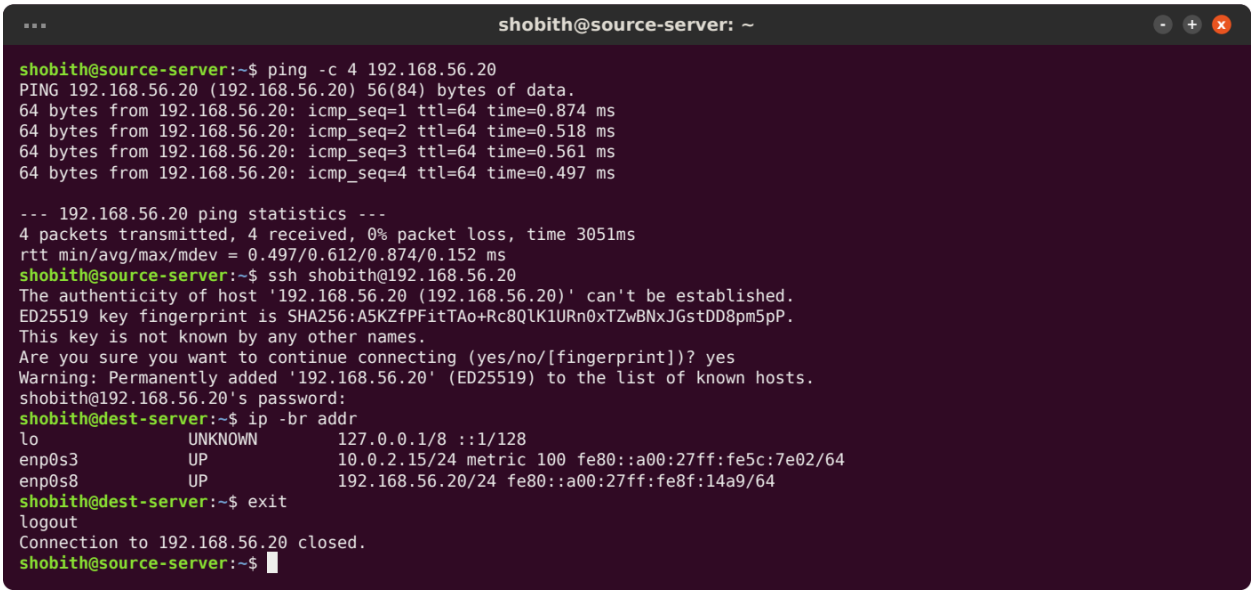
The examples in this report use 192.168.56.10 for the source and 192.168.56.20 for the destination. Connectivity and SSH are tested from the source:

```
ping -c 4 192.168.56.20
ssh your_destination_username@192.168.56.20
```

`your_destination_username` is replaced with an existing administrative account on the destination. If SSH fails, the checks are: `sshd` is running, both VMs are on the same reachable

network, and the firewall permits SSH. The firewall is not disabled globally just to make the connection work.

Figure 3.4: Network configuration and connectivity test



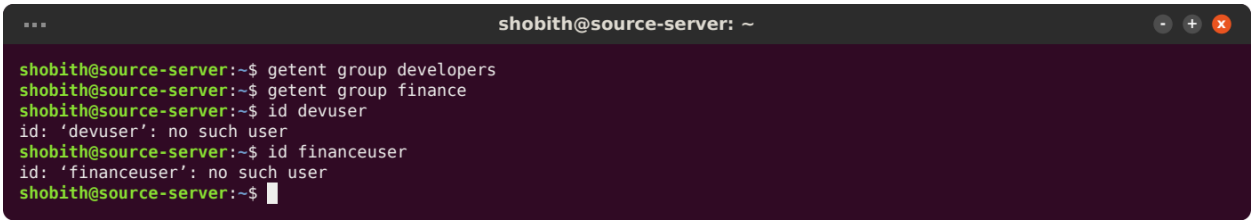
3.3 User and Group Management

User and group management is central to this project because Linux ownership is based on numeric identities. The source server simulates a small organization with a development team and a finance team.

3.3.1 Group and User Creation on the Source

The names are checked first to avoid creating duplicates:

```
getent group developers
getent group finance
id devuser
id financeuser
```



If they do not exist, the groups and users are created and passwords are set:

```
sudo groupadd developers
sudo groupadd finance
sudo useradd -m -s /bin/bash -g developers devuser
sudo useradd -m -s /bin/bash -g finance financeuser
sudo passwd devuser
sudo passwd financeuser
```



3.3.2 Password Management

Passwords are set with `sudo passwd`. The operating system stores them in hashed form and does not display them during normal account operations. Passwords are not migrated by this project's data transfer and must be set separately on the destination if required.

3.3.3 Recording the Numeric IDs

The numeric IDs of the accounts and groups are recorded on the source and saved as before-migration evidence:

```
id devuser
id financeuser
getent group developers
getent group finance
```

Why numeric IDs matter: Linux stores ownership using UIDs and GIDs. If the destination assigns a different UID to devuser, a file owned by the source user's UID may appear to belong to another user on the destination. Preserving names alone is insufficient.

Figure 3.5: Users, groups and numeric IDs on the source server



3.3.4 Recreating the Identities on the Destination

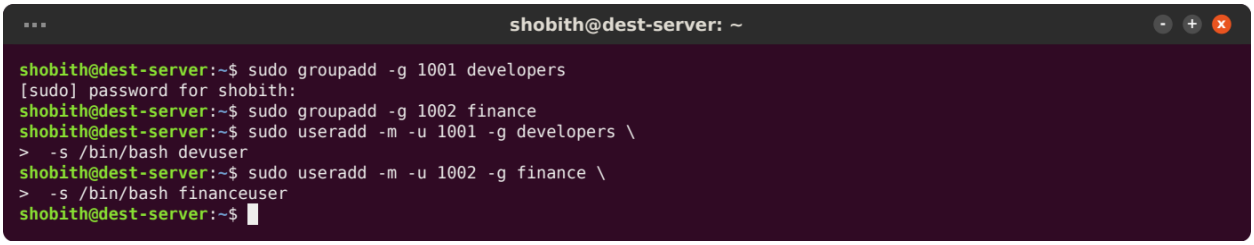
On the destination, matching groups and users are created with the same numeric GIDs and UIDs as the source. For example, if the source reports UID 1001 for devuser and GID 1001 for developers, the destination must use those IDs provided they are not already allocated. The numbers are examples only; the actual recorded values are used. Conflicts are checked first:

```
getent passwd 1001
getent group 1001
getent passwd devuser
getent group developers
```



If the IDs are available, the identities are created with explicit IDs (each placeholder is replaced with the recorded numeric value):

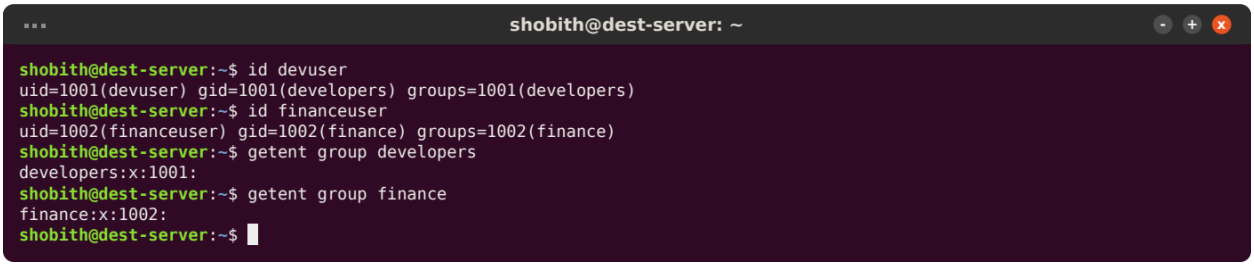
```
sudo groupadd -g <DEVELOPERS_GID> developers
sudo groupadd -g <FINANCE_GID> finance
sudo useradd -m -u <DEVUSER_UID> -g developers \
 -s /bin/bash devuser
sudo useradd -m -u <FINANCEUSER_UID> -g finance \
 -s /bin/bash financeuser
```



The result is verified with `id devuser`, `id financeuser`, `getent group developers` and `getent group finance`. If a group or user already exists it is inspected instead of being created again.

The source's entire `/etc/passwd`, `/etc/shadow` or `/etc/group` files are not copied over the destination files. They contain system accounts and distribution-specific configuration, so only the required project identities are recreated. Creating accounts also does not migrate passwords, SSH keys, supplementary group memberships or other account-specific settings; these are handled separately if the scope requires them.

Figure 3.6: Matching users and groups on the destination server

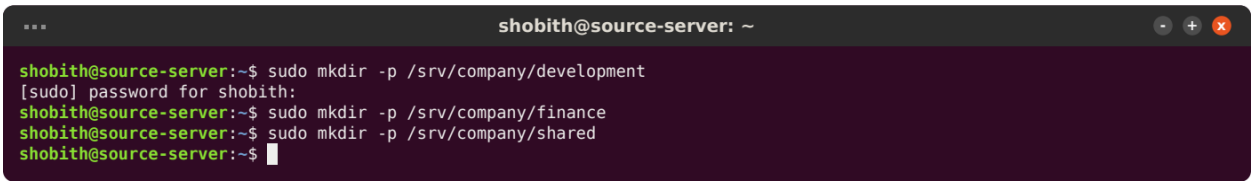


3.4 File and Directory Management

3.4.1 Directory Structure

A shared data structure is created on the source server under `/srv/company`, with separate directories for the development team, the finance team and shared documentation:

```
sudo mkdir -p /srv/company/development
sudo mkdir -p /srv/company/finance
sudo mkdir -p /srv/company/shared
```



3.4.2 Ownership and Permissions

Ownership is assigned and permissions are set on the directories:

```
sudo chown devuser:developers /srv/company/development
sudo chown financeuser:finance /srv/company/finance
sudo chown root:developers /srv/company/shared
sudo chmod 2770 /srv/company/development
sudo chmod 2770 /srv/company/finance
sudo chmod 2775 /srv/company/shared
```

```
shobith@source-server: ~
shobith@source-server:~$ sudo chown devuser:developers /srv/company/development
shobith@source-server:~$ sudo chown financeuser:finance /srv/company/finance
shobith@source-server:~$ sudo chown root:developers /srv/company/shared
shobith@source-server:~$ sudo chmod 2770 /srv/company/development
shobith@source-server:~$ sudo chmod 2770 /srv/company/finance
shobith@source-server:~$ sudo chmod 2775 /srv/company/shared
shobith@source-server:~$
```

The leading 2 in 2770 and 2775 enables the setgid bit on a directory. New items created within it normally inherit the directory's group, which keeps team data organized.

3.4.3 Sample Files

```
echo "Development server configuration" |
 sudo tee /srv/company/development/config.txt
echo "Finance migration records" |
 sudo tee /srv/company/finance/records.txt
echo "Enterprise shared documentation" |
 sudo tee /srv/company/shared/readme.txt
```

```
shobith@source-server: ~
shobith@source-server:~$ echo "Development server configuration" |
>  sudo tee /srv/company/development/config.txt
Development server configuration
shobith@source-server:~$ echo "Finance migration records" |
>  sudo tee /srv/company/finance/records.txt
Finance migration records
shobith@source-server:~$ echo "Enterprise shared documentation" |
>  sudo tee /srv/company/shared/readme.txt
Enterprise shared documentation
shobith@source-server:~$
```

Ownership and permissions are then applied to the files:

```
sudo chown devuser:developers /srv/company/development/config.txt
sudo chown financeuser:finance /srv/company/finance/records.txt
sudo chown root:developers /srv/company/shared/readme.txt
sudo chmod 640 /srv/company/development/config.txt
sudo chmod 640 /srv/company/finance/records.txt
sudo chmod 644 /srv/company/shared/readme.txt
```

```
shobith@source-server: ~
shobith@source-server:~$ sudo chown devuser:developers /srv/company/development/config.txt
shobith@source-server:~$ sudo chown financeuser:finance /srv/company/finance/records.txt
shobith@source-server:~$ sudo chown root:developers /srv/company/shared/readme.txt
shobith@source-server:~$ sudo chmod 640 /srv/company/development/config.txt
shobith@source-server:~$ sudo chmod 640 /srv/company/finance/records.txt
shobith@source-server:~$ sudo chmod 644 /srv/company/shared/readme.txt
shobith@source-server:~$
```

Table 3.3 shows the owner, group and permission mode set on the sample data.

Table 3.3: Source data ownership and permissions

Path	Owner : Group	Mode	Meaning
`/srv/company/development`	devuser : developers	2770	Owner and group full access, others none; setgid

Path	Owner : Group	Mode	Meaning
`/srv/company/finance`	financeuser : finance	2770	Owner and group full access, others none; setgid
`/srv/company/shared`	root : developers	2775	Owner and group full access, others read/execute; setgid
`development/config.txt`	devuser : developers	640	Owner read/write, group read, others none
`finance/records.txt`	financeuser : finance	640	Owner read/write, group read, others none
`shared/readme.txt`	root : developers	644	Owner read/write, group and others read

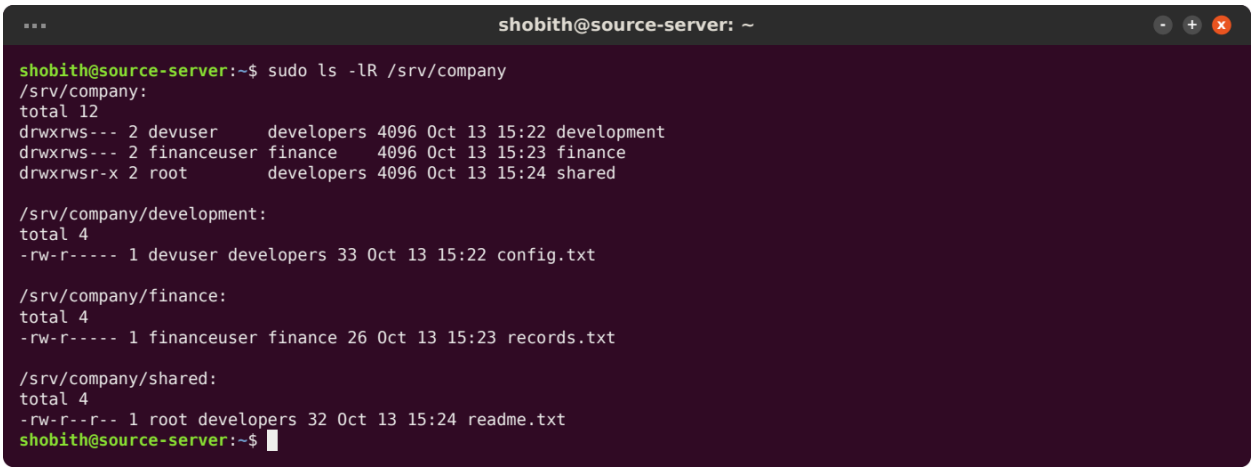
3.4.4 Verification and File Searching

The source data is verified and recorded as the before-migration reference:

```
sudo ls -lR /srv/company
```

The find utility is later used to produce a sorted inventory of the files and directories that can be compared between the servers (Section 4.1).

Figure 3.7: Source data tree with ownership and permissions



3.5 Package and Process Management

3.5.1 Package Installation and Updates

Packages on EndeavourOS are managed with pacman. `pacman -Syu` synchronizes the package databases and upgrades the system, and `pacman -S` installs the required packages. The packages used by the project are lvm2, rsync, openssh, acl and attr, plus cronie for scheduling and firewalld for the firewall (Sections 3.7 and 3.8).

3.5.2 Dependency and Tool Verification

Tool availability is verified with commands such as `rsync --version`, `ssh -V` and `lvm version`. Package-manager dependency resolution ensures that the supporting libraries of each tool are installed with it.

3.5.3 Service Management

Services are managed and checked with`systemctl`. The project depends on three services:

Table 3.4 lists the services used by the project and the role of each.

Table 3.4: Services used by the project

Service	Role	Check
sshd	Encrypted remote access and transfer between servers	`systemctl is-active sshd`
cronie	Runs the scheduled backup script	`systemctl is-active cronie`
firewalld	Controls network access	`systemctl is-active firewalld`

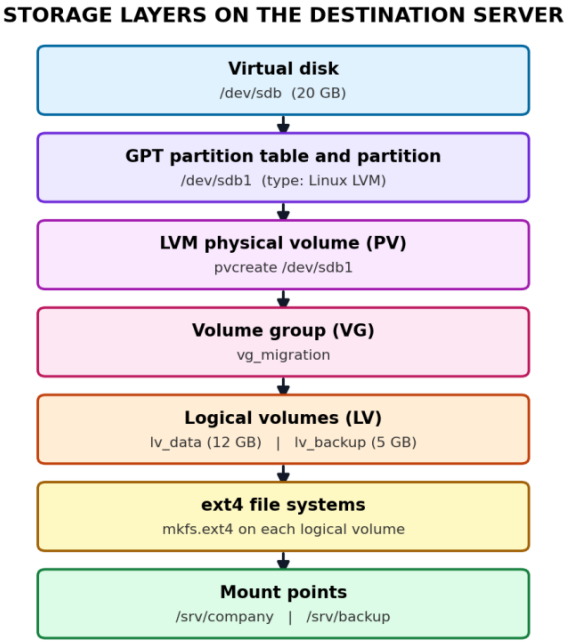
3.5.4 Transfer Monitoring

During the rsync transfers, progress is monitored with the `--info=progress2` option. The backup script records its start, completion and any error in a log file, so that automated activity can be reviewed afterwards.

3.6 Storage and File System Configuration

The destination server is prepared from the basics: a new disk is partitioned, LVM is configured, ext4 file systems are created and mounted, and the mounts are made persistent. Figure 3.8 shows the storage layers that are built.

Figure 3.8: Storage layers built on the destination server



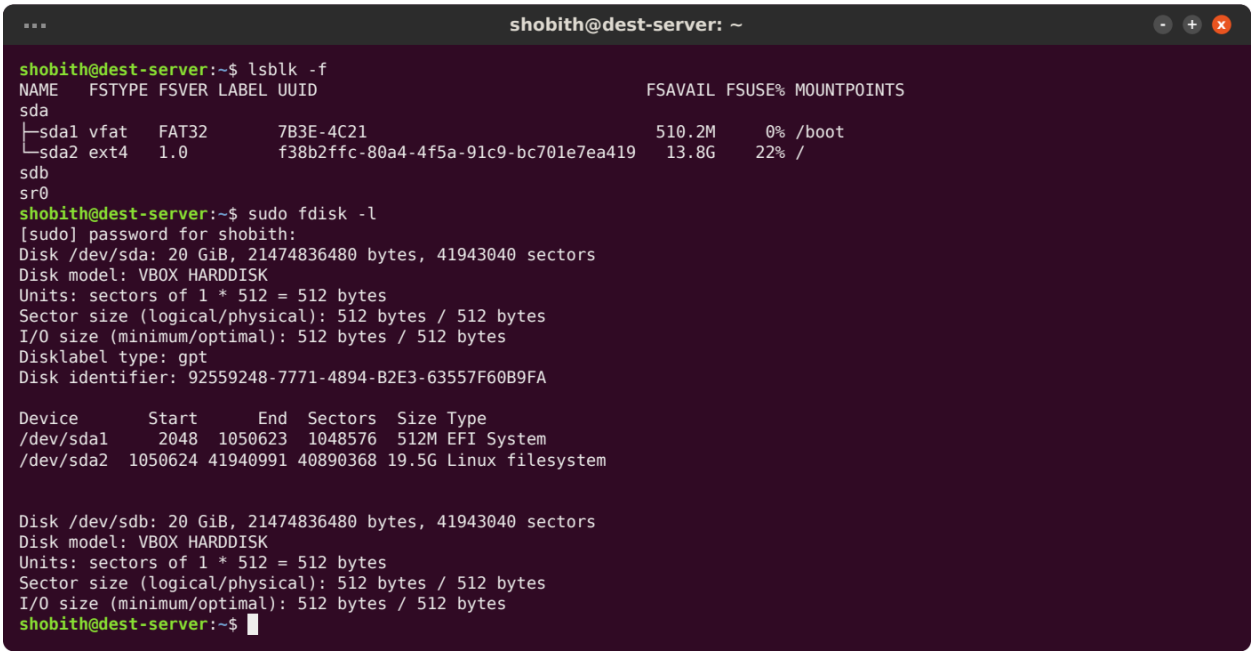
3.6.1 Storage Fundamentals

- **Disk partitioning:** divides a disk into independently addressable regions.
- **File system:** organizes files, directories, metadata and permissions within a storage device.
- **LVM:** pools storage into volume groups and allocates it as logical volumes.
- **Mount point:** a directory through which a file system becomes accessible.
- **Ownership:** identifies the owning user and group, normally by numeric UID and GID.
- **Permissions:** control read, write and execute access.
- **Backup:** a separate recoverable copy of data.
- **Migration:** transfers data and the relevant metadata to the destination while maintaining the required behaviour.

3.6.2 Identifying the Destination Disk

On the destination VM the disks are inspected before anything is written:

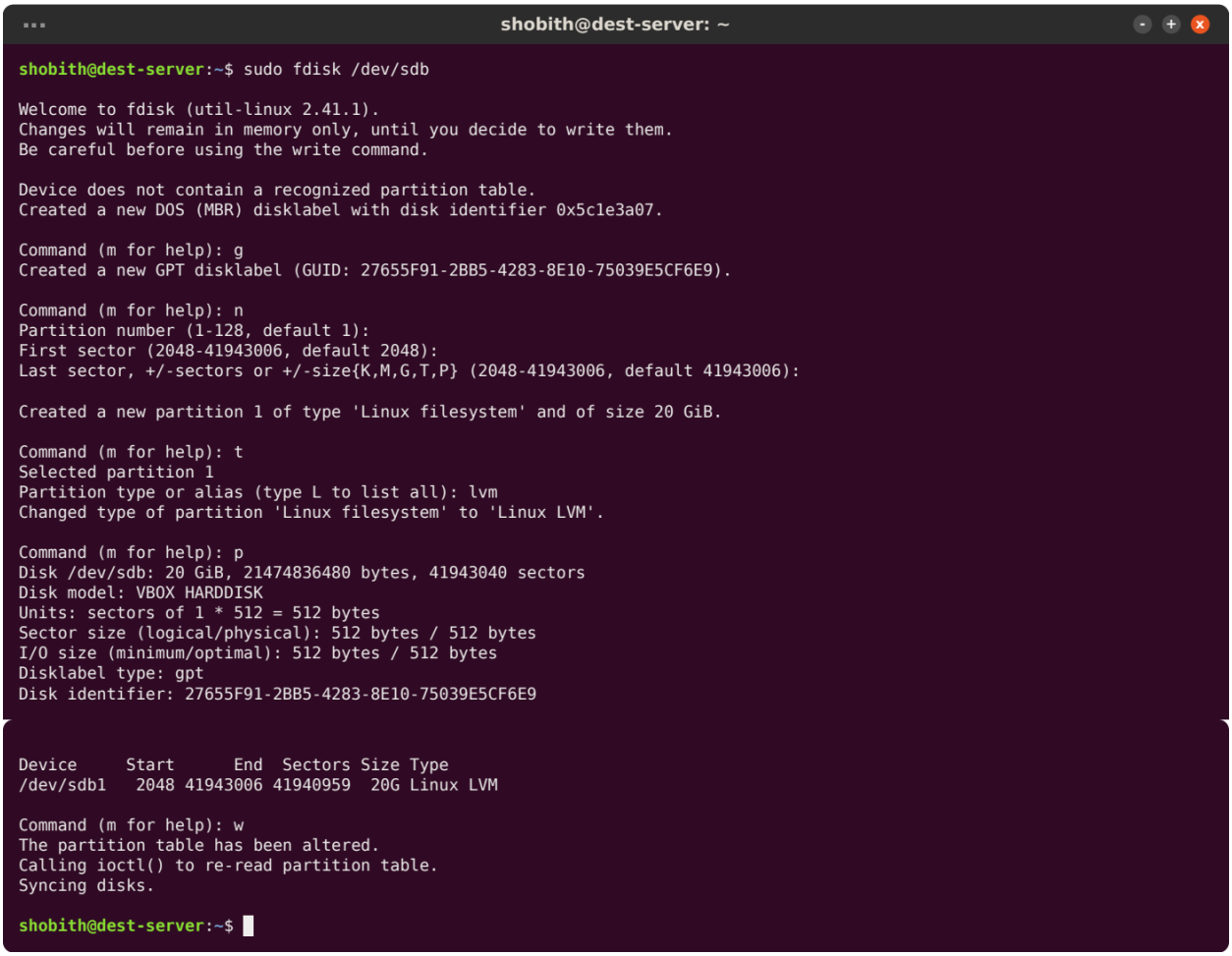
```
lsblk -f
sudo fdisk -l
```



The size, existing partitions and file systems are recorded, and it is confirmed that `/dev/sdb` is the new, empty virtual disk. The work does not proceed unless the target is certain.

3.6.3 Creating the GPT Partition

```
sudo fdisk /dev/sdb
```



Inside fdisk the following inputs are entered one at a time:

Table 3.5 gives the inputs entered in fdisk to create the GPT LVM partition.

Table 3.5: fdisk inputs for the GPT LVM partition

Input	Action
g	Create a new GPT partition table
n	Create a new partition
Enter	Accept default partition number
Enter	Accept default first sector
Enter	Accept default last sector to use the remaining space
t	Change the partition type
L	List available partition types if prompted
Select Linux LVM type	Mark the partition for LVM

Input	Action
p	Print the partition table
w	Write changes and exit

The exact type-selection prompts vary between fdisk versions. On current versions the Linux LVM type is identified as Linux LVM. After the table is written:

```
sudo partprobe /dev/sdb
lsblk /dev/sdb
```

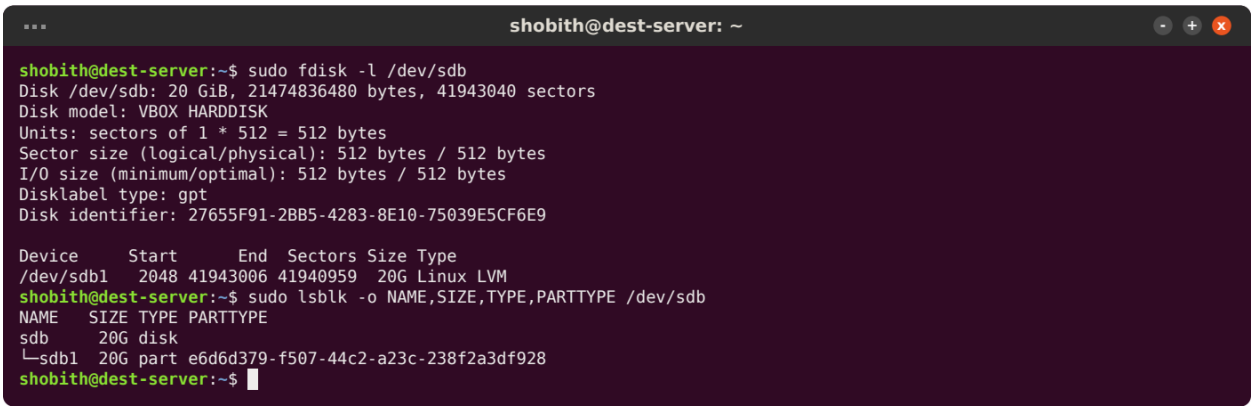
A partition such as `/dev/sdb1` should now be visible. This is the first major practical deliverable.

Figure 3.9: New GPT partition on /dev/sdb



3.6.4 Validating the Partition

```
sudo fdisk -l /dev/sdb
sudo lsblk -o NAME,SIZE,TYPE,PARTTYPE /dev/sdb
```



The checks confirm that the partition exists, its size is approximately as expected, it is the intended LVM partition, and the operating-system disk has not been modified. The partition now exists but is not yet a usable file system.

3.6.5 Configuring LVM

Physical volume. The partition is initialized as an LVM physical volume and verified:

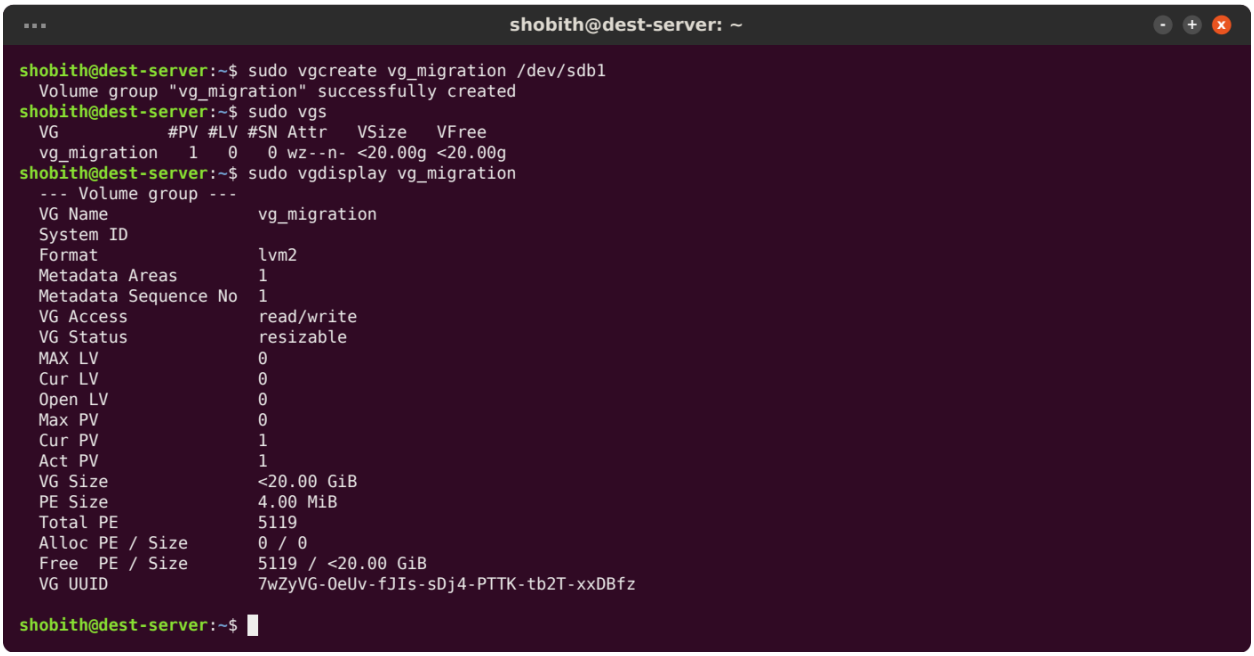
```
sudo pvcreate /dev/sdb1
sudo pvs
sudo pvdisplay /dev/sdb1
```



`pvcreate` writes LVM metadata to the target. It is used only on the verified, empty project partition.

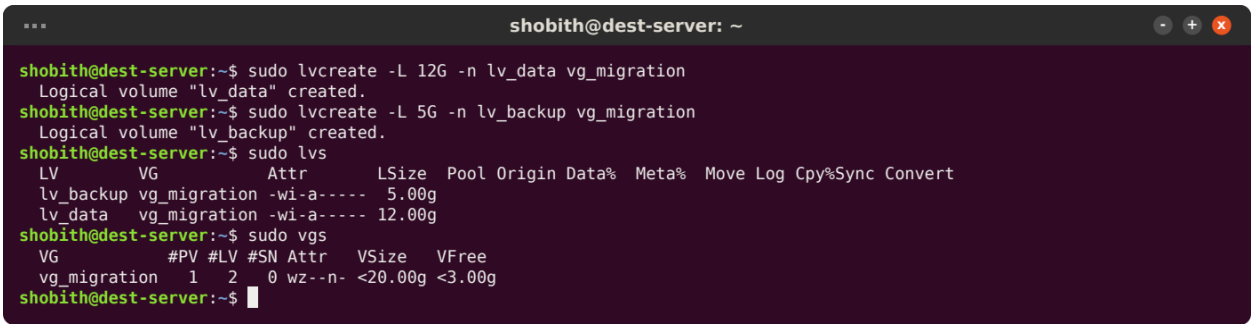
Volume group. A storage pool called`vg_migration` is created and verified. Its capacity is approximately that of the partition, minus LVM metadata and alignment overhead:

```
sudo vgcreate vg_migration /dev/sdb1
sudo vgs
sudo vgdisplay vg_migration
```



Logical volumes. The pool is divided into separate logical volumes for enterprise data and backups:

```
sudo lvcreate -L 12G -n lv_data vg_migration
sudo lvcreate -L 5G -n lv_backup vg_migration
sudo lvs
sudo vgs
```



This example needs at least 17 GB of allocatable capacity, so a 20 GB disk is generally sufficient, but the available space is checked first. The remaining space stays free in the volume group, so more storage can be allocated later without redesigning the partition layout.

Figure 3.10: LVM physical volume, volume group and logical volumes

```
shobith@dest-server: ~
shobith@dest-server:~$ sudo pvs
  PV         VG           Fmt  Attr PSize   PFree
  /dev/sdb1  vg_migration lvm2 a--  <20.00g <3.00g
shobith@dest-server:~$ sudo vgs
  VG           #PV #LV #SN Attr   VSize   VFree
  vg_migration   1   2   0 wz--n- <20.00g <3.00g
shobith@dest-server:~$ sudo lvs
  LV        VG           Attr       LSize  Pool Origin Data%  Meta%  Move Log Cpy%Sync Convert
  lv_backup vg_migration -wi-a-----  5.00g
  lv_data   vg_migration -wi-a----- 12.00g
shobith@dest-server:~$
```

3.6.6 Creating ext4 File Systems

```
sudo mkfs.ext4 /dev/vg_migration/lv_data
sudo mkfs.ext4 /dev/vg_migration/lv_backup
lsblk -f
sudo blkid
```

```
shobith@dest-server: ~
shobith@dest-server:~$ sudo mkfs.ext4 /dev/vg_migration/lv_data
mke2fs 1.47.3 (8-Jul-2025)
Creating filesystem with 3145728 4k blocks and 786432 inodes
Filesystem UUID: f3f49249-dc28-4f90-a5ae-c7978306d03b
Superblock backups stored on blocks:
        32768, 98304, 163840, 229376, 294912, 819200, 884736, 1605632, 2654208

Allocating group tables: done
Writing inode tables: done
Creating journal (16384 blocks): done
Writing superblocks and filesystem accounting information: done

shobith@dest-server:~$ sudo mkfs.ext4 /dev/vg_migration/lv_backup
mke2fs 1.47.3 (8-Jul-2025)
Creating filesystem with 1310720 4k blocks and 327680 inodes
Filesystem UUID: e5121482-3929-4d22-a255-accb1a466884
Superblock backups stored on blocks:
        32768, 98304, 163840, 229376, 294912, 819200, 884736

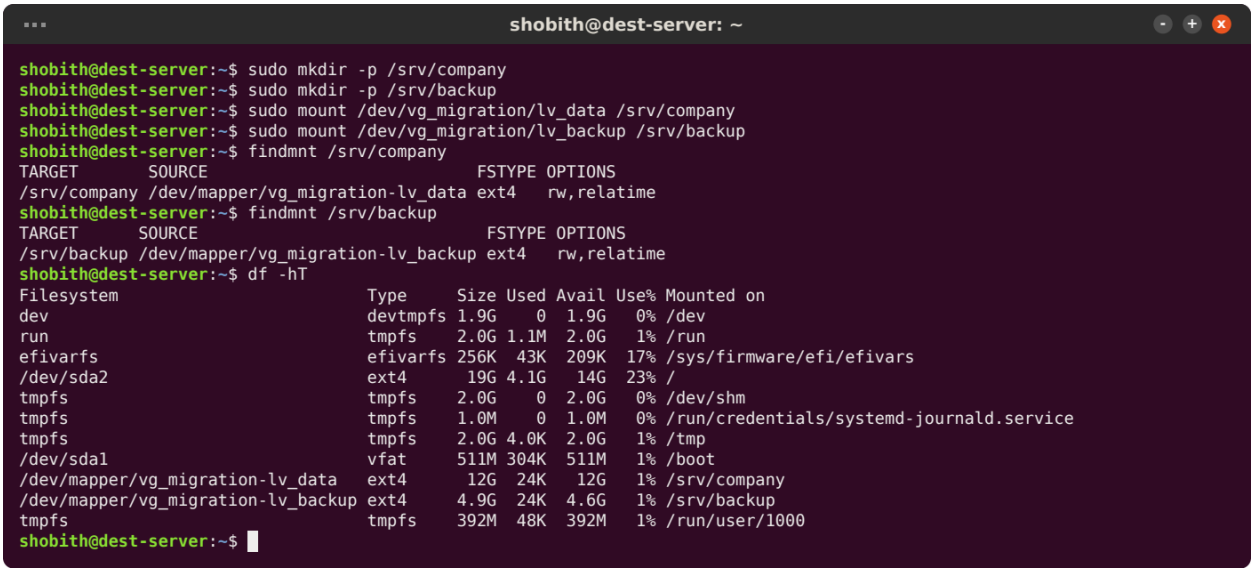
Allocating group tables: done
Writing inode tables: done
Creating journal (16384 blocks): done
Writing superblocks and filesystem accounting information: done

shobith@dest-server:~$ lsblk -f
NAME                      FSTYPE      FSVER    LABEL UUID                                   FSAVAIL FSUSE% MOUNTPOINTS
sda
├─sda1                    vfat        FAT32          7B3E-4C21                             510.2M     0% /boot
└─sda2                    ext4        1.0            f38b2ffc-80a4-4f5a-91c9-bc701e7ea419   13.8G    22% /
sdb
└─sdb1                    LVM2_member LVM2 001       YKJ1UX-XFWX-LCpa-Nfhs-0gCa-05RF-Runzqw
  ├─vg_migration-lv_data  ext4        1.0            f3f49249-dc28-4f90-a5ae-c7978306d03b
  └─vg_migration-lv_backup ext4       1.0            e5121482-3929-4d22-a255-accb1a466884
sr0
shobith@dest-server:~$ sudo blkid
/dev/mapper/vg_migration-lv_backup: UUID="e5121482-3929-4d22-a255-accb1a466884" BLOCK_SIZE="4096" TYPE="ext4"
/dev/mapper/vg_migration-lv_data: UUID="f3f49249-dc28-4f90-a5ae-c7978306d03b" BLOCK_SIZE="4096" TYPE="ext4"
/dev/sda2: UUID="f38b2ffc-80a4-4f5a-91c9-bc701e7ea419" BLOCK_SIZE="4096" TYPE="ext4" PARTUUID="1c7a52de-90b3-4e1d-a6f4-3d2
8b5c9e7a1"
/dev/sda1: UUID="7B3E-4C21" BLOCK_SIZE="512" TYPE="vfat" PARTUUID="9924eb8b-7575-4c2b-961f-8bfe9f93693f"
/dev/sdb1: UUID="YKJ1UX-XFWX-LCpa-Nfhs-0gCa-05RF-Runzqw" TYPE="LVM2_member" PARTUUID="6f00e7c4-34fd-42f6-8505-67c2975cef7a
"
shobith@dest-server:~$
```

ext4 is a mature Linux file system with journaling and broad support. `mkfs.ext4` is never run on a device containing data that is still needed, because formatting creates a new file system and destroys access to the previous file-system data.

3.6.7 Mounting the Logical Volumes

```
sudo mkdir -p /srv/company
sudo mkdir -p /srv/backup
sudo mount /dev/vg_migration/lv_data /srv/company
sudo mount /dev/vg_migration/lv_backup /srv/backup
findmnt /srv/company
findmnt /srv/backup
df -hT
```

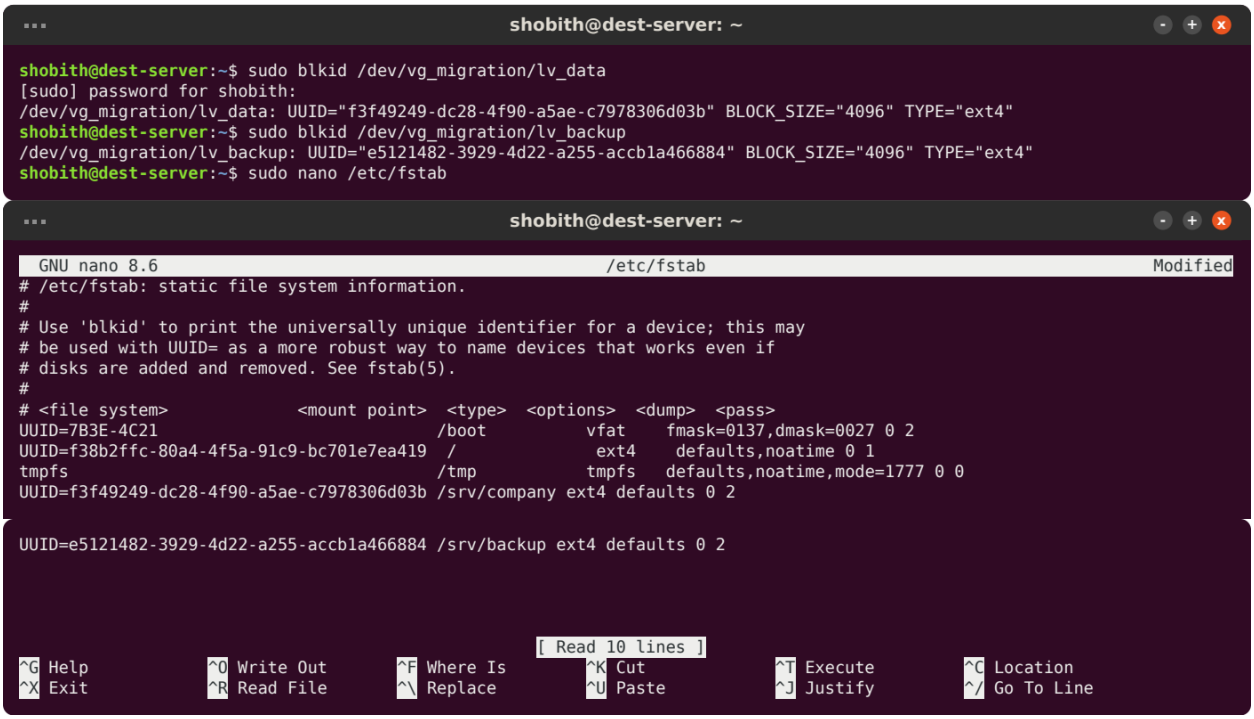


The destination's `/srv/company` is now backed by the new logical volume rather than by the root file system.

3.6.8 Persistent Mounts with /etc/fstab

A manually mounted file system may not be mounted after a reboot, so UUID-based entries are added to `/etc/fstab`. The UUIDs are retrieved with:

```
sudo blkid /dev/vg_migration/lv_data
sudo blkid /dev/vg_migration/lv_backup
sudo nano /etc/fstab
```



The entries are added in the following format, with the placeholders replaced by the real UUID values:

```
UUID=<DATA_UUID> /srv/company ext4 defaults 0 2
UUID=<BACKUP_UUID> /srv/backup ext4 defaults 0 2
```

The file is saved and tested before any reboot; if verification fails, `/etc/fstab` is corrected first:

```
sudo findmnt --verify --verbose
sudo mount -a
findmnt /srv/company
findmnt /srv/backup
```

Figure 3.11: Mounted logical volumes and file systems



3.6.9 Storage Allocation Summary

Table 3.6 summarises how the destination disk is allocated across the storage layers.

Table 3.6: Destination storage allocation

Layer	Name / Device	Size	Mount point
Disk	`/dev/sdb`	20 GB	-
Partition (GPT, Linux LVM)	`/dev/sdb1`	remaining disk space	-
Volume group	`vg_migration`	~ partition size	-
Logical volume (ext4)	`lv_data`	12 GB	`/srv/company`
Logical volume (ext4)	`lv_backup`	5 GB	`/srv/backup`

3.7 Migration, Backup and Automation

A migration is never the only copy of the data. Backup, migration, verification and automation are therefore implemented together.

3.7.1 Backup Strategy

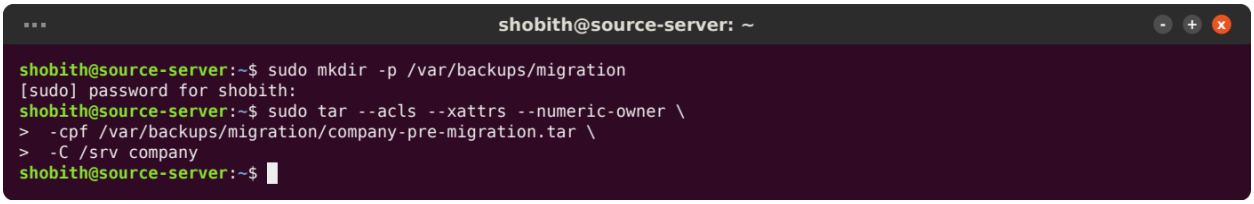
1. **Pre-migration backup:** a recoverable copy of the source data.
2. **Initial migration:** transfer of the main data set.
3. **Final synchronization:** transfer of changes made since the initial copy.
4. **Verification:** confirmation that the destination matches the source.
5. **Recovery test:** restoration of a deleted or modified test file from backup.

The backup logical volume is mounted at `/srv/backup` on the destination. To reduce the risk of losing both copies together, a separate backup is also kept on the source or on another virtual disk, because a backup on the same physical disk as the migrated data is not fully independent.

3.7.2 Source-side Backup

On the source VM a backup directory is created on a location with sufficient free space, and a timestamped archive is created. The archive preserves numeric ownership and includes ACL and extended-attribute metadata supported by the source system.

```
sudo mkdir -p /var/backups/migration
sudo tar --acls --xattrs --numeric-owner \
 -cpf /var/backups/migration/company-pre-migration.tar \
 -C /srv company
```



The archive is checked and a checksum is created and verified:

```
sudo tar -tf /var/backups/migration/company-pre-migration.tar
cd /var/backups/migration
sudo sha256sum company-pre-migration.tar |
 sudo tee company-pre-migration.tar.sha256
sudo sha256sum -c company-pre-migration.tar.sha256
```

A successful checksum check proves only that the archive currently matches its recorded checksum. It does not independently prove that every file inside the archive is correct, so the archive contents are also inspected and restoration is tested later. On a single-disk lab, `/var/backups` is not treated as an independent disaster-recovery backup; an additional copy is stored on the separate backup volume.

Figure 3.12: Pre-migration archive and checksum



3.7.3 Migration Using rsync

rsync transfers files efficiently and preserves file metadata when run with suitable privileges. The project uses `-aHAX --numeric-ids`:

Option	Meaning
`-a`	Archive mode: recursive copy, symlinks, permissions, timestamps and ownership preservation where permitted
`-H`	Preserve hard links

Option	Meaning
`-A`	Preserve access control lists
`-X`	Preserve extended attributes
`--numeric-ids`	Preserve numeric user and group IDs instead of translating names

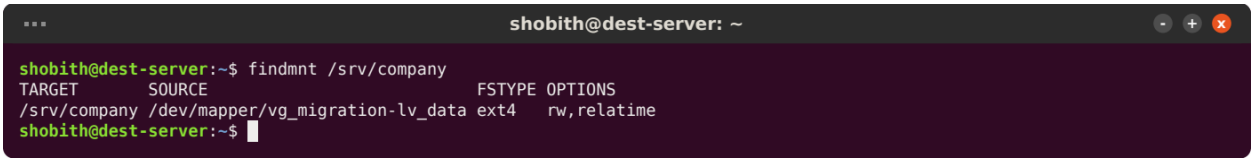
Table 3.7 explains the rsync options used for the migration.

Table 3.7: rsync options used for the migration

Option	Meaning
`--rsync-path="sudo rsync"`	Run the receiving rsync through the restricted sudo rule so that ownership and ACLs can be written
`-v`	Verbose output, used in the dry run and final synchronization to list the changes

rsync must run as root on the receiving side, or through a correctly configured privileged remote transfer, to preserve ownership. Both systems must support the relevant metadata features. Before the transfer, the destination mount is confirmed:

```
findmnt /srv/company
```



This command needs a destination account that can write the data with the necessary privileges. A normal SSH user generally cannot set arbitrary owners, groups, ACLs or privileged attributes. For a reliable production-style transfer, a restricted, authenticated root-capable rsync receiver, or rsync through an appropriately secured privileged remote command, is configured. Unrestricted root SSH login is not enabled for convenience.

In this lab the privileged remote command is provided by a restricted sudo rule on the destination that allows only `/usr/bin/rsync` to run as root without a password; rsync is told to use it with `--rsync-path`. The rule is created on the destination and syntax-checked:

```
echo 'your_destination_username ALL=(root) NOPASSWD: /usr/bin/rsync' |
 sudo tee /etc/sudoers.d/migration-rsync
sudo chmod 440 /etc/sudoers.d/migration-rsync
sudo visudo -c
```

```
shobith@dest-server:~$ echo 'shobith ALL=(root) NOPASSWD: /usr/bin/rsync' |
>  sudo tee /etc/sudoers.d/migration-rsync
[sudo] password for shobith:
shobith ALL=(root) NOPASSWD: /usr/bin/rsync
shobith@dest-server:~$ sudo chmod 440 /etc/sudoers.d/migration-rsync
shobith@dest-server:~$ sudo visudo -c
/etc/sudoers: parsed OK
/etc/sudoers.d/10-installer: parsed OK
/etc/sudoers.d/migration-rsync: parsed OK
shobith@dest-server:~$
```

Initial transfer. From the source VM:

```
sudo rsync -aHAX --numeric-ids \
 --info=progress2 \
 --rsync-path="sudo rsync" \
 /srv/company/ \
 your_destination_username@192.168.56.20:/srv/company/
```

```
shobith@source-server:~$ sudo rsync -aHAX --numeric-ids \
>  --info=progress2 \
>  --rsync-path="sudo rsync" \
>  /srv/company/ \
>  shobith@192.168.56.20:/srv/company/
[sudo] password for shobith:
The authenticity of host '192.168.56.20 (192.168.56.20)' can't be established.
ED25519 key fingerprint is SHA256:A5KZfPFitTAo+Rc8QlK1URn0xTZwBNxJGstDD8pm5pP.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '192.168.56.20' (ED25519) to the list of known hosts.
shobith@192.168.56.20's password:
           91 100%   29.62kB/s    0:00:00 (xfr#3, to-chk=0/7)
shobith@source-server:~$
```

Student-lab alternative. The archive is copied to the destination with SSH and extracted as root:

```
sudo scp /var/backups/migration/company-pre-migration.tar \
 your_destination_username@192.168.56.20:/tmp/
```

```
shobith@source-server:~$ sudo scp /var/backups/migration/company-pre-migration.tar \
>  shobith@192.168.56.20:/tmp/
shobith@192.168.56.20's password:
company-pre-migration.tar                                     100%   20KB   9.4MB/s   00:00
shobith@source-server:~$
```

On the destination, the archive is extracted into the mounted file system:

```
sudo tar --acls --xattrs --numeric-owner \
 -xpf /tmp/company-pre-migration.tar \
 -C /srv
```

```
shobith@dest-server:~$ sudo tar --acls --xattrs --numeric-owner \
>  -xpf /tmp/company-pre-migration.tar \
>  -C /srv
[sudo] password for shobith:
shobith@dest-server:~$
```

The archive contains the top-level company directory, so extracting it under `/srv` creates `/srv/company`. It is not extracted into an already populated directory without understanding the overwrite behaviour. The archive then serves as the rollback copy, and rsync can still perform the subsequent synchronization.

3.7.4 Final Synchronization

After the initial transfer, further changes to the source data are avoided during the final synchronization. On a real server the application is placed in maintenance mode or writes to the data set are stopped. A dry run first previews the changes, including destination files that `--delete` would remove:

```
sudo rsync -aHAX --numeric-ids \
 -v --delete --dry-run \
 --rsync-path="sudo rsync" \
 /srv/company/ \
 your_destination_username@192.168.56.20:/srv/company/
```



The output is reviewed carefully. `--delete` is not used if the destination contains unrelated data or if the paths have not been confirmed. Once the paths are verified and the destination is meant to be an exact mirror, the real final synchronization is performed with the same secured privileged rsync setup:

```
sudo rsync -aHAX --numeric-ids \
 -v --delete --info=progress2 \
 --rsync-path="sudo rsync" \
 /srv/company/ \
 your_destination_username@192.168.56.20:/srv/company/
```

If the source data changes while rsync is running, the result may not represent a single consistent point in time, so writes are frozen for the final pass.

Figure 3.13: rsync transfer and final synchronization



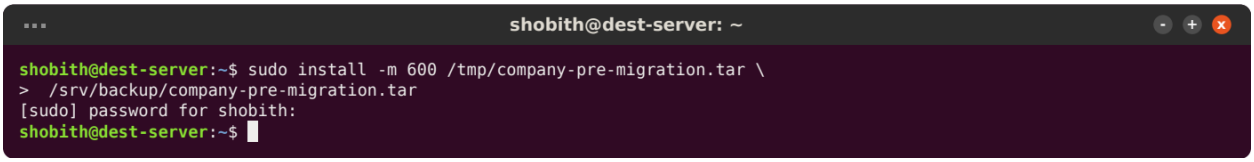
3.7.5 Copying the Backup to the Destination Backup Volume

```
sudo scp /var/backups/migration/company-pre-migration.tar \
 your_destination_username@192.168.56.20:/tmp/
```



On the destination, the archive is copied into the backup logical volume:

```
sudo install -m 600 /tmp/company-pre-migration.tar \
 /srv/backup/company-pre-migration.tar
```



The checksum file is copied as well and verified against the archive. The SSH transfer is encrypted, but the checksum remains useful for integrity validation.

3.7.6 Recovery Procedure

The recovery test is performed only after the backup has been verified and only on sample data. On the destination, a copy of the file is kept, data loss is simulated, and the loss is confirmed:

```
sudo cp /srv/company/development/config.txt \
 /tmp/config.txt.before-test
sudo rm /srv/company/development/config.txt
sudo ls -l /srv/company/development/config.txt
```

```
shobith@dest-server:~$ sudo cp /srv/company/development/config.txt \
>  /tmp/config.txt.before-test
[sudo] password for shobith:
shobith@dest-server:~$ sudo rm /srv/company/development/config.txt
shobith@dest-server:~$ sudo ls -l /srv/company/development/config.txt
ls: cannot access '/srv/company/development/config.txt': No such file or directory
shobith@dest-server:~$
```

The archive is extracted into a separate temporary directory first, so that the live data is not overwritten:

```
sudo mkdir -p /tmp/migration-restore
sudo tar --acls --xattrs --numeric-owner \
 -xpf /srv/backup/company-pre-migration.tar \
 -C /tmp/migration-restore
```

```
shobith@dest-server:~$ sudo mkdir -p /tmp/migration-restore
shobith@dest-server:~$ sudo tar --acls --xattrs --numeric-owner \
>  -xpf /srv/backup/company-pre-migration.tar \
>  -C /tmp/migration-restore
shobith@dest-server:~$
```

The restored file is located at the path below. Its contents and metadata are verified before it is copied back:

```
/tmp/migration-restore/company/development/config.txt
sudo sha256sum \
 /tmp/migration-restore/company/development/config.txt
sudo cp -a \
 /tmp/migration-restore/company/development/config.txt \
 /srv/company/development/config.txt
sudo stat /srv/company/development/config.txt
sudo sha256sum /srv/company/development/config.txt
```

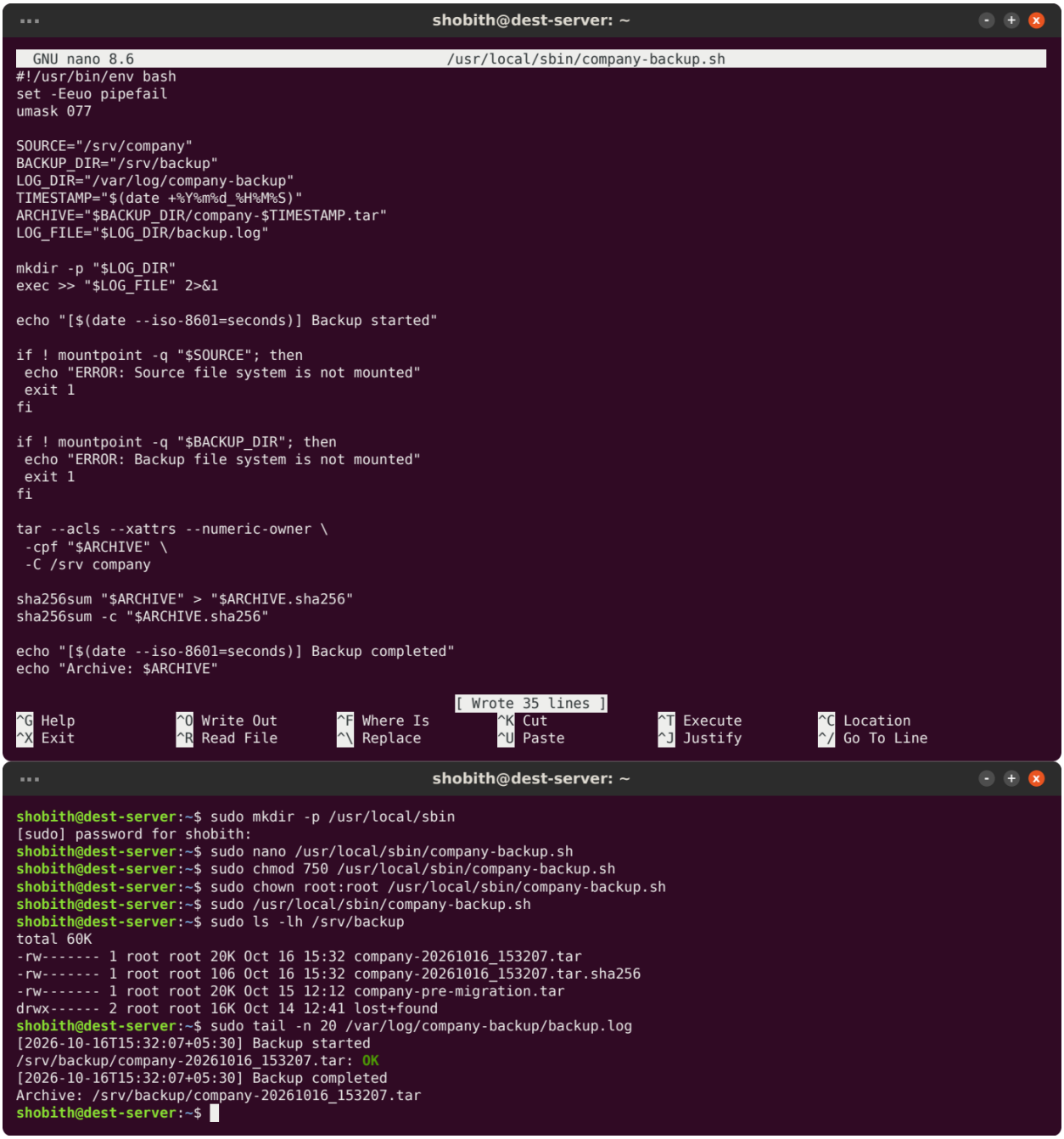
Figure 3.14: Recovery test: file deleted and restored from backup

```
shobith@dest-server:~$ sudo sha256sum \
>  /tmp/migration-restore/company/development/config.txt
cf3debbc84344f997f8dc83dc37b2dbf458ceb205246cb9eefe6045bec2c720b  /tmp/migration-restore/company/development/config.txt
shobith@dest-server:~$ sudo cp -a \
>  /tmp/migration-restore/company/development/config.txt \
>  /srv/company/development/config.txt
shobith@dest-server:~$ sudo stat /srv/company/development/config.txt
  File: /srv/company/development/config.txt
  Size: 33              Blocks: 8          IO Block: 4096   regular file
Device: 253,0   Inode: 14          Links: 1
Access: (0640/-rw-r-----)  Uid: ( 1001/ devuser)   Gid: ( 1001/developers)
Access: 2026-10-16 14:48:12.304117256 +0530
Modify: 2026-10-13 15:22:41.612903448 +0530
Change: 2026-10-16 14:48:12.304117256 +0530
 Birth: 2026-10-16 14:48:12.304117256 +0530
shobith@dest-server:~$ sudo sha256sum /srv/company/development/config.txt
cf3debbc84344f997f8dc83dc37b2dbf458ceb205246cb9eefe6045bec2c720b  /srv/company/development/config.txt
shobith@dest-server:~$
```

3.7.7 Shell Script Automation

A Bash script on the destination server makes the project more than a one-time migration. It creates a timestamped archive of `/srv/company` on the backup volume, writes a checksum and verifies it, and logs the result. It first checks that both file systems are mounted, which prevents an important failure mode: accidentally saving a backup to the root file system because the backup volume was not mounted. The complete script is in Appendix C.1.

```
sudo mkdir -p /usr/local/sbin
sudo nano /usr/local/sbin/company-backup.sh
sudo chmod 750 /usr/local/sbin/company-backup.sh
sudo chown root:root /usr/local/sbin/company-backup.sh
sudo /usr/local/sbin/company-backup.sh
sudo ls -lh /srv/backup
sudo tail -n 20 /var/log/company-backup/backup.log
```

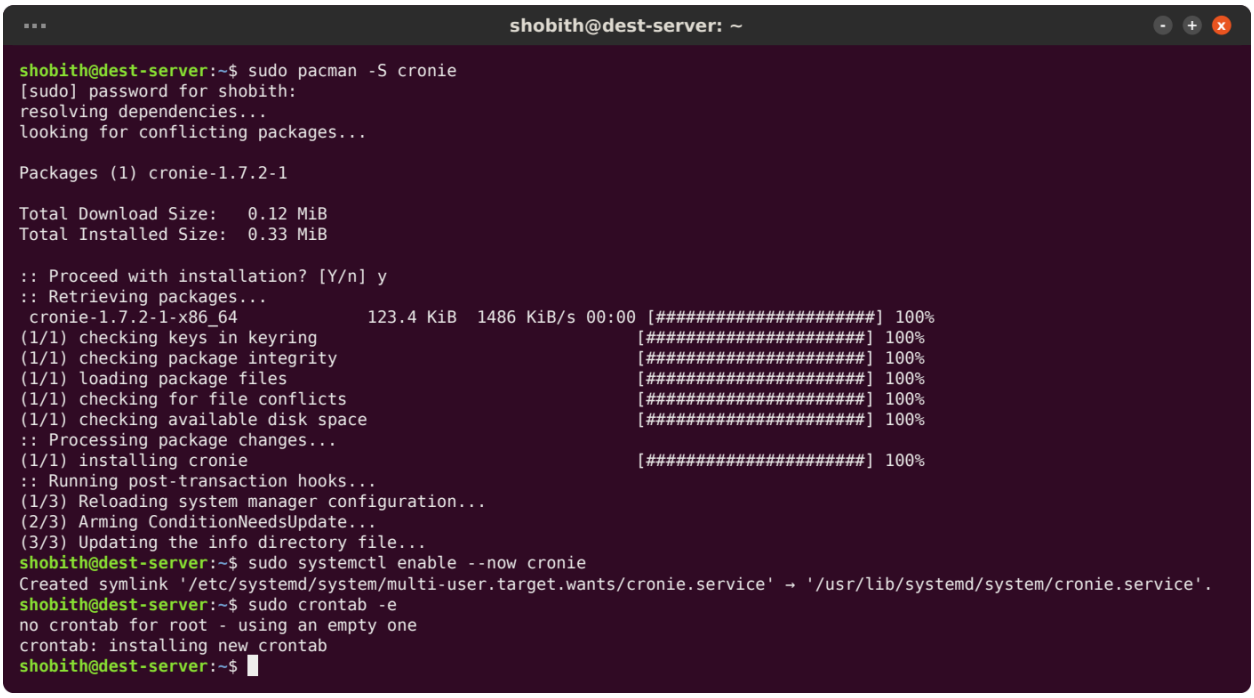


Operational note: the script creates timestamped archives and does not yet delete old ones. A tested retention policy should be added before running it indefinitely, free disk space should be monitored, and an independent copy of important backups should be kept outside the destination disk.

3.7.8 Cron Configuration

Cron is installed and enabled, and the script is scheduled in root's crontab to run a daily backup at 2:00 AM:

```
sudo pacman -S cronie
sudo systemctl enable --now cronie
sudo crontab -e
0 2 * * * /usr/local/sbin/company-backup.sh
```

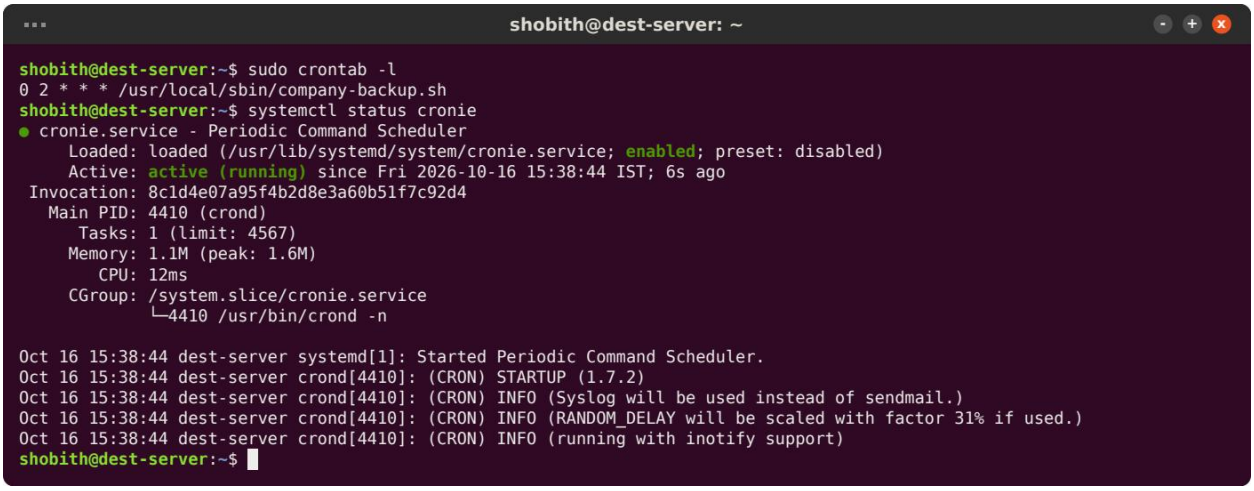


The scheduled job and the service are checked with:

```
sudo crontab -l
systemctl status cronie
```

Cron uses the server's configured local time zone. To test the automation immediately, the script is run manually and its log is inspected instead of waiting until 2:00 AM.

Figure 3.15: Backup script execution, log and Cron entry



3.7.9 Log Generation

The backup script records its activity in `/var/log/company-backup/backup.log`: start time, mount checks, checksum verification, completion time and the archive name. The log is reviewed with `tail -n 20 /var/log/company-backup/backup.log`.

3.7.10 Benefits of Automation

Automation reduces manual administrative effort by running backups at scheduled intervals, produces consistent timestamped archives with checksums, and records each run for later review. Combined with the verified backup and the tested recovery procedure, it gives the destination server a repeatable data-protection routine.

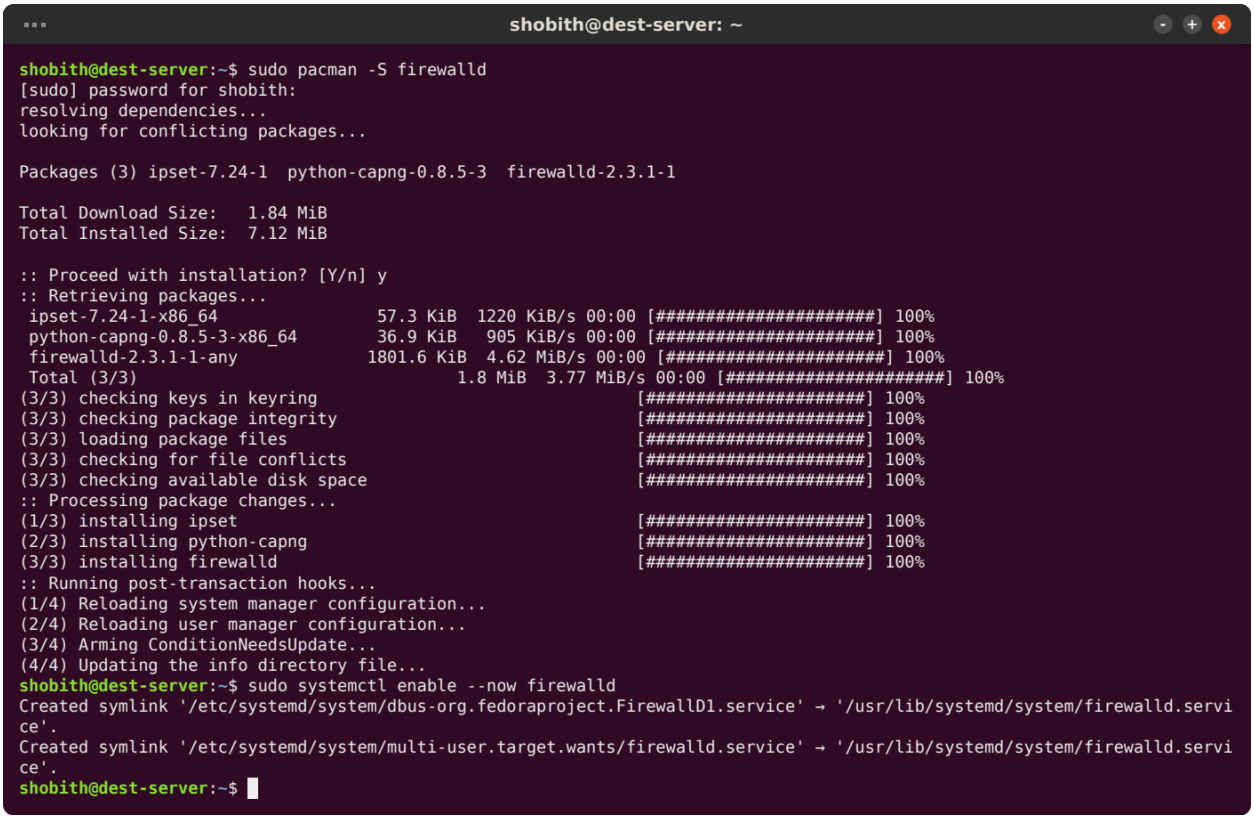
3.8 Security and Access Control

Security and access control protect the migrated data, restrict who can reach it, and reduce the risk of unauthorized modification. The project covers a firewall, SSH access, file permissions and ACLs, and least privilege.

3.8.1 Firewall Configuration

firewalld is used as the example firewall. A firewall rule set is not enabled until it has been confirmed that it will not lock out SSH. On both VMs:

```
sudo pacman -S firewalld
sudo systemctl enable --now firewalld
```

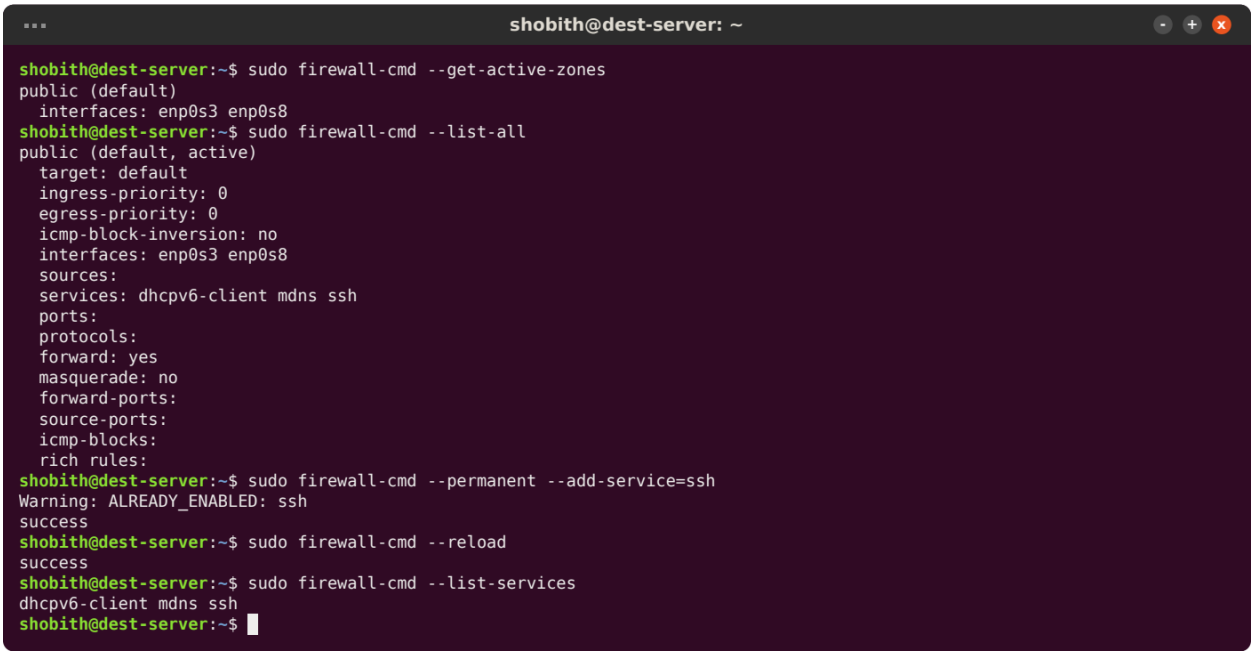


The active zone is inspected, and SSH is allowed in the appropriate zone:

```
sudo firewall-cmd --get-active-zones
sudo firewall-cmd --list-all
sudo firewall-cmd --permanent --add-service=ssh
sudo firewall-cmd --reload
sudo firewall-cmd --list-services
```

For a stricter deployment, SSH access is limited to the private migration network or specific source IP addresses, only the services the server actually needs are allowed, and SSH is not exposed to the public internet unnecessarily.

Figure 3.16: Firewall status and allowed services



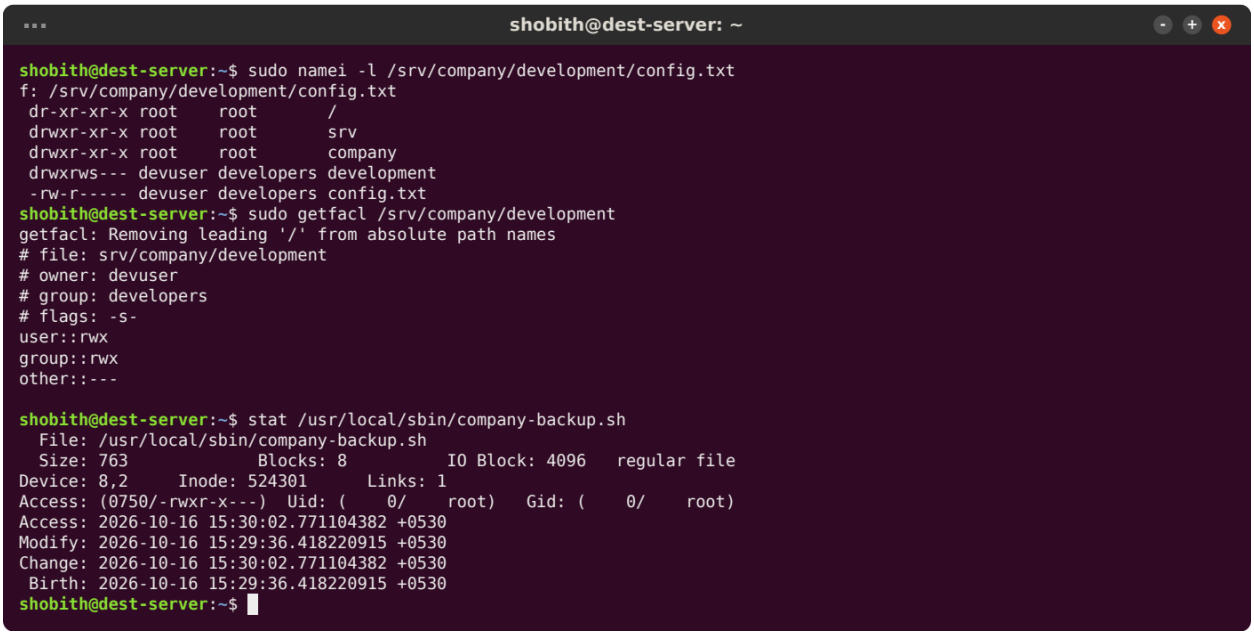
3.8.2 SSH Access

SSH provides the encrypted channel for the migration. Access is limited to authorized accounts, unrestricted root SSH login is not enabled for convenience, and the service status is verified with `systemctl is-active sshd`.

3.8.3 Least-privilege Permissions

Permissions and ACLs are inspected, and the following conditions are checked:

```
sudo namei -l /srv/company/development/config.txt
sudo getfacl /srv/company/development
stat /usr/local/sbin/company-backup.sh
```



- Only the intended users can access restricted team data.
- The backup script is writable only by root.
- The backup archive is not world-readable.
- The destination's data directories are owned by the intended users and groups.
- SSH access is limited to authorized accounts.
- Files and directories are never made world-writable simply to resolve permission errors.

3.8.4 Troubleshooting Common Issues

Table 3.8 lists common problems, their likely causes and the checks to run.

Table 3.8: Troubleshooting guide

Problem	Likely cause	What to check
`pvcreate` cannot access a device	Wrong partition or device busy	`lsblk`, `pvs`, `fdisk -l`
Logical volume cannot be mounted	Missing file system or wrong device	`lvs`, `blkid`, `lsblk -f`
Mount disappears after reboot	Missing or incorrect `/etc/fstab` entry	`findmnt --verify`, UUIDs
SSH connection refused	SSH daemon not running or firewall issue	`systemctl status sshd`, firewall rules
rsync reports permission errors	Receiver lacks required privileges	SSH account, destination ownership, rsync receiver

Problem	Likely cause	What to check
Files have the wrong owner	UID/GID mismatch or non-privileged transfer	`id`, `stat`, `getent`
ACLs or xattrs are missing	Transfer did not preserve metadata or destination lacks support	`getfacl`, `getfattr`, rsync options
Backup script fails	File system not mounted or disk full	Script log, `findmnt`, `df -h`
Destination data is incomplete	Source changed during migration or transfer was interrupted	Rerun final sync, compare manifests

3.8.5 Security Benefits

- **User and group management:** organizes access to shared project files.
- **Numeric identity preservation:** prevents files from silently changing owner during migration.
- **File permissions and ACLs:** limit reading, writing and execution according to ownership and mode.
- **Firewall and SSH controls:** restrict network access to what the migration needs.
- **Protected backup script and archives:** root-owned script and non-world-readable archives.
- **Logging:** supports troubleshooting and review of automated activity.

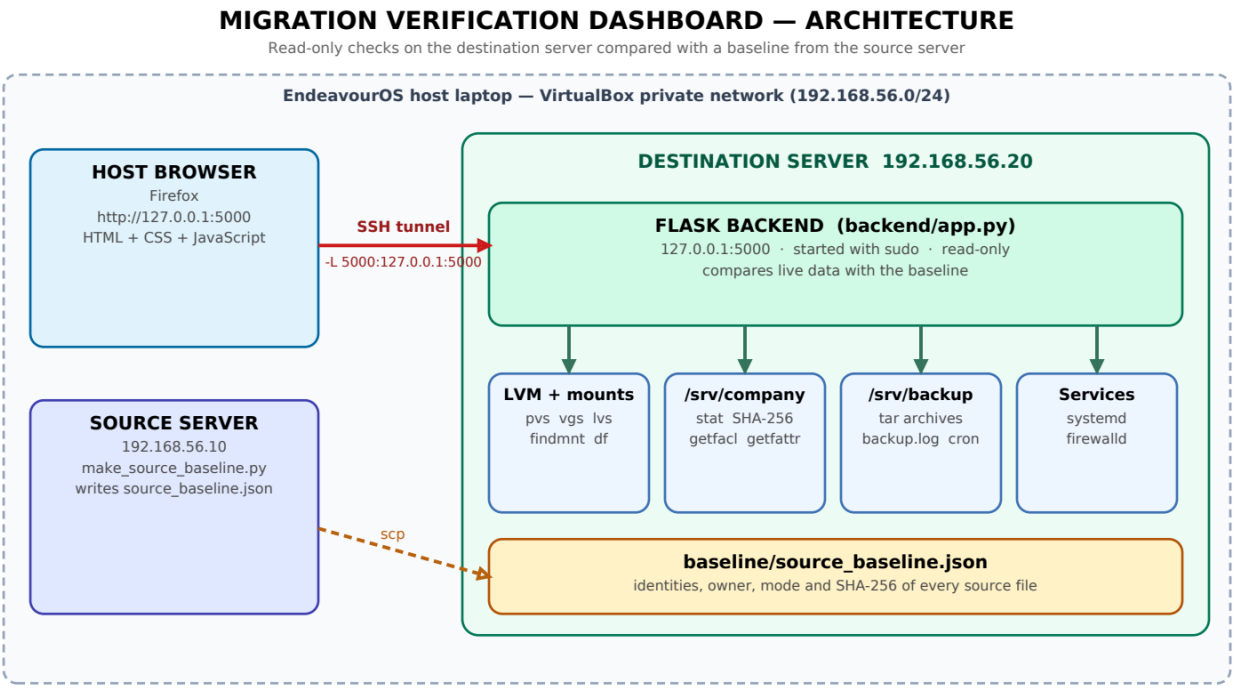
3.9 Migration Verification Dashboard

Sections 3.2 to 3.8 verify each stage of the migration with separate commands and screenshots. To show the final outcome of the project in one place, a web-based **Migration Verification Dashboard** was added. It runs on the destination server, reads the live system, compares it with a baseline recorded on the source server, and shows every check as PASS or FAIL on one page. The dashboard is read-only: it does not change data, users, volumes, backups or services. It complements the evidence in Sections 3.2 to 3.8 and Chapter 4 and does not replace it.

3.9.1 Design and Architecture

The dashboard has two parts. A small Python **Flask** backend runs on the destination server and runs the same read-only Linux commands used earlier in this report (lvs, findmnt, df, stat, sha256, systemctl, firewall-cmd and others). A **browser front end** written in HTML, CSS and JavaScript calls the backend through HTTP API routes and draws the results. Figure 3.17 shows how the components are connected.

Figure 3.17: Architecture of the Migration Verification Dashboard



The data flow is:

1. **Baseline on the source.** The script `make_source_baseline.py` records the numeric UID/GID, mode and SHA-256 checksum of every item in `/srv/company` together with the numeric IDs of the project users and groups. The result is saved as `source_baseline.json` and copied to the destination with scp.

2. **Backend on the destination.** `backend/app.py` is started with sudo and listens only on `127.0.0.1:5000`. Root is needed to read the 2770 data directories, the root-only backup archives, the LVM metadata and the cron table.

3. **Collection and comparison.** When the page is opened it requests `/api/summary`. The backend collects the live storage, identity, data, backup and security information, compares it with the baseline and returns JSON.

4. **Display.** JavaScript draws status cards, tables and an acceptance checklist, and refreshes the values every 30 seconds. The Generate Report button saves the checklist as `reports/migration_report.txt`.

5. **Access from the host.** The EndeavourOS host reaches the dashboard through an SSH tunnel, so no additional firewall port is opened on the destination server.

3.9.2 Tools and Components

Table 3.9 describes the components of the dashboard and the role of each.

Table 3.9: Components of the dashboard

Component	Purpose
`Python 3 and Flask (python-flask)`	Backend web framework; serves the page and the JSON API routes.
`HTML, CSS and JavaScript`	Front end in a single file; uses fetch() to call the API and shows PASS/FAIL results.
`make_source_baseline.py`	Records the source identities, ownership, modes and SHA-256 checksums (run on the source).
`source_baseline.json`	The reference the destination is compared against.
`lvs, pvs, vgs, lsblk, findmnt, df`	Read the storage layers, mounts and capacity of the destination.
`getfacl, getfattr, runuser`	Check ACLs, extended attributes and the allowed/denied access tests.
`systemctl, firewall-cmd, crontab`	Check sshd, cronie and firewalld, the allowed firewall services and the cron entry.
`SSH port forwarding`	Gives the host browser access to 127.0.0.1:5000 on the destination.

3.9.3 Project Folder and File Locations

The dashboard is stored in a single folder, `~/migration-dashboard`, in the home directory of the administrative user (/home/shobith in this lab). The same folder layout is used on the destination (runs the dashboard) and on the source (only the baseline script is needed there).

```
migration-dashboard/
├── backend/
│   ├── app.py Flask backend (destination)
│   └── templates/
│   └── index.html Dashboard page (destination)
├── baseline/
│   └── source_baseline.json Created on the source, copied to the destination
├── scripts/
│   └── make_source_baseline.py Run on the source
└── reports/
    └── migration_report.txt Created by the Generate Report button
```

Table 3.10 shows the dashboard files, where each is stored and where it runs.

Table 3.10: Dashboard files and where they are stored

File	Location	Server
app.py	/home/shobith/migration-dashboard/backend/app.py	Destination
index.html	/home/shobith/migration-dashboard/backend/templates/i ndex.html	Destination
make_source_basel i ne.py	/home/shobith/migration-dashboard/scripts/make_source _baseline.py	Source
source_baseline.j s on	/home/shobith/migration-dashboard/baseline/source_bas eline.json	Source, then copied
migration_report. t xt	/home/shobith/migration-dashboard/reports/migration_r eport.txt	Destination

The full source code of all files is given in Appendix E.

3.9.4 Installation and Setup

On the source server, the folder is created, the script from Appendix E.4 is saved, and the baseline is recorded after the data is final (after the last synchronization of Section 3.7.4). The script runs as root because the data directories are readable only by their groups:

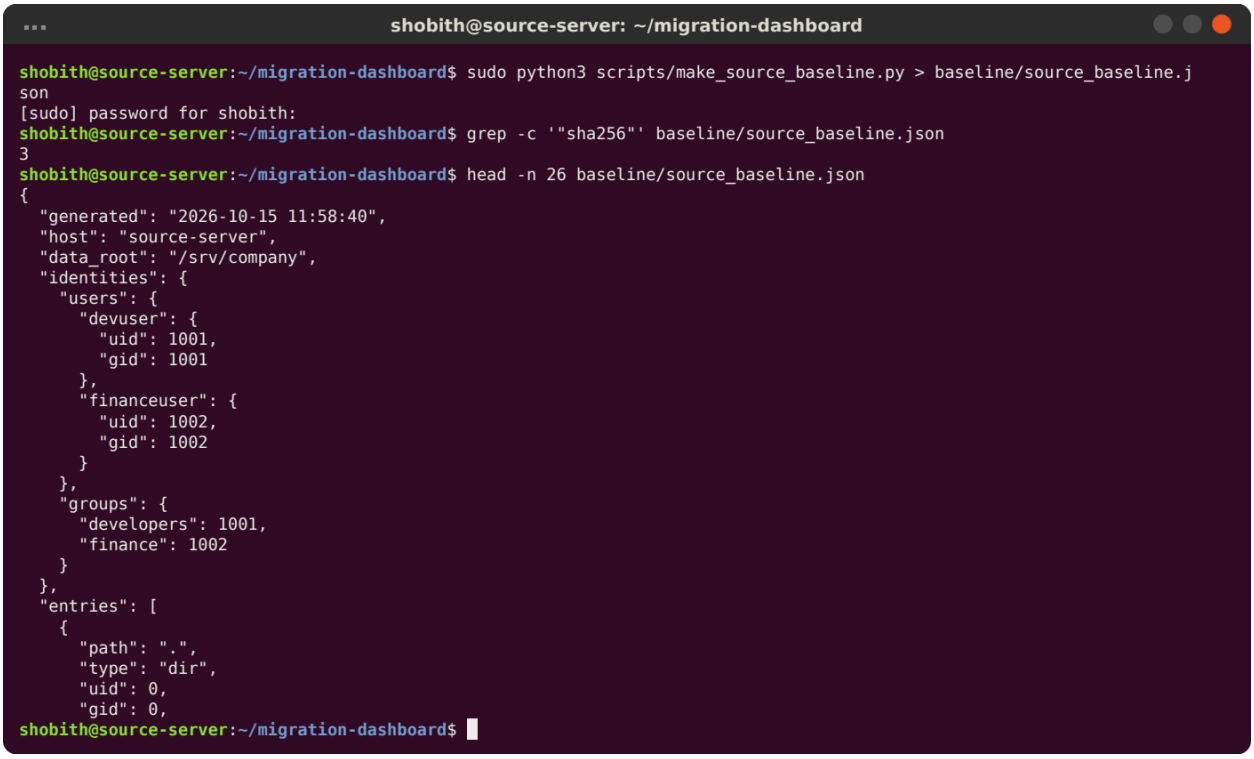
```
mkdir -p ~/migration-dashboard/scripts ~/migration-dashboard/baseline
nano ~/migration-dashboard/scripts/make_source_baseline.py
cd ~/migration-dashboard
sudo python3 scripts/make_source_baseline.py > baseline/source_baseline.json
head -n 26 baseline/source_baseline.json
scp baseline/source_baseline.json shobith@192.168.56.20:~/migration-dashboard/baseline/
```

On the destination server, the folders are created first (so the scp above has a target), Flask is installed with pacman, and the two dashboard files from Appendix E.2 and E.3 are saved:

```
mkdir -p ~/migration-dashboard/backend/templates ~/migration-dashboard/baseline
mkdir -p ~/migration-dashboard/scripts ~/migration-dashboard/reports
sudo pacman -S python-flask
nano ~/migration-dashboard/backend/app.py
nano ~/migration-dashboard/backend/templates/index.html
cd ~/migration-dashboard
python -m py_compile backend/app.py
ls baseline/
```

Flask is installed from the Arch repository (`python-flask`) instead of pip because the dashboard is started with sudo, which uses the system Python, and because Arch protects the system Python from pip installs. `py_compile` prints nothing when the file has no syntax errors, and `ls baseline/` must show `source_baseline.json`.

Figure 3.18: Baseline recorded on the source server



3.9.5 How the Dashboard Checks the Migration

The dashboard performs 23 checks in six groups. A check passes only when the live value equals the expected value or the recorded baseline; the overall status is VERIFIED only when all 23 pass, otherwise it is ATTENTION.

Table 3.11 lists the checks performed by the dashboard.

Table 3.11: Checks performed by the dashboard

Group	Checks (count)	How the check is made
Storage	GPT table, LVM partition type, PV and VG, lv_data 12 GB, lv_backup 5 GB, both mounts backed by the LVs, UUID entries in /etc/fstab (8)	lsblk, pvs, vgs, lvs, findmnt, df, findmnt --verify and /etc/fstab
Identities	UIDs and GIDs of devuser, financeuser, developers and finance (1)	Python pwd and grp compared with the baseline
Migration	Inventory, file and directory counts, ownership, modes including setgid, SHA-256 of every file, ACLs and extended attributes (6)	os.walk, lstat, SHA-256 of each file, getfacl, getfattr
Access	devuser and financeuser can read their data; financeuser cannot write to development; devuser cannot read finance (1)	runuser -u USER -- test -r / -w (no file is created)

Group	Checks (count)	How the check is made
Backup	Pre-migration archive matches the source tree, latest archive passes its .sha256, archives not world-readable, backup script root:root 0750, cron entry (5)	tarfile listing, SHA-256, stat, crontab -l
Security	sshd, cronie and firewalld active; firewall allows ssh (2)	systemctl is-active, firewall-cmd --list-services

The backend exposes the following routes. Every route only reads data, with the exception of `/api/report`, which also writes the text report file.

Table 3.12 lists the routes exposed by the dashboard backend.

Table 3.12: Dashboard routes

Route	Purpose	Result
/	Loads the dashboard page	HTML page
/api/summary	Runs all 23 checks and returns the full result	JSON
/api/storage	Storage layers, mounts and usage	JSON
/api/identities	Source and destination UID/GID comparison	JSON
/api/migration	File comparison, ACL/xattr count and access tests	JSON
/api/backup	Backup archives, content check, checksum and cron entry	JSON
/api/security	Service states and firewall services	JSON
/api/logs	Last lines of /var/log/company-backup/backup.log	JSON
/api/report	Builds and saves reports/migration_report.txt	JSON

3.9.6 Running the Dashboard

On the destination server the backend is started from the project folder. It refuses to start without root privileges and listens on the local address only:

```
cd ~/migration-dashboard
sudo python backend/app.py
```

Flask prints the address it is serving and then logs every request, as shown in Figure 3.19. The terminal must stay open while the dashboard is used; Ctrl+C stops the server. On the EndeavourOS host a second terminal opens the SSH tunnel, and the dashboard is then opened in Firefox:

```
ssh -N -L 5000:127.0.0.1:5000 shobith@192.168.56.20
# then open http://127.0.0.1:5000 in Firefox on the host
```

Figure 3.19: Starting the dashboard backend on the destination server



The backend binds to 127.0.0.1 and not to 0.0.0.0, so it is not reachable from the network without the tunnel, and no firewall rule is added for port 5000. Flask's built-in server is a development server and is suitable for this laboratory demonstration, not for production use. Common start-up problems are listed in Table 3.13.

Table 3.13 lists common dashboard problems, their causes and the remedies.

Table 3.13: Dashboard troubleshooting

Message or symptom	Cause	Fix
Run with sudo: the dashboard must read root-only data...	Started without root	Start with sudo python backend/app.py
ModuleNotFoundError: No module named 'flask'	Flask not installed for the system Python	sudo pacman -S python-flask
Address already in use	A copy of the dashboard is already running	Use the running copy, or stop it with Ctrl+C
Internal Server Error; FileNotFoundError ... source_baseline.json	Baseline not copied to the destination	Run the baseline script on the source and scp the file to baseline/
Checks show FAIL for backup	No archive yet, or backup script not run	Run sudo /usr/local/sbin/company-backup.sh once
Browser cannot connect to 127.0.0.1:5000	SSH tunnel not running	Start the ssh -N -L command on the host

4. Results and Testing

4.1 Implementation Results

The Linux Server Migration project is implemented using Linux administration commands, LVM-based storage, SSH, tar and rsync, a Bash backup script, Cron scheduling and firewalld. The implementation covers partitioning, LVM and file-system configuration, user and group management, permissions and ownership, migration, verification, backup and recovery, automation and security.

4.1.1 Screenshots and Command Outputs

Terminal screenshots provide the evidence for each stage. The documentation lead collects them throughout the project.

Table 4.1 lists the commands whose output is captured as evidence at each stage.

Table 4.1: Evidence to be captured

Stage	Commands to capture	Server
Disk identification	`lsblk -f`, `fdisk -l`	Destination
Partition	`fdisk -l /dev/sdb`, `lsblk -o NAME,SIZE,TYPE,PARTTYPE /dev/sdb`	Destination
LVM	`pvs`, `vgs`, `lvs`	Destination
File systems and mounts	`lsblk -f`, `blkid`, `df -hT`, `findmnt`, `findmnt --verify`	Destination
Source identities and data	`id`, `getent group`, `ls -lR /srv/company`	Source
Destination identities	`id`, `getent group`	Destination
Backup	`tar -tf`, `sha256sum -c`	Source / Destination
Migration	`rsync` (dry run and final synchronization)	Source
Verification	`find`, `stat`, `getfacl`, `getfattr`, `sha256sum`	Both
Recovery	`rm`, `tar -xpf`, `cp -a`, `stat`, `sha256sum`	Destination
Automation	`company-backup.sh`, `backup.log`, `crontab -l`	Destination
Security	`firewall-cmd`, `namei -l`, `getfacl`, `stat`	Destination

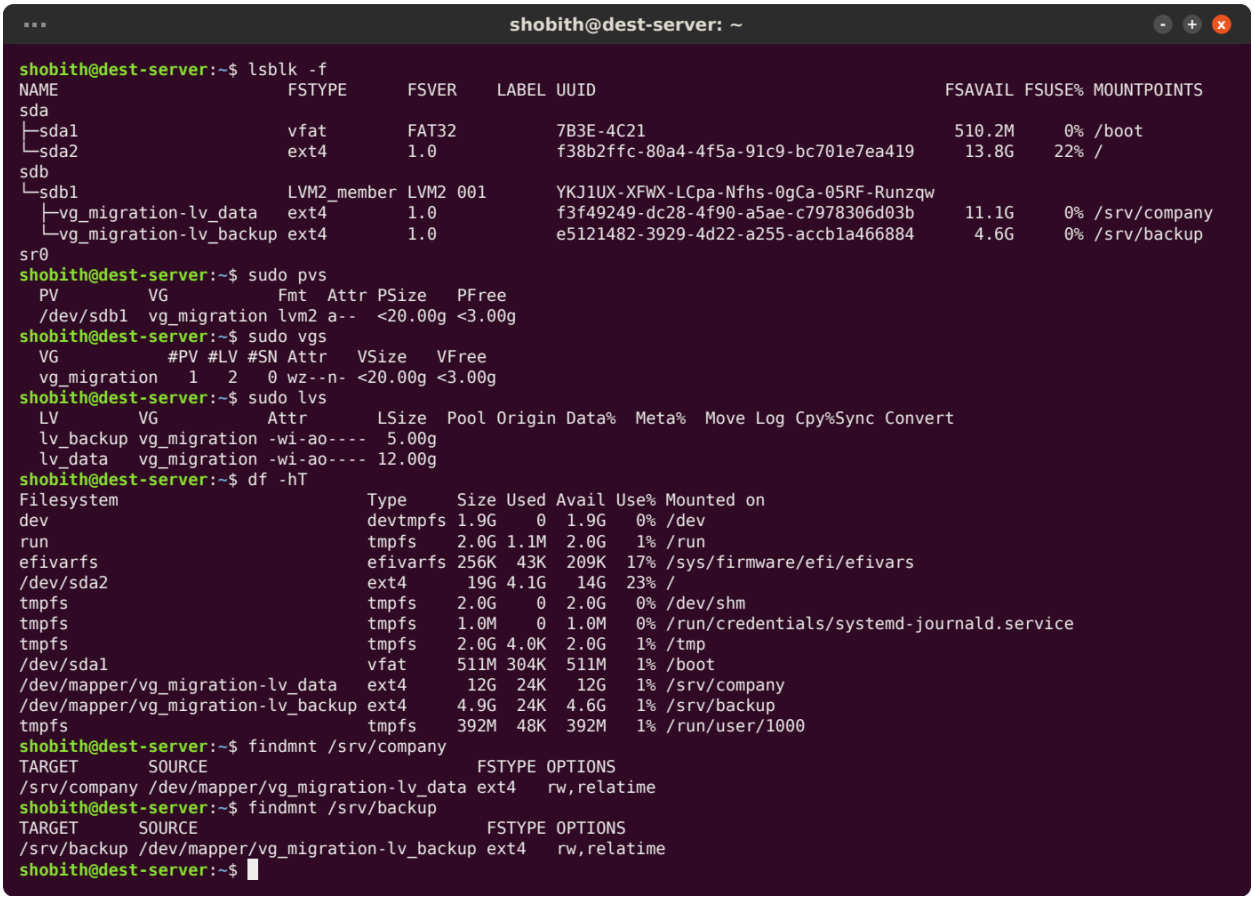
4.1.2 Storage Results

The final storage checks on the destination demonstrate partitioning, LVM, file-system creation and mounting:

```
lsblk -f
sudo pvs
sudo vgs
sudo lvs
df -hT
findmnt /srv/company
findmnt /srv/backup
```

The expected result is a GPT partition `/dev/sdb1`, a physical volume on it, the volume group *vg_migration*, the logical volumes *lv_data* (12 GB) and *lv_backup* (5 GB), ext4 file systems on both, and mount points `/srv/company` and`/srv/backup` backed by the logical volumes rather than the root file system.

Figure 4.1: Final storage configuration of the destination server



4.1.3 Identity Results

The numeric IDs of devuser, financeuser, developers and finance are compared between the servers. The UID and GID values on the destination must be identical to those recorded on the source.

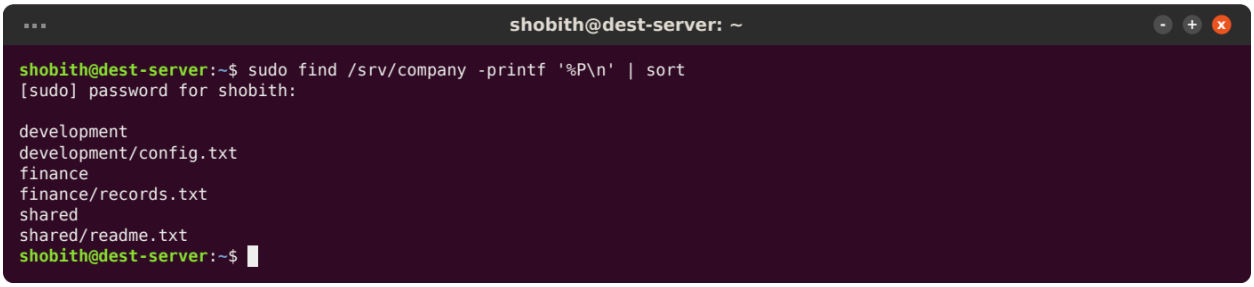
Figure 4.2: Source and destination user identities compared



4.1.4 Migration Verification

Directory structure. A sorted inventory is produced on each server, saved and compared:

```
sudo find /srv/company -printf '%P\n' | sort
```



File and directory counts and sizes are also compared:

```
sudo du -sh /srv/company
sudo find /srv/company -type f | wc -l
sudo find /srv/company -type d | wc -l
```

```
shobith@dest-server:~$ sudo du -sh /srv/company
28K     /srv/company
shobith@dest-server:~$ sudo find /srv/company -type f | wc -l
3
shobith@dest-server:~$ sudo find /srv/company -type d | wc -l
4
shobith@dest-server:~$
```

The counts should match when the migration scope is the same. Directory size may differ slightly because of file-system allocation, so matching du output alone is not sufficient proof.

Permissions and ownership. On each server:

```
sudo ls -lR /srv/company
sudo stat /srv/company/development/config.txt
sudo stat /srv/company/finance/records.txt
sudo stat -c '%n %u:%g %a' \
 /srv/company/development/config.txt \
 /srv/company/finance/records.txt \
 /srv/company/shared/readme.txt
```

```
shobith@dest-server:~$ sudo ls -lR /srv/company
/srv/company:
total 12
drwxrws--- 2 devuser     developers 4096 Oct 13 15:22 development
drwxrws--- 2 financeuser finance    4096 Oct 13 15:23 finance
drwxrwsr-x 2 root        developers 4096 Oct 13 15:24 shared

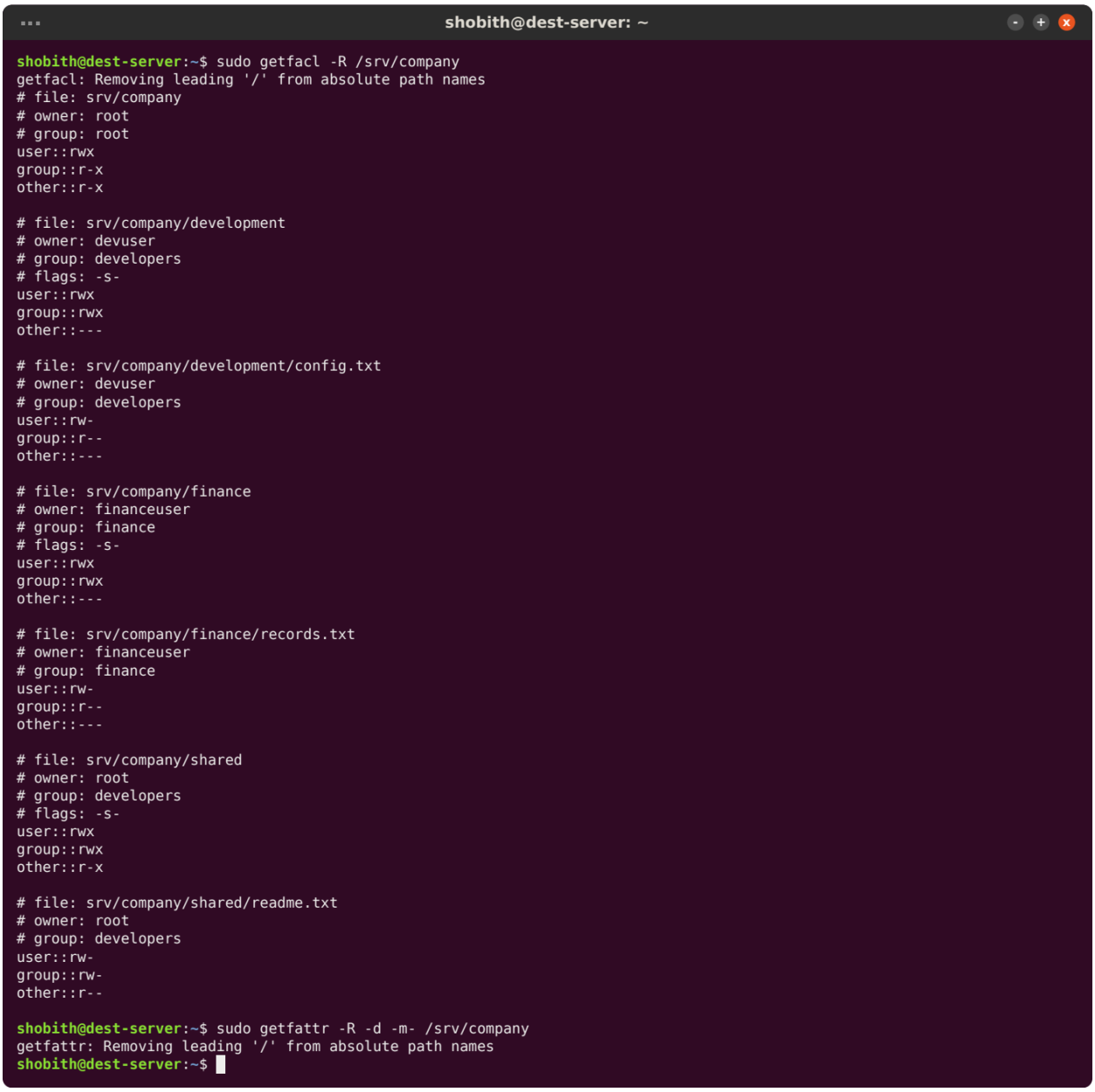
/srv/company/development:
total 4
-rw-r----- 1 devuser developers 33 Oct 13 15:22 config.txt

/srv/company/finance:
total 4
-rw-r----- 1 financeuser finance 26 Oct 13 15:23 records.txt

/srv/company/shared:
total 4
-rw-r--r-- 1 root developers 32 Oct 13 15:24 readme.txt
shobith@dest-server:~$ sudo stat /srv/company/development/config.txt
  File: /srv/company/development/config.txt
  Size: 33              Blocks: 8          IO Block: 4096   regular file
Device: 253,0   Inode: 14          Links: 1
Access: (0640/-rw-r-----)  Uid: ( 1001/ devuser)   Gid: ( 1001/developers)
Access: 2026-10-16 14:48:12.304117256 +0530
Modify: 2026-10-13 15:22:41.612903448 +0530
Change: 2026-10-16 14:48:12.304117256 +0530
 Birth: 2026-10-16 14:48:12.304117256 +0530
shobith@dest-server:~$ sudo stat /srv/company/finance/records.txt
  File: /srv/company/finance/records.txt
  Size: 26              Blocks: 8          IO Block: 4096   regular file
Device: 253,0   Inode: 16          Links: 1
Access: (0640/-rw-r-----)  Uid: ( 1002/financeuser)   Gid: ( 1002/ finance)
Access: 2026-10-15 12:09:26.540188227 +0530
Modify: 2026-10-13 15:23:18.097552310 +0530
Change: 2026-10-15 12:09:26.540188227 +0530
 Birth: 2026-10-15 12:09:26.540188227 +0530
shobith@dest-server:~$ sudo stat -c '%n %u:%g %a' \
>  /srv/company/development/config.txt \
>  /srv/company/finance/records.txt \
>  /srv/company/shared/readme.txt
/srv/company/development/config.txt 1001:1001 640
/srv/company/finance/records.txt 1002:1002 640
/srv/company/shared/readme.txt 0:1001 644
shobith@dest-server:~$
```

The numeric owner, group and permission mode must match the source. ACLs and extended attributes are inspected too:

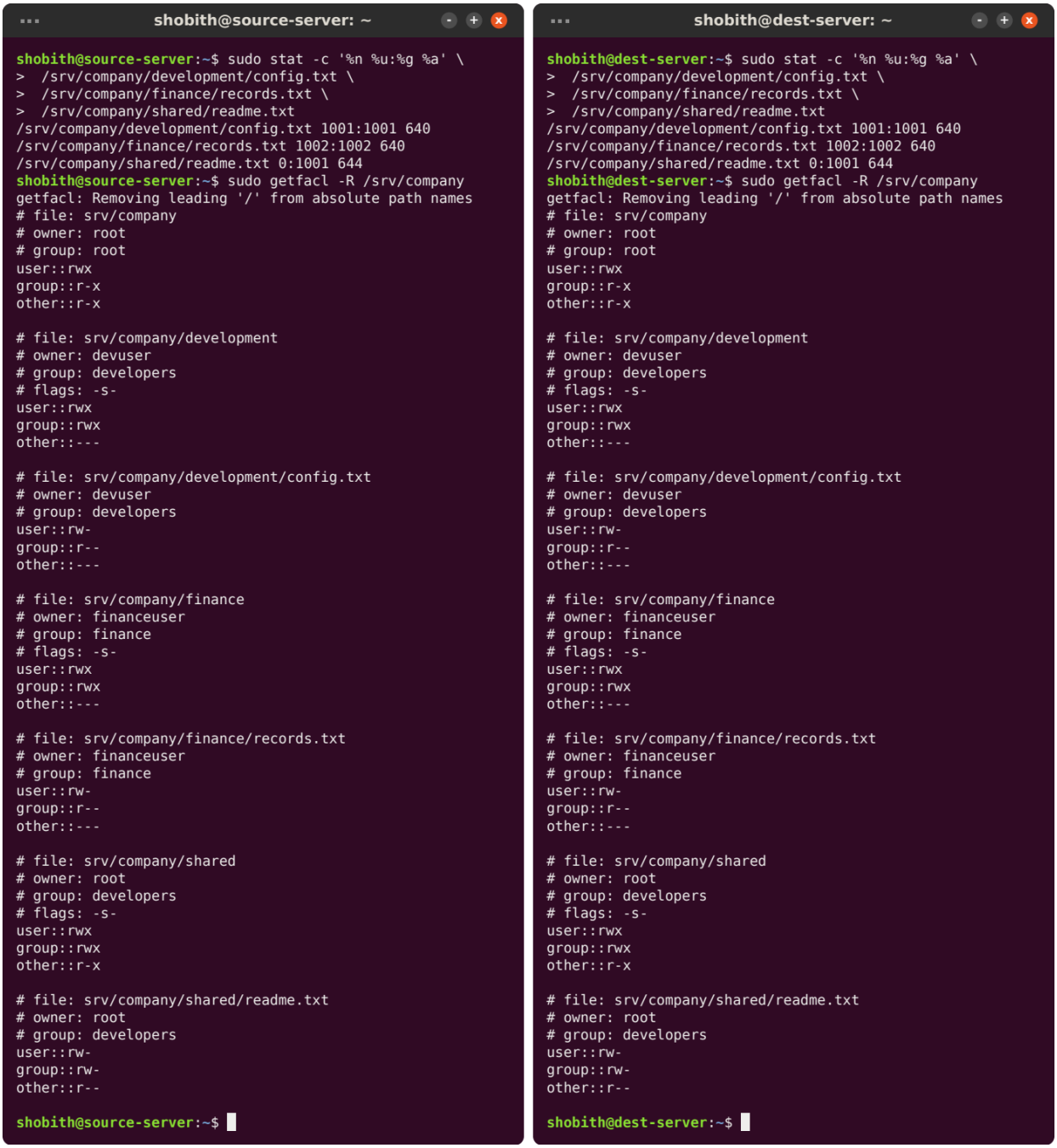
```
sudo getfacl -R /srv/company
sudo getfattr -R -d -m- /srv/company
```



For the sample data, ACLs and extended attributes may be minimal, but the migration method is still shown to be capable of preserving them.

Source server (left) and destination server (right)

Figure 4.3: Ownership, mode, ACL and attribute comparison



File contents. Checksums of the individual files are generated on both systems and compared:

```
sudo sha256sum \
 /srv/company/development/config.txt \
 /srv/company/finance/records.txt \
 /srv/company/shared/readme.txt
```

```
shobith@dest-server: ~
shobith@dest-server:~$ sudo sha256sum \
>  /srv/company/development/config.txt \
>  /srv/company/finance/records.txt \
>  /srv/company/shared/readme.txt
cf3debbc84344f997f8dc83dc37b2dbf458ceb205246cb9eefe6045bec2c720b  /srv/company/development/config.txt
ad6e39141719a12a93d3e180abff6fc3ca01ac8cbc5a5602d1f576c8a4e7a2e3  /srv/company/finance/records.txt
dbe54134ce0490b6e54ce9be17f091dbadfb1b4de669d1142595eaabd042a184  /srv/company/shared/readme.txt
shobith@dest-server:~$
```

Matching checksums show that the contents of the tested files are identical. For a large data set, a manifest containing relative paths and checksums is generated (Appendix C.4) and the source and destination manifests are compared, with the commands run in a consistent path context so that the paths are comparable.

Figure 4.4: SHA-256 checksum comparison between source and destination

```
shobith@source-server: ~
shobith@source-server:~$ sudo sha256sum \
>  /srv/company/development/config.txt \
>  /srv/company/finance/records.txt \
>  /srv/company/shared/readme.txt
cf3debbc84344f997f8dc83dc37b2dbf458ceb205246cb9eefe6045bec2c720b  /srv/company/development/config.txt
ad6e39141719a12a93d3e180abff6fc3ca01ac8cbc5a5602d1f576c8a4e7a2e3  /srv/company/finance/records.txt
dbe54134ce0490b6e54ce9be17f091dbadfb1b4de669d1142595eaabd042a184  /srv/company/shared/readme.txt
shobith@source-server:~$
shobith@dest-server: ~
shobith@dest-server:~$ sudo sha256sum \
>  /srv/company/development/config.txt \
>  /srv/company/finance/records.txt \
>  /srv/company/shared/readme.txt
cf3debbc84344f997f8dc83dc37b2dbf458ceb205246cb9eefe6045bec2c720b  /srv/company/development/config.txt
ad6e39141719a12a93d3e180abff6fc3ca01ac8cbc5a5602d1f576c8a4e7a2e3  /srv/company/finance/records.txt
dbe54134ce0490b6e54ce9be17f091dbadfb1b4de669d1142595eaabd042a184  /srv/company/shared/readme.txt
shobith@dest-server:~$
```

User access. The migrated users must be able to reach their files:

```
sudo -u devuser cat /srv/company/development/config.txt
sudo -u financeuser cat /srv/company/finance/records.txt
```

The finance user must not be able to write to the development directory:

```
sudo -u financeuser touch \
 /srv/company/development/testfile
```

This should fail when the intended permissions are applied and the user has no other access through supplementary groups or ACLs. If a write unexpectedly succeeds, the test file is removed, the reason is investigated and the corrected permissions are documented.

Figure 4.5: Access-control test results



4.1.5 Backup and Recovery Results

The backup archive is verified (`tar -tf`, `sha256sum -c`) and a deleted test file is restored. The evidence shows the file before deletion, the simulated failure, the restore operation and the final verification of ownership, permissions and checksum. This demonstrates an actual recovery test rather than merely claiming that a backup exists.

4.1.6 Automation Results

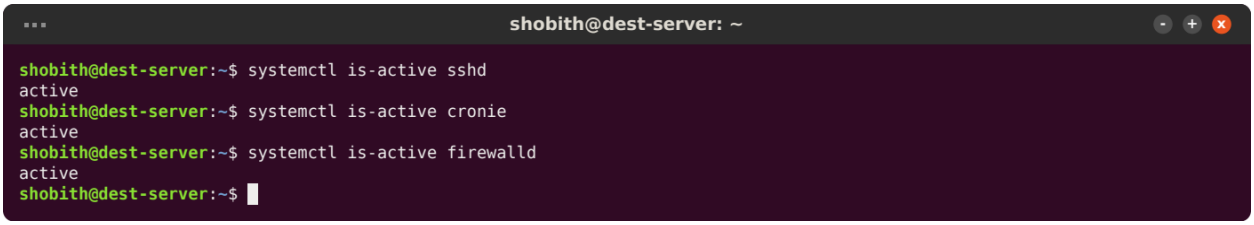
The backup script is run manually, then its archive, checksum and log entries are inspected. The Cron entry `0 2 * * * /usr/local/sbin/company-backup.sh` and the status of the cronie service are shown. The script and its scheduling reduce the need for repeated manual execution.

4.1.7 Security Configuration Results

The firewalld service state, the allowed services, the permission chain of the data files and the ownership and mode of the backup script are recorded. The expected state is: the firewall is active with SSH allowed, restricted data is reachable only by its intended users, and the backup script is owned by root and writable only by root.

The final service check on the destination is:

```
systemctl is-active sshd
systemctl is-active cronie
systemctl is-active firewalld
```



4.1.8 Migration Acceptance Checklist

A check is completed only after the output has been examined and the evidence recorded; running the command is not sufficient.

Table 4.2 gives the checklist used to confirm that the migration is complete.

Table 4.2: Migration acceptance checklist

Check	Acceptance criterion	Done
Storage	GPT partition, LVM PV, VG, logical volumes and ext4 file systems verified.	✓
Mounts	Data and backup volumes mounted at the correct paths and configured to persist after reboot.	✓
User identities	Required users and groups exist on the destination with matching numeric IDs.	✓
Data integrity	File inventory and SHA-256 checksums match for the agreed migration scope.	✓
Permissions	Ownership, permission modes, ACLs and relevant extended attributes verified.	✓
Access control	Authorized users can access their data and unauthorized access tests fail.	✓
Backup and recovery	Backup archive validated and a deleted test file successfully restored.	✓
Automation	Backup script runs successfully and Cron is configured.	✓
Security	SSH connectivity works, firewall rules are active and privileged scripts are protected.	✓
Documentation	Before-and-after evidence, configuration details and test results recorded.	✓

4.2 Testing and Validation

Testing and validation verify the main functions of the migration: storage configuration, user identities, data migration, ownership and permissions, backup and recovery, automation and security. Table 4.3 lists each test case with its expected result and the actual result and status recorded from the captured evidence.

Table 4.3 records the tests carried out, with the expected and actual results.

Table 4.3: Testing and validation

Test Case	Expected Result	Actual Result	Status

Test Case	Expected Result	Actual Result	Status
Disk identification	The new empty disk `/dev/sdb` is identified; the OS disk is unmodified.	`/dev/sdb` (20 GB, no partitions) identified; `/dev/sda` unchanged.	**Pass**
GPT partition	`/dev/sdb1` exists with type Linux LVM and the expected size.	`/dev/sdb1` created on a GPT label, type Linux LVM, size 20G.	**Pass**
LVM configuration	PV, VG *vg_migration*, *lv_data* (12 GB) and *lv_backup* (5 GB) are listed.	PV `/dev/sdb1`, VG *vg_migration*, *lv_data* 12.00g and *lv_backup* 5.00g listed.	**Pass**
File systems and mounts	ext4 file systems are mounted at `/srv/company` and `/srv/backup`; `findmnt --verify` and `mount -a` succeed.	Both ext4 volumes mounted; `findmnt --verify` reported success and `mount -a` returned no errors.	**Pass**
User identities	UIDs and GIDs of devuser, financeuser, developers and finance match the source.	devuser 1001:1001 and financeuser 1002:1002; developers 1001 and finance 1002 on both servers.	**Pass**
Pre-migration backup	Archive contents are listed and the checksum verifies.	`tar -tf` listed the company tree; `sha256sum -c` reported OK.	**Pass**
Data migration	File and directory counts and the sorted inventory match the source.	Identical inventory; 3 files and 4 directories on both servers.	**Pass**
Ownership and permissions	Numeric owner, group, mode and ACLs match the source.	`stat`, `ls -lR` and `getfacl` output identical on source and destination.	**Pass**
Content integrity	SHA-256 checksums of the tested files match.	Checksums of config.txt, records.txt and readme.txt identical on both servers.	**Pass**
Access control	Authorized reads succeed; financeuser cannot write to the development directory.	Reads by devuser and financeuser succeeded; write by financeuser failed with Permission denied.	**Pass**
Recovery	A deleted file is restored from the archive with matching metadata and checksum.	config.txt restored; mode 0640, owner 1001:1001 and SHA-256 checksum match the original.	**Pass**
Automation	The script creates an archive and verified checksum, logs the run, and Cron entry exists.	Archive and .sha256 created (OK), log entry written; Cron entry `0 2 * * *` present.	**Pass**
Security	firewalld is active with SSH allowed; the backup script is root-owned and not group/world-writable.	firewalld active with ssh allowed; script is root:root with mode 0750.	**Pass**

4.2.1 Recovery Testing

The recovery test checks that a deleted file can be restored from the verified archive. The archive is extracted into a separate temporary directory, the restored file's checksum and metadata are

verified, and only then is the file copied back with`cp -a`. Extracting into a separate directory avoids overwriting live data.

4.2.2 Data Integrity Verification

Integrity is verified with SHA-256 checksums of the archive and of the migrated files. A checksum comparison shows that file contents are identical; it is combined with inventory, ownership and ACL checks because content checksums alone do not prove that metadata was preserved.

4.2.3 Testing Summary and Limitations

The tests cover storage, identities, migration, ownership and permissions, backup and recovery, automation and security. One important limitation is that the project migrates a defined data set and the required user identities. It does not automatically reproduce every aspect of a production server, such as installed applications, databases, service configurations, scheduled jobs, encryption keys or live application state; those would need their own migration and validation procedures if they were included in the scope.

The backup script does not yet delete old archives, and the lab keeps its backup volume on the same virtual disk as the migrated data, so an additional independent copy is recommended.

4.3 Dashboard Results and Testing

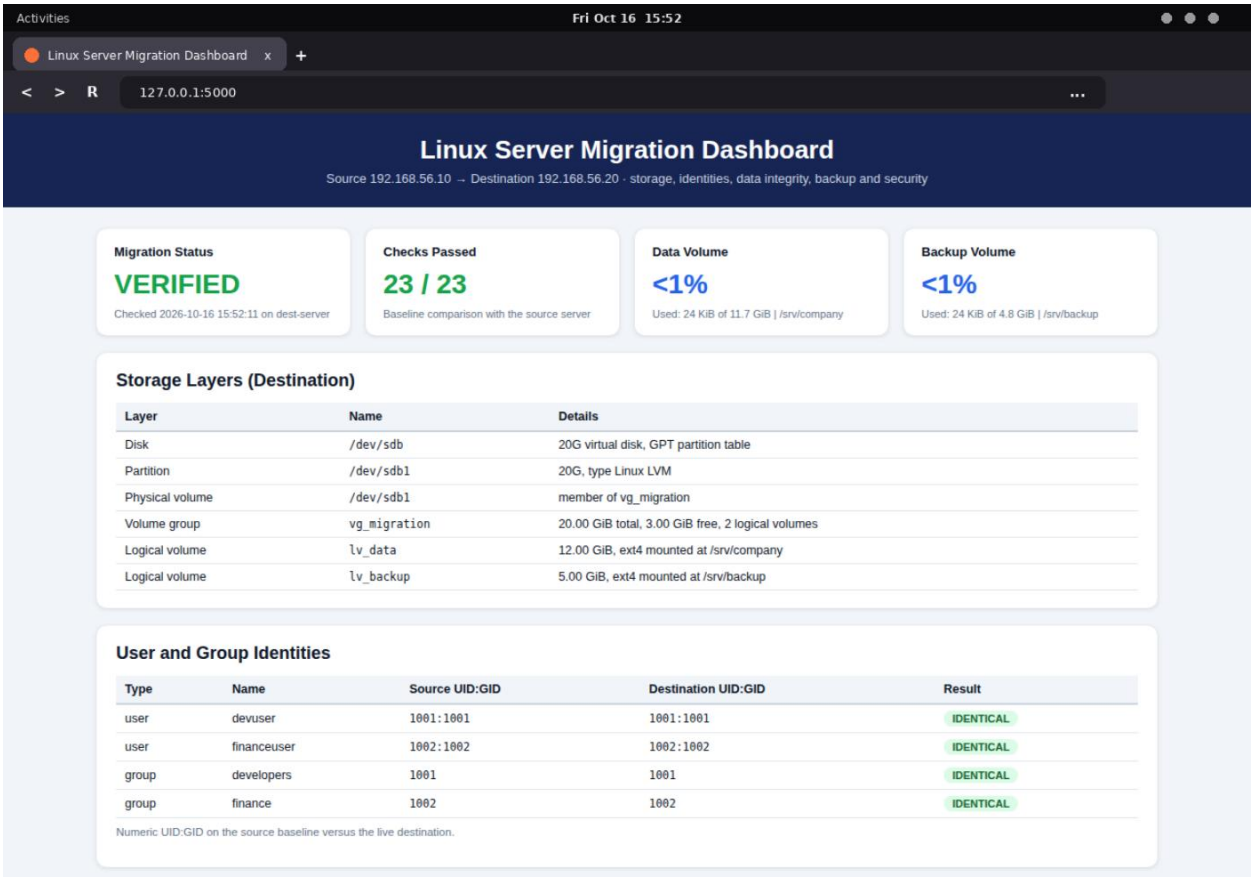
The dashboard was started on the destination server after the migration, backup, automation and security steps of Chapter 3 were complete, and was opened in Firefox on the host through the SSH tunnel at http://127.0.0.1:5000. The page presents the final output of the project in one place: the values below come from the live system and the source baseline, not from typed text.

4.3.1 Dashboard Output

Status cards, storage layers and identities. The four cards at the top show the overall status (VERIFIED), the number of checks passed (23 / 23), and the usage of the data and backup volumes. The Storage Layers table lists the chain disk, partition, physical volume, volume group and logical volumes, matching Figures 3.8 and 4.1. The identity table shows the numeric

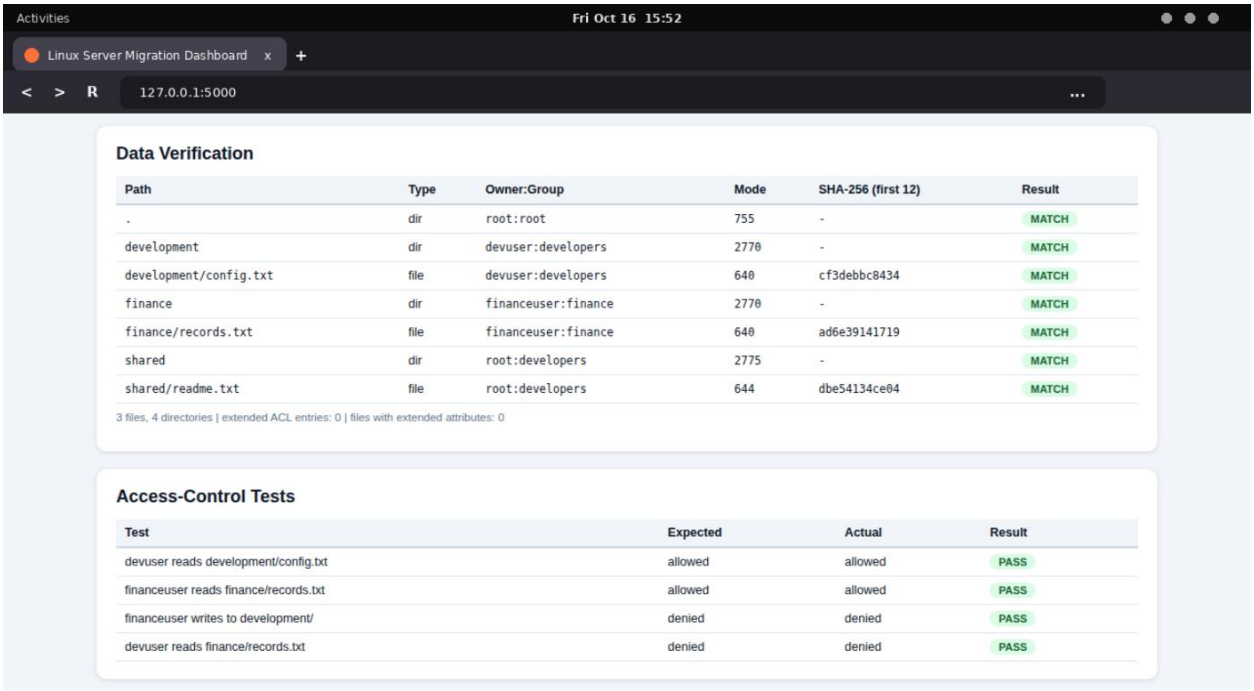
UID:GID of every user and group on the source baseline and on the destination (compare Figure 4.2).

Figure 4.6: Dashboard — status cards, storage layers and user/group identities



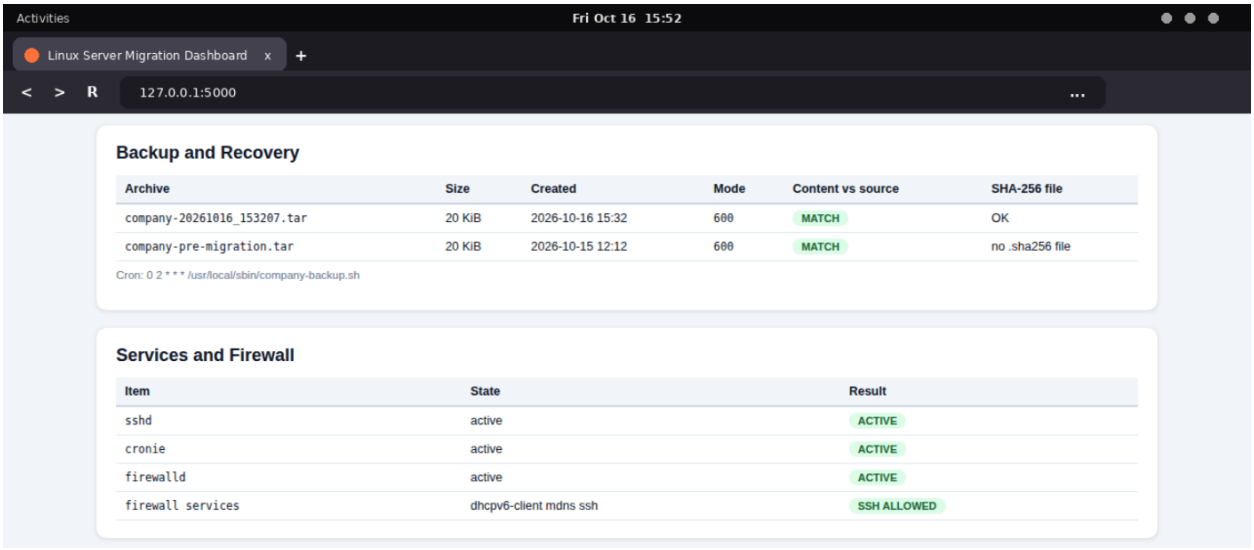
Data verification and access control. Each migrated item is listed with its owner and group names, octal mode (including the setgid bit in 2770 and 2775), the first 12 characters of its SHA-256 checksum and a MATCH badge. The checksum prefixes equal the values in Figure 4.4. Below the table, the four access tests show that the intended users can read their data while financeuser cannot write to the development directory (compare Figure 4.5).

Figure 4.7: Dashboard — data verification and access-control tests



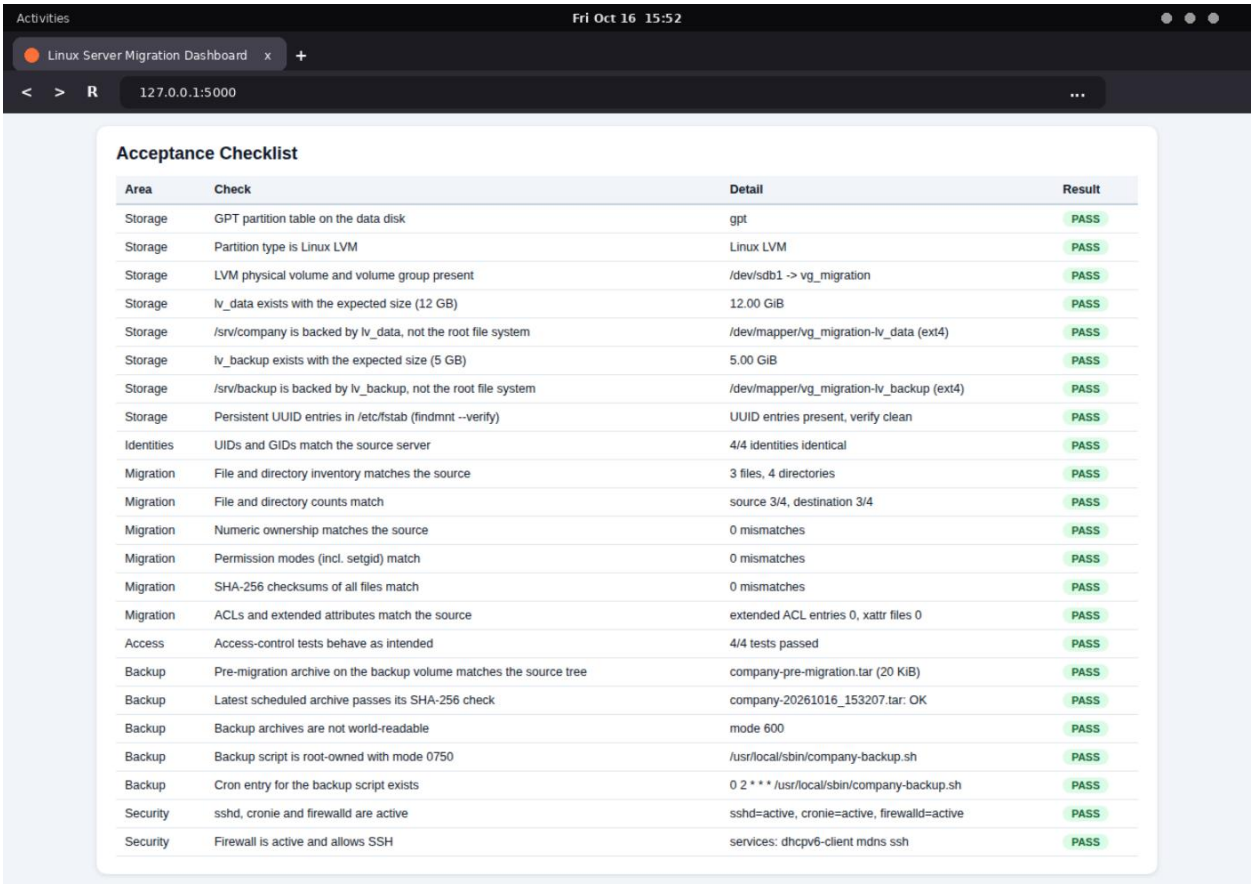
Backup, recovery, services and firewall. Each archive on the backup volume is listed with its size, creation time and mode 600; the content column confirms that the archive holds exactly the source tree, and the checksum column shows the result of verifying the .sha256 file. The cron entry and the states of sshd, cronie, firewalld and the allowed firewall services follow.

Figure 4.8: Dashboard — backup archives, cron entry, services and firewall



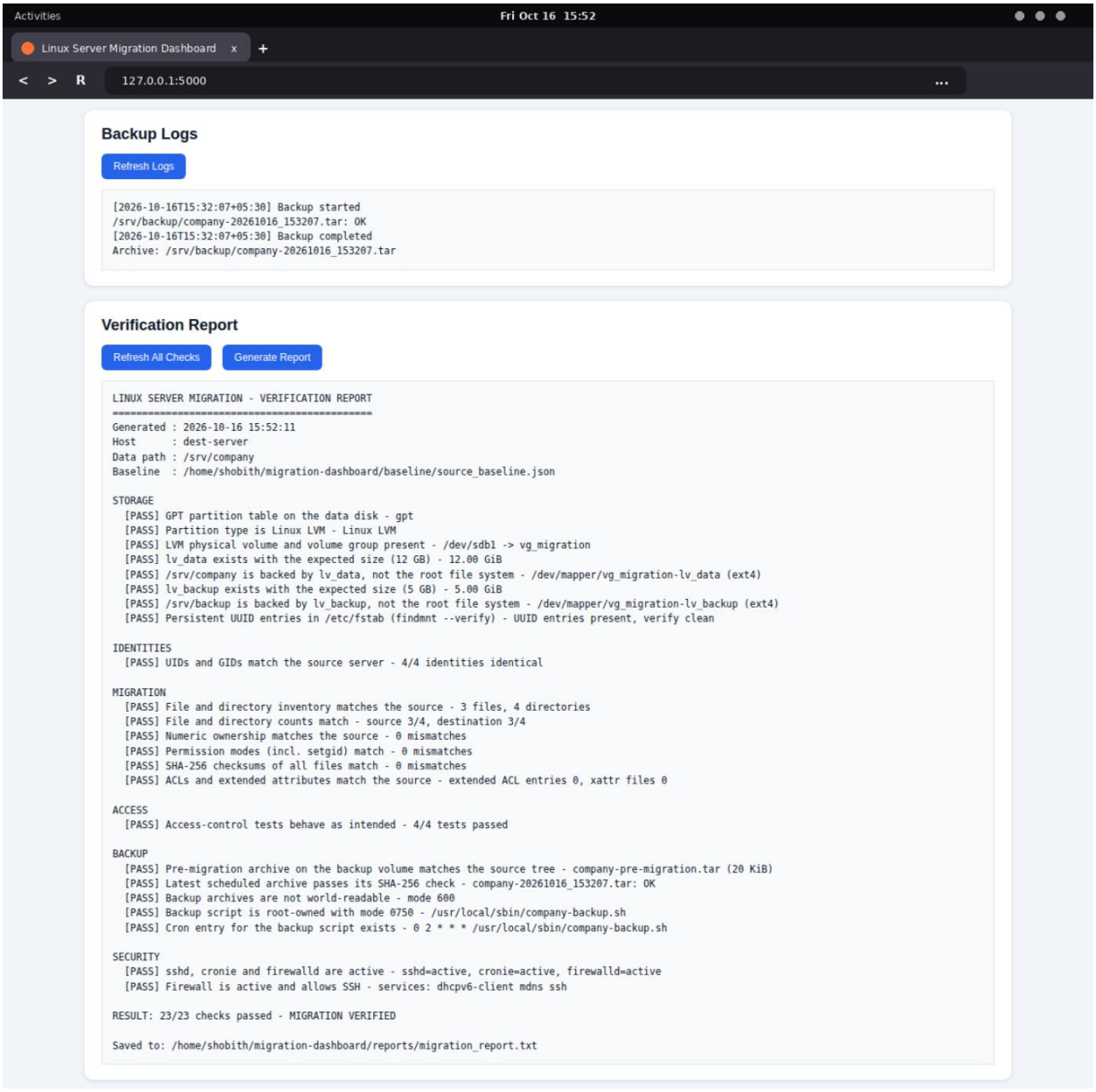
Acceptance checklist. All 23 checks are listed with their detail and a PASS badge. This list is the live version of the acceptance checklist in Table 4.2.

Figure 4.9: Dashboard — acceptance checklist (23 of 23 checks passed)



Backup log and saved report. The Backup Logs section shows the last lines of `/var/log/company-backup/backup.log` written by the backup script. The Generate Report button saves the checklist to`reports/migration_report.txt` and shows it on the page.

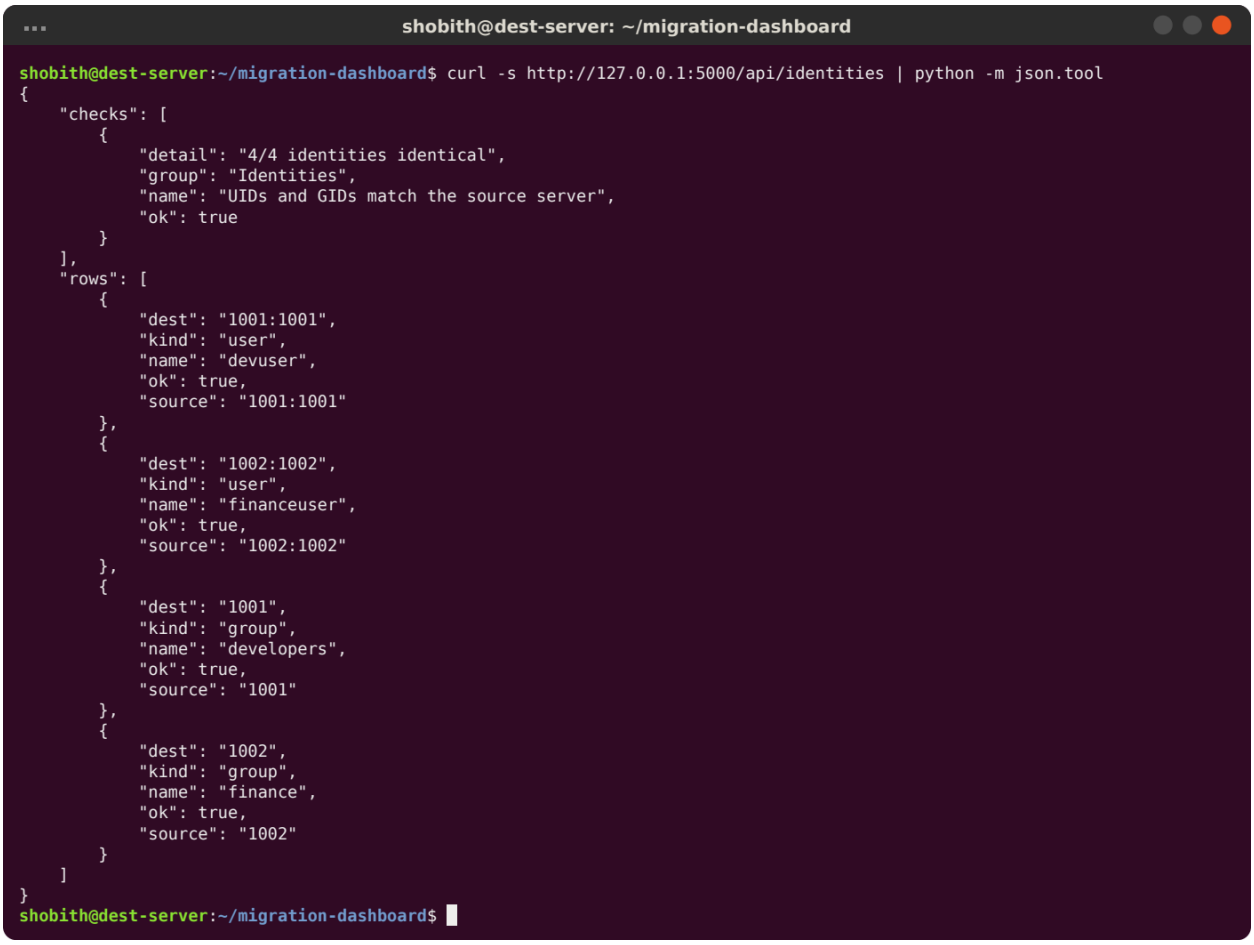
Figure 4.10: Dashboard — backup log and generated verification report



4.3.2 Testing the Dashboard APIs with curl

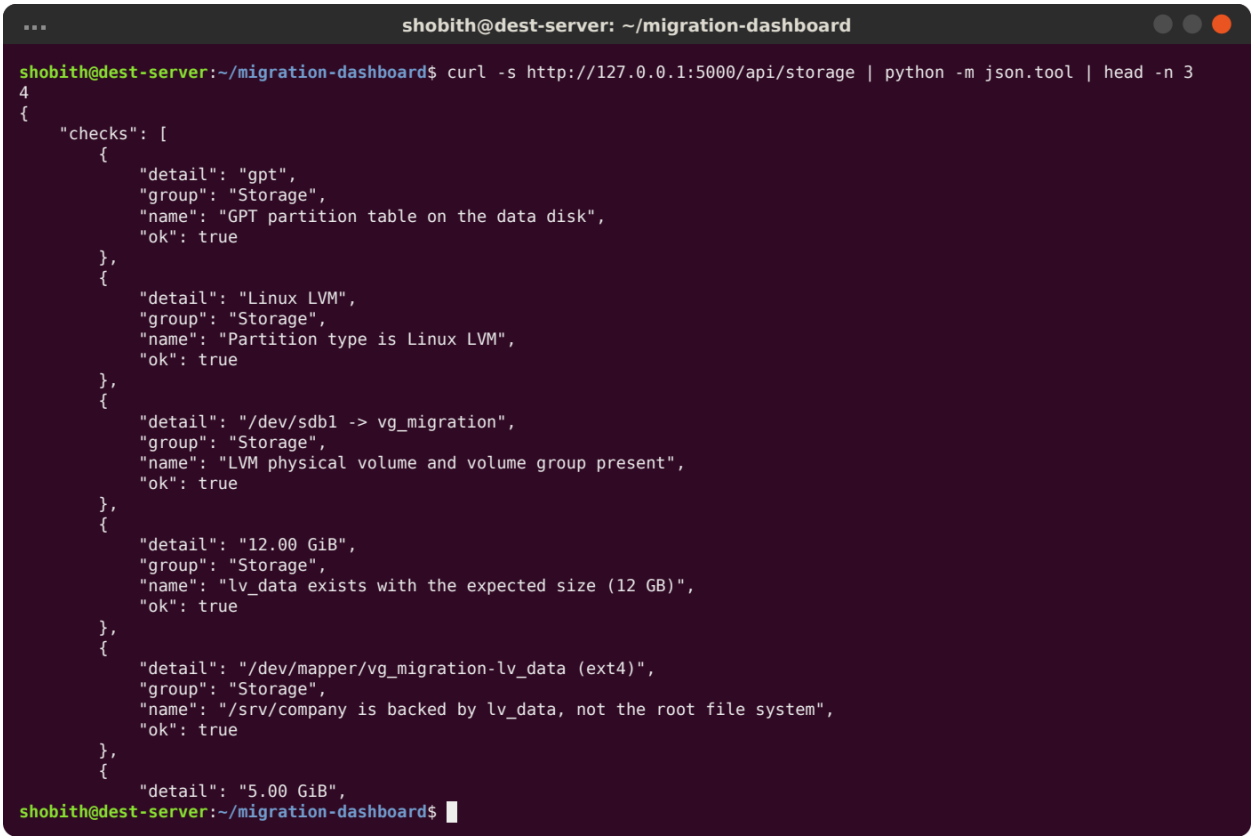
The API routes were also tested from a second terminal with curl, which proves that the backend returns the results independently of the browser. The identity route returns the source and destination IDs for the four identities:

Figure 4.11: Response of the /api/identities route



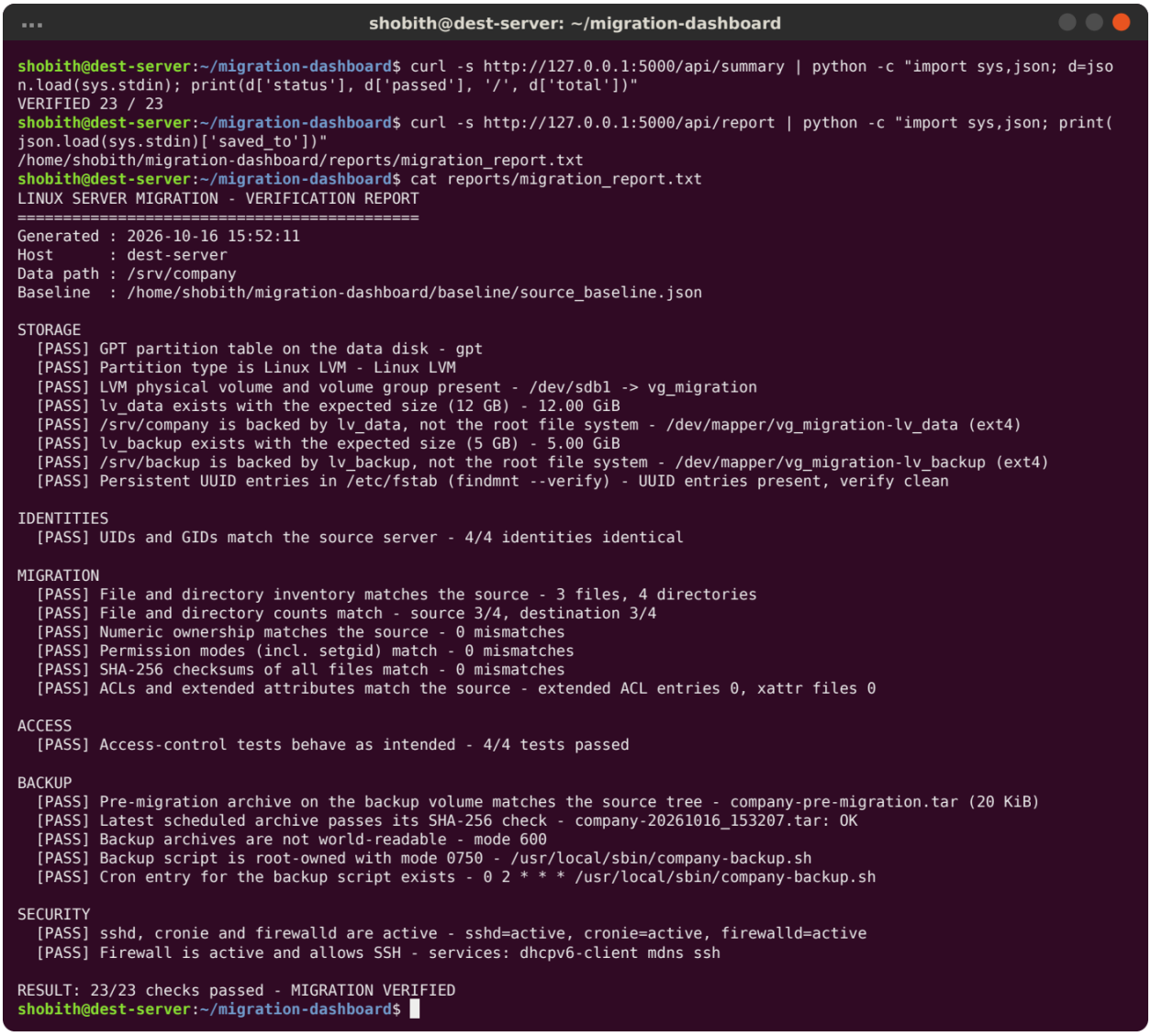
The storage route returns the layers, the mounted volumes with their usage and the storage checks (first lines shown):

Figure 4.12: Response of the /api/storage route



The summary route gives the overall status, and the report route saves the text report, which is then displayed with cat:

Figure 4.13: Summary status and the saved verification report



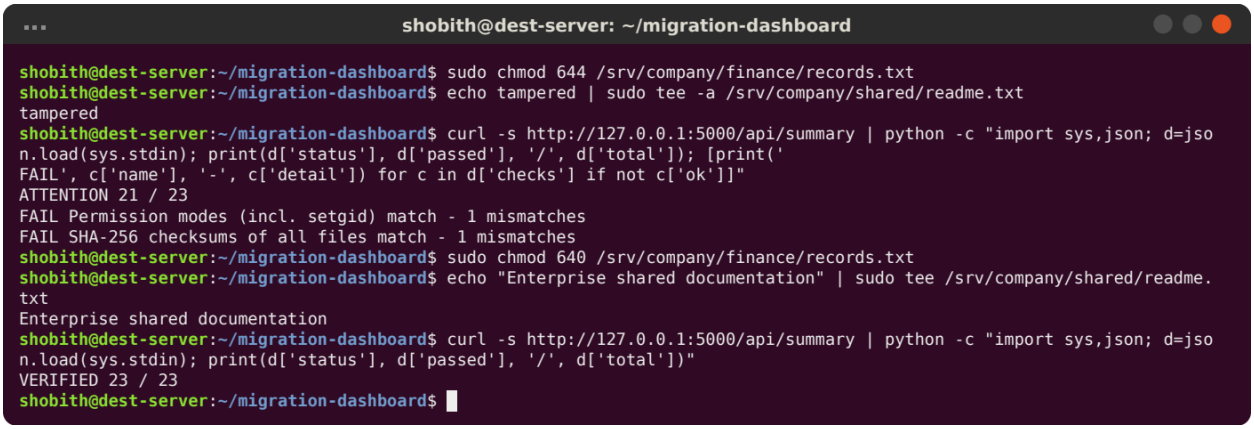
4.3.3 Negative Test: Detecting a Faulty Migration

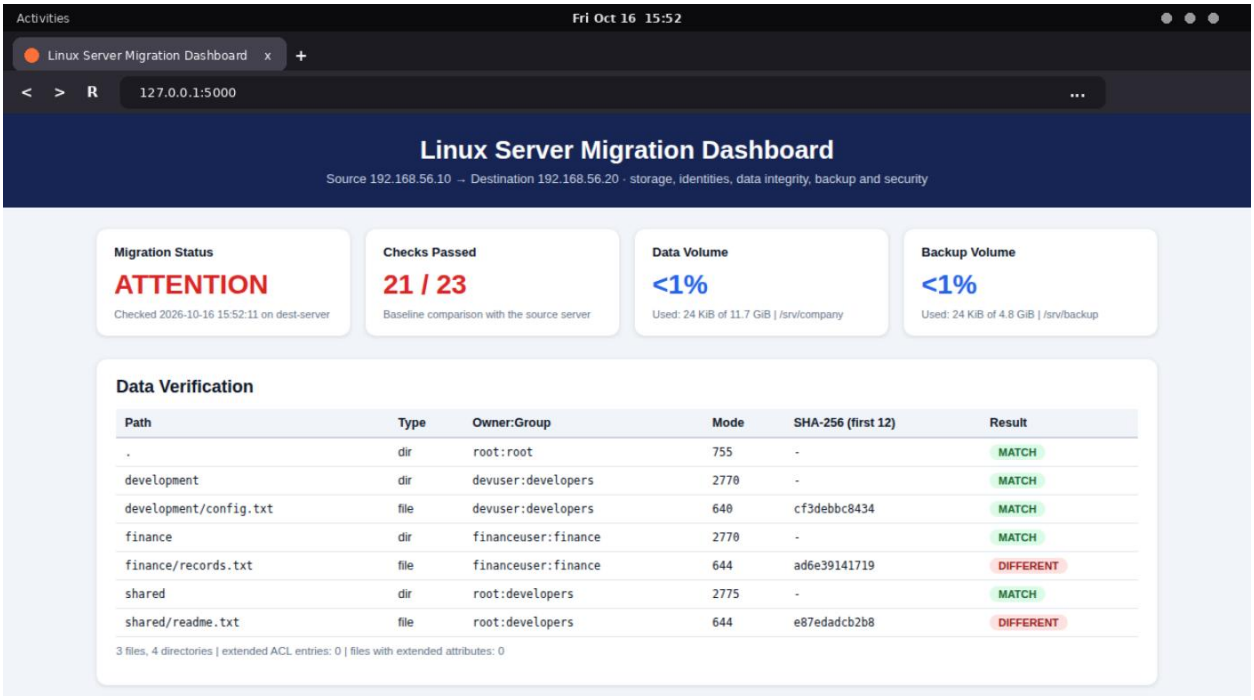
A verification tool is only useful if it fails when something is wrong. Two deliberate faults were introduced on the destination: the permission mode of`finance/records.txt` was loosened from 640 to 644, and a line was appended to`shared/readme.txt` so that its content no longer matches the source. The dashboard changed to ATTENTION with 21 of 23 checks passed,

named the two failing checks, and marked both files DIFFERENT. After the faults were reverted, the status returned to VERIFIED with 23 of 23 checks passed.

Figure 4.14: Deliberate faults detected by the dashboard and then reverted

Figure 4.15: Dashboard in the ATTENTION state while the two faults were present





4.3.4 Dashboard Test Cases

Test case	Expected result	Actual result	Status
Start-up without root	Backend refuses to start and asks for sudo	Message "Run with sudo: the dashboard must read root-only data and backups." and exit	Pass

Test case	Expected result	Actual result	Status
Start-up with sudo	Flask serves on 127.0.0.1:5000	Running on http://127.0.0.1:5000 (Figure 3.19)	Pass
Overall status	VERIFIED with 23 of 23 checks	VERIFIED 23 / 23 (Figures 4.6, 4.9, 4.13)	Pass
Storage layers	GPT, Linux LVM, vg_migration, lv_data 12 GB, lv_backup 5 GB, ext4 mounts	All storage checks passed; both mounts backed by the logical volumes	Pass

Table 4.4 records the dashboard tests, with the expected and actual results.

Table 4.4: Dashboard testing and validation

Test case	Expected result	Actual result	Status
Identities	UID/GID equal to the source baseline	4 of 4 identities identical	Pass
Data verification	All items MATCH; counts 3 files and 4 directories	7 of 7 items MATCH; 0 ownership, 0 mode and 0 checksum mismatches	Pass
ACLs and extended attributes	Equal to the source baseline	Extended ACL entries 0, xattr files 0 on both	Pass
Access control	Intended reads allowed; financeuser write and devuser read of finance denied	4 of 4 tests passed	Pass
Backup archives	Pre-migration archive matches source tree; latest archive checksum OK; mode 600	Both archives MATCH; company-20261016_153207.tar: OK	Pass
Cron, script, services, firewall	Cron entry present; script root:root 0750; services active; ssh allowed	All present; sshd, cronie and firewalld active; ssh allowed	Pass
API routes with curl	JSON returned by /api/identities, /api/storage, /api/summary, /api/report	Valid JSON returned by every tested route (Figures 4.11 to 4.13)	Pass
Report generation	reports/migration_report.txt created	File created; ends with MIGRATION VERIFIED	Pass
Refresh controls	Refresh Logs reloads the log; page values refresh every 30 seconds	Refresh Logs called /api/logs; /api/summary was requested again after 30 seconds	Pass
Fault detection	Loosened mode and changed content are reported	ATTENTION 21 / 23; mode and checksum checks FAIL (Figures 4.14, 4.15)	Pass
Recovery after repair	Status returns to VERIFIED	VERIFIED 23 / 23 after the faults were reverted	Pass

4.3.5 Notes and Limitations

- The dashboard reports the state at the moment of each refresh. It does not freeze the data, so changes made while it is open appear at the next refresh.
- The comparison is only as trustworthy as the baseline. The baseline must be created on the source after the final synchronization and copied over a trusted channel; if the source data changes afterwards, the affected items correctly show as DIFFERENT until a new baseline is recorded.
- The checks cover the defined data set (/srv/company), the project identities and the backup and security items listed in Table 3.11. Applications, databases and service configurations remain outside the scope, as stated in Section 4.2.3.
- The backup checks need the pre-migration archive and at least one scheduled archive. Before the backup script has run once, the "latest scheduled archive" check shows FAIL. Only the newest archive is checked against its .sha256 file; the retention and off-disk copy recommendations of Section 4.2.3 still apply.
- Storage values such as used space are read live and will differ from the figures in this report as the volumes are used.
- Flask's development server is run as root for this laboratory demonstration only. It listens on 127.0.0.1 and is reached through an SSH tunnel; a production deployment would use a production web server and a less privileged helper for the privileged reads.

5. Conclusion and Future Scope

5.1 Conclusion

The Linux Server Migration project provides a complete, step-by-step procedure for migrating storage, file systems and user data from a source Linux server to a destination Linux server while preserving permissions and ownership. It starts from disk partitioning and builds the destination storage with GPT, LVM and ext4, mounts it persistently, recreates the required identities with matching numeric IDs, protects the data with a verified pre-migration backup, and transfers it using SSH, tar and rsync with metadata-preserving options.

The project also covers verification and operations: inventory, ownership, ACL and checksum comparison; user access tests; a backup-and-restore test; a Bash backup script scheduled with Cron; and firewall, SSH and least-privilege access controls. Together these exercise the main areas of Linux system administration: storage management, user and group administration, file permissions, package and service management, networking, automation, security and documentation.

A migration is considered successful only when the acceptance checklist in Section 4.1.8 is satisfied with recorded evidence. The project is limited to a defined data set and the required identities; migrating applications, databases and service configurations would need additional procedures.

5.2 Future Scope

The project can be extended in the following ways:

- **Full server migration:** extend the scope to applications, databases, service configurations, scheduled jobs and encryption keys, each with its own validation.
- **Identity migration:** migrate passwords, SSH keys and supplementary group memberships securely, or adopt centralized identity management such as LDAP.
- **Restricted rsync receiver:** configure a restricted, authenticated root-capable rsync receiver for production-style transfers without unrestricted root SSH login.

- **Backup retention and off-site copies:** add a tested retention policy, monitor free space, and keep independent backups on another device or location.
- **LVM snapshots:** use snapshots to freeze a consistent point in time for the final synchronization.
- **Monitoring and notification:** alert administrators on backup failures, mount problems or low disk space.
- **Hardened security:** restrict SSH to specific source addresses, enforce password policies and review firewall zones.
- **Configuration management:** automate server preparation and migration with tools such as Ansible for repeatable deployments.

References

[1] Arch Wiki, LVM. Available at: https://wiki.archlinux.org/title/LVM.

[2] Arch Wiki, rsync. Available at: https://wiki.archlinux.org/title/Rsync.

[3] Arch Wiki, fstab. Available at: https://wiki.archlinux.org/title/Fstab.

[4] Arch Wiki, cron. Available at: https://wiki.archlinux.org/title/Cron.

[5] Arch Wiki, Access Control Lists. Available at: https://wiki.archlinux.org/title/Access_Control_Lists.

[6] EndeavourOS, EndeavourOS Documentation. Available at: https://discovery.endeavouros.com/.

[7] rsync project, rsync Documentation. Available at: https://rsync.samba.org/documentation.html.

[8] GNU Project, GNU Tar Manual. Available at: https://www.gnu.org/software/tar/manual/tar.html.

[9] GNU Project, GNU Coreutils Manual. Available at: https://www.gnu.org/software/coreutils/manual/coreutils.html.

[10] GNU Project, GNU Bash Reference Manual. Available at: https://www.gnu.org/software/bash/manual/bash.html.

[11] firewalld, firewalld Documentation. Available at: https://firewalld.org/documentation/.

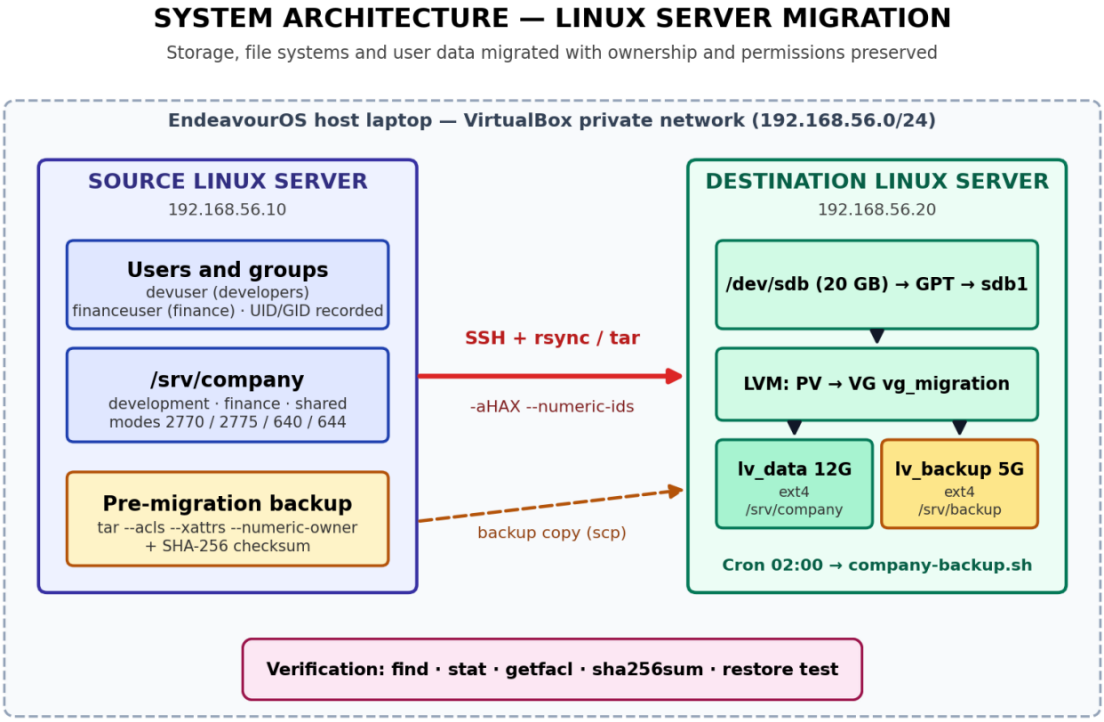
[12] Linux Manual Pages Project, Linux Manual Pages (man pages). Available at: https://www.kernel.org/doc/man-pages/.

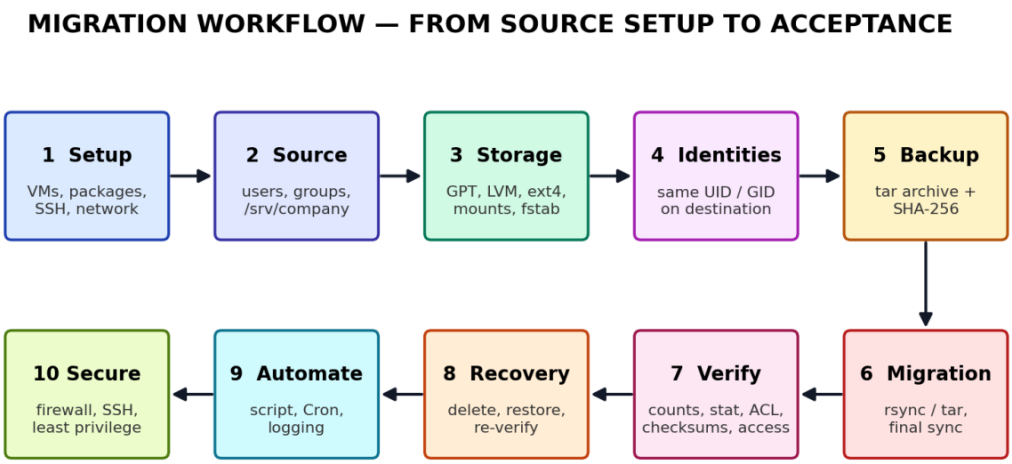
Appendices

Appendix A: System Architecture and Workflow

Figure A1: System architecture of the Linux Server Migration project

Figure A2: Migration workflow from source setup to acceptance





Appendix B: Linux Commands Quick Reference

This appendix summarizes the important Linux commands used in the project. Detailed explanations and screenshots are given in Section 3 and Section 4.

B.1 System, Disk and Network Information

Command	Purpose
`lsblk -f`	Show block devices, file systems and mount points
`lsblk -o NAME,SIZE,TYPE,FSTYPE,MOUNTPOINTS,MODEL`	Identify the OS disk and the new data disk
`sudo fdisk -l`	List disks and partition tables
`ip -br addr / ip route`	Show IP addresses and routes
`ping -c 4 192.168.56.20`	Test connectivity to the destination

B.2 Packages and Services

Command	Purpose
`sudo pacman -Syu`	Synchronize and upgrade the system
`sudo pacman -S lvm2 rsync openssh acl attr`	Install the required tools
`sudo systemctl enable --now sshd`	Enable and start SSH
`sudo pacman -S cronie / firewalld`	Install the scheduler and the firewall
`systemctl is-active sshd cronie firewalld`	Check service state

B.3 Users, Groups and Data (Source)

Command	Purpose
`sudo groupadd developers / finance`	Create the groups
`sudo useradd -m -s /bin/bash -g developers devuser`	Create a user with a home directory
`id devuser; getent group developers`	Record numeric IDs
`sudo chown devuser:developers /srv/company/development`	Set ownership
`sudo chmod 2770 /srv/company/development`	Set permissions with setgid
`sudo ls -lR /srv/company`	List data with ownership and permissions

B.4 Partitioning, LVM and File Systems (Destination)

Command	Purpose
`sudo fdisk /dev/sdb`	Create the GPT partition (g, n, t, p, w)
`sudo partprobe /dev/sdb`	Re-read the partition table
`sudo pvcreate /dev/sdb1`	Create the LVM physical volume
`sudo vgcreate vg_migration /dev/sdb1`	Create the volume group
`sudo lvcreate -L 12G -n lv_data vg_migration`	Create the data logical volume
`sudo lvcreate -L 5G -n lv_backup vg_migration`	Create the backup logical volume
`sudo mkfs.ext4 /dev/vg_migration/lv_data`	Create an ext4 file system
`sudo mount /dev/vg_migration/lv_data /srv/company`	Mount the data volume
`sudo blkid`	Show file-system UUIDs for /etc/fstab
`sudo findmnt --verify --verbose; sudo mount -a`	Test /etc/fstab
`df -hT; findmnt /srv/company`	Check capacity, type and mount

B.5 Backup, Migration and Recovery

Command	Purpose
`sudo tar --acls --xattrs --numeric-owner -cpf ... -C /srv company`	Create the pre-migration archive
`sudo tar -tf company-pre-migration.tar`	List archive contents
`sudo sha256sum -c company-pre-migration.tar.sha256`	Verify the archive checksum

Command	Purpose
`sudo scp ... your_destination_username@192.168.56.20:/tmp/`	Copy the archive to the destination
`sudo tar --acls --xattrs --numeric-owner -xpf ... -C /srv`	Extract the archive as root
`sudo rsync -aHAX --numeric-ids -v --delete --dry-run --rsync-path="sudo rsync" ...`	Preview the final synchronization
`sudo rsync -aHAX --numeric-ids -v --delete --info=progress2 --rsync-path="sudo rsync" ...`	Perform the final synchronization
`sudo cp -a /tmp/migration-restore/... /srv/company/...`	Restore a file preserving attributes

B.6 Verification

Command	Purpose
`sudo find /srv/company -printf '%P\n' | sort`	Sorted inventory of files and directories
`sudo du -sh /srv/company`	Total size
`sudo find /srv/company -type f | wc -l`	File count
`sudo stat -c '%n %u:%g %a' FILE...`	Numeric owner, group and mode
`sudo getfacl -R /srv/company`	Inspect ACLs
`sudo getfattr -R -d -m- /srv/company`	Inspect extended attributes
`sudo sha256sum FILE...`	Checksum of file contents
`sudo -u financeuser touch /srv/company/development/testfile`	Negative access test (should fail)

B.7 Automation and Security

Command	Purpose
`sudo /usr/local/sbin/company-backup.sh`	Run the backup script manually
`sudo tail -n 20 /var/log/company-backup/backup.log`	Review the backup log
`sudo crontab -e` / `sudo crontab -l`	Edit and list root's scheduled jobs
`sudo firewall-cmd --get-active-zones`	Show active firewall zones
`sudo firewall-cmd --permanent --add-service=ssh`	Allow SSH
`sudo firewall-cmd --reload` / `--list-services`	Apply and verify firewall rules
`sudo namei -l FILE`; `stat /usr/local/sbin/company-backup.sh`	Check permission chain and script protection

Appendix C: Scripts and Configuration Files

C.1 Automated Backup Script — /usr/local/sbin/company-backup.sh

```
#!/usr/bin/env bash
set -Eeuo pipefail
umask 077

SOURCE="/srv/company"
BACKUP_DIR="/srv/backup"
LOG_DIR="/var/log/company-backup"
TIMESTAMP="$(date +%Y%m%d_%H%M%S)"
ARCHIVE="$BACKUP_DIR/company-$TIMESTAMP.tar"
LOG_FILE="$LOG_DIR/backup.log"

mkdir -p "$LOG_DIR"
exec >> "$LOG_FILE" 2>&1

echo "[$(date --iso-8601=seconds)] Backup started"

if ! mountpoint -q "$SOURCE"; then
 echo "ERROR: Source file system is not mounted"
 exit 1
fi

if ! mountpoint -q "$BACKUP_DIR"; then
 echo "ERROR: Backup file system is not mounted"
 exit 1
fi

tar --acls --xattrs --numeric-owner \
 -cpf "$ARCHIVE" \
 -C /srv company

sha256sum "$ARCHIVE" > "$ARCHIVE.sha256"
sha256sum -c "$ARCHIVE.sha256"

echo "[$(date --iso-8601=seconds)] Backup completed"
echo "Archive: $ARCHIVE"
```

The script uses`set -Eeuo pipefail` so that it stops on errors, sets a restrictive`umask`, refuses to run if either file system is not mounted, writes a timestamped archive with a checksum, and logs its activity.

C.2 Persistent Mount Entries — /etc/fstab

```
UUID=<DATA_UUID> /srv/company ext4 defaults 0 2
UUID=<BACKUP_UUID> /srv/backup ext4 defaults 0 2
```

C.3 Cron Entry — root crontab

```
0 2 * * * /usr/local/sbin/company-backup.sh
```

C.4 Checksum Manifest for Large Data Sets

A manifest of relative paths and checksums is generated in the same path context on both servers, then the two manifests are compared:

```
cd /srv/company
sudo find . -type f -print0 |
 sort -z |
 sudo xargs -0 sha256sum > /tmp/manifest-$(hostname).sha256
# copy both manifests to one machine, then compare
diff /tmp/manifest-source.sha256 /tmp/manifest-destination.sha256
```

An empty diff output indicates that the manifests, and therefore the checked file contents, are identical.

C.5 Important Project Files and Locations

Table C1 lists the important project files and the purpose of each.

Table C1: Important project files

File or Directory	Purpose
`/srv/company`	Migrated data (source and destination)
`/srv/backup`	Backup logical volume mount point (destination)
`/var/backups/migration/company-pre-migration.tar`	Pre-migration archive (source)
`/var/backups/migration/company-pre-migration.tar .sha256`	Checksum of the pre-migration archive
`/srv/backup/company-pre-migration.tar`	Copy of the pre-migration archive on the backup volume
`/usr/local/sbin/company-backup.sh`	Automated backup script
`/var/log/company-backup/backup.log`	Backup log
`/etc/fstab`	Persistent mount configuration
`/tmp/migration-restore`	Temporary directory for recovery testing

Appendix E: Migration Verification Dashboard — Source Code

This appendix lists every file of the dashboard (Section 3.9) with the path where it is stored. Each file is saved exactly as shown. The same folder name,`~/migration-dashboard`, is used on both servers.

E.1 Files, Locations and Where They Run

Table E1 lists the dashboard files, where each is stored and where it runs.

Table E1: Dashboard files

File	Stored at	Run on	Command
make_source_base line.py	~/migration-dashboard/scri pts/	Source	sudo python3 scripts/make_source_baseline.py > baseline/source_baseline.json
source_baseline. json	~/migration-dashboard/base line/	Source, then copied	scp to the destination baseline/ folder
app.py	~/migration-dashboard/back end/	Destination	sudo python backend/app.py
index.html	~/migration-dashboard/back end/templates/	Destination	Served by app.py at /
migration_repor t .txt	~/migration-dashboard/repo rts/	Destination	Created by Generate Report or /api/report

Quick start (after the files are saved and the baseline is copied):

```
# destination server
cd ~/migration-dashboard
sudo python backend/app.py

# EndeavourOS host (second terminal)
ssh -N -L 5000:127.0.0.1:5000 shobith@192.168.56.20
# browser: http://127.0.0.1:5000
```

E.2 Backend — ~/migration-dashboard/backend/app.py

```
#!/usr/bin/env python3 """Linux Server Migration Verification Dashboard. Runs on the DESTINATION server (as root, bound to 127.0.0.1:5000). It reads the live system (LVM, mounts, users, files, backups, services) and compares it with baseline/source_baseline.json created on the SOURCE server. """ import grp import hashlib import json import os import pwd import socket import stat import subprocess import tarfile from datetime import datetime from pathlib import Path from flask import Flask, jsonify, render_template BASE = Path(__file__).resolve().parent.parent CFG = {
```

```
  "data": os.environ.get("MIG_DATA", "/srv/company"),
  "backup": os.environ.get("MIG_BACKUP", "/srv/backup"),
  "log": os.environ.get("MIG_LOG", "/var/log/company-backup/backup.log"),
  "fstab": os.environ.get("MIG_FSTAB", "/etc/fstab"),
  "script": os.environ.get("MIG_SCRIPT", "/usr/local/sbin/company-backup.sh"),
  "baseline": os.environ.get("MIG_BASELINE", str(BASE / "baseline" /
  "source_baseline.json")),
  "report": str(BASE / "reports" / "migration_report.txt"),
  "disk": "/dev/sdb", "part": "/dev/sdb1", "vg": "vg_migration",
  "lv_size_gb": {"lv_data": 12.0, "lv_backup": 5.0},
```

```
} app = Flask(__name__)

def now():

  return datetime.now()

def hostname():

  return socket.gethostname()

def run(cmd):

  try:
     p = subprocess.run(cmd, capture_output=True, text=True, timeout=20)
     return p.returncode, p.stdout.strip()
  except (OSError, subprocess.SubprocessError):
     return 127, ""

def chk(group, name, ok, detail):

  return {"group": group, "name": name, "ok": bool(ok), "detail": detail}

def human(n):

  for unit in ("B", "KiB", "MiB", "GiB", "TiB"):
     if n < 1024 or unit == "TiB":
         return f"{n:.0f} {unit}" if unit in ("B", "KiB") else f"{n:.1f} {unit}"
     n /= 1024

def baseline():

  with open(CFG["baseline"]) as fh:
     return json.load(fh)

def sha256_of(path):

  h = hashlib.sha256()
  with open(path, "rb") as fh:
     for block in iter(lambda: fh.read(1 << 20), b""):
         h.update(block)
  return h.hexdigest()

def usage(mount):

  rc, out = run(["df", "-P", "-B1", mount])
  cols = out.splitlines()[-1].split() if rc == 0 and out else []
  if len(cols) < 6:
     return {"size": 0, "used": 0, "avail": 0, "pct": 0}
  size, used, avail = int(cols[1]), int(cols[2]), int(cols[3])
  return {"size": size, "used": used, "avail": avail,
         "pct": round(used * 100 / size, 1) if size else 0}
```

```
def table(cmd):
    rc, out = run(cmd)
    if rc != 0:
        return []
    return [[c.strip() for c in ln.split("|")] for ln in out.splitlines() if ln.strip()]

# ---------------------------------------------------------------------- storage
def storage():
    g, checks = "Storage", []
    rc, out = run(["lsblk", "-dn", "-o", "SIZE,PTTYPE", CFG["disk"]])
    disk_size, pttype = (out.split() + ["?", "?"])[:2]
    rc, out = run(["lsblk", "-n", "-o", "SIZE,PARTTYPENAME", CFG["part"]])
    part_size, part_type = (out.split(None, 1) + ["?", "?"])[:2]
    pvs = table(["pvs", "--noheadings", "--separator", "|", "--units", "g", "--nosuffix",
                 "-o", "pv_name,vg_name,pv_size,pv_free"])
    vgs = table(["vgs", "--noheadings", "--separator", "|", "--units", "g", "--nosuffix",
                 "-o", "vg_name,vg_size,vg_free,lv_count"])
    lvs = table(["lvs", "--noheadings", "--separator", "|", "--units", "g", "--nosuffix",
                 "-o", "lv_name,vg_name,lv_size,lv_attr"])

    def num(s):
        return float(s.replace("<", "") or 0)

    layers = [
        {"layer": "Disk", "name": CFG["disk"],
         "detail": f"{disk_size} virtual disk, {pttype.upper()} partition table"},
        {"layer": "Partition", "name": CFG["part"], "detail": f"{part_size}, type
{part_type}"},
    ]
    pv = next((r for r in pvs if r[0] == CFG["part"]), None)
    vg = next((r for r in vgs if r[0] == CFG["vg"]), None)
    layers.append({"layer": "Physical volume", "name": CFG["part"],
                   "detail": f"member of {pv[1]}" if pv else "not an LVM physical volume"})
    if vg:
        layers.append({"layer": "Volume group", "name": vg[0],
                       "detail": f"{num(vg[1]):.2f} GiB total, {num(vg[2]):.2f} GiB free, "
                                 f"{vg[3]} logical volumes"})
    mounts, lv_checks = {}, []
    for lv, mp in (("lv_data", CFG["data"]), ("lv_backup", CFG["backup"])):
        row = next((r for r in lvs if r[0] == lv and r[1] == CFG["vg"]), None)
        rc, out = run(["findmnt", "-n", "-o", "SOURCE,FSTYPE", mp])
        src, fst = (out.split() + ["", ""])[:2]
        mounts[lv] = {"mount": mp, "source": src, "fstype": fst, **usage(mp)}
        layers.append({"layer": "Logical volume", "name": lv,
                       "detail": f"{num(row[2]):.2f} GiB, {fst or 'no fs'} mounted at {mp}"
                       if row else "missing"})
        want = CFG["lv_size_gb"][lv]
        lv_checks.append(chk(g, f"{lv} exists with the expected size ({want:.0f} GB)",
                             row and abs(num(row[2]) - want) < 0.1,
                             f"{num(row[2]):.2f} GiB" if row else "not found"))
        lv_checks.append(chk(g, f"{mp} is backed by {lv}, not the root file system",
                             src.endswith(f"{CFG['vg']}-{lv}") and fst == "ext4",
                             f"{src or 'not mounted'} ({fst or '-'})"))
    checks += [
        chk(g, "GPT partition table on the data disk", pttype == "gpt", pttype),
        chk(g, "Partition type is Linux LVM", "LVM" in part_type, part_type),
        chk(g, "LVM physical volume and volume group present", pv and vg,
            f"{CFG['part']} -> {CFG['vg']}" if pv and vg else "missing"),
    ] + lv_checks
    try:
        fstab = Path(CFG["fstab"]).read_text().splitlines()
    except OSError:
```

```
    fstab = []
  persistent = all(any(ln.startswith("UUID=") and ln.split()[1:2] == [mp] for ln in fstab)
                   for mp in (CFG["data"], CFG["backup"]))
  rc, _ = run(["findmnt", "--verify"])
  checks.append(chk(g, "Persistent UUID entries in /etc/fstab (findmnt --verify)",
                    persistent and rc == 0,
                    "UUID entries present, verify clean" if persistent and rc == 0 else "fstab
  problem"))
  return {"layers": layers, "mounts": mounts, "checks": checks}
```

```
# ------------------------------------------------------------------ identities
```

```
def identities():
```

```
  g, base, rows = "Identities", baseline()["identities"], []
  for name, want in base["users"].items():
     try:
         pw = pwd.getpwnam(name)
         got = {"uid": pw.pw_uid, "gid": pw.pw_gid}
     except KeyError:
         got = {"uid": None, "gid": None}
     rows.append({"kind": "user", "name": name, "source": f"{want['uid']}:{want['gid']}",
                  "dest": f"{got['uid']}:{got['gid']}", "ok": got == want})
  for name, gid in base["groups"].items():
     try:
         got = grp.getgrnam(name).gr_gid
     except KeyError:
         got = None
     rows.append({"kind": "group", "name": name, "source": str(gid), "dest": str(got),
                  "ok": got == gid})
  checks = [chk(g, "UIDs and GIDs match the source server", all(r["ok"] for r in rows),
              f"{sum(r['ok'] for r in rows)}/{len(rows)} identities identical")]
  return {"rows": rows, "checks": checks}
```

```
# ------------------------------------------------------------------ migration
```

```
def inventory(root):
```

```
  out = {".": root}
  for dp, dns, fns in os.walk(root):
     for n in dns + fns:
         full = os.path.join(dp, n)
         out[os.path.relpath(full, root)] = full
  return out
```

```
def owner(uid, gid):
```

```
  try:
     u = pwd.getpwuid(uid).pw_name
  except KeyError:
     u = str(uid)
  try:
     gname = grp.getgrgid(gid).gr_name
  except KeyError:
     gname = str(gid)
  return f"{u}:{gname}"
```

```
def migration():
```

```
  g, base, root = "Migration", baseline(), CFG["data"]
  want = {e["path"]: e for e in base["entries"]}
  have = inventory(root)
  rows, bad_own, bad_mode, bad_sum = [], 0, 0, 0
  for path in sorted(set(want) | set(have)):
     e = want.get(path)
```

```
   if path not in have or e is None:
       rows.append({"path": path, "type": "-", "owner": "-", "mode": "-", "sha": "-",
"ok": False,
                    "note": "missing on destination" if e else "not on source"})
       continue
   st = os.lstat(have[path])
   isdir = stat.S_ISDIR(st.st_mode)
   mode = format(stat.S_IMODE(st.st_mode), "o")
   sha = "" if isdir else sha256_of(have[path])
   o_ok = (st.st_uid, st.st_gid) == (e["uid"], e["gid"])
   m_ok = mode == e["mode"]
   s_ok = isdir or sha == e["sha256"]
   bad_own += not o_ok
   bad_mode += not m_ok
   bad_sum += not s_ok
   rows.append({"path": path, "type": "dir" if isdir else "file",
                "owner": owner(st.st_uid, st.st_gid), "mode": mode,
                "sha": sha[:12] if sha else "-", "ok": o_ok and m_ok and s_ok, "note": ""})
nf = sum(1 for r in rows if r["type"] == "file")
nd = sum(1 for r in rows if r["type"] == "dir")
bf = sum(1 for e in base["entries"] if e["type"] == "file")
bd = sum(1 for e in base["entries"] if e["type"] == "dir")
rc, acl = run(["getfacl", "-R", "-s", "-p", root])
acl_n = len([ln for ln in acl.splitlines() if ln and not ln.startswith("#")]) if rc == 0
else -1
rc, xat = run(["getfattr", "-R", "-d", "-m", "-", root])
xat_n = len([ln for ln in xat.splitlines() if ln.startswith("# file")]) if rc == 0 else -
1
inv_ok = set(want) == set(have)
checks = [
   chk(g, "File and directory inventory matches the source", inv_ok,
       f"{nf} files, {nd} directories"),
   chk(g, "File and directory counts match", (nf, nd) == (bf, bd),
       f"source {bf}/{bd}, destination {nf}/{nd}"),
   chk(g, "Numeric ownership matches the source", inv_ok and bad_own == 0, f"{bad_own}
mismatches"),
   chk(g, "Permission modes (incl. setgid) match", inv_ok and bad_mode == 0, f"{bad_mode}
mismatches"),
   chk(g, "SHA-256 checksums of all files match", inv_ok and bad_sum == 0, f"{bad_sum}
mismatches"),
   chk(g, "ACLs and extended attributes match the source",
       (acl_n, xat_n) == (base["acl_entries"], base["xattr_entries"]),
       f"extended ACL entries {acl_n}, xattr files {xat_n}"),
]
tests = []
for user, flag, path, expect, label in (
       ("devuser", "-r", "development/config.txt", True, "devuser reads
development/config.txt"),
       ("financeuser", "-r", "finance/records.txt", True, "financeuser reads
finance/records.txt"),
       ("financeuser", "-w", "development", False, "financeuser writes to development/"),
       ("devuser", "-r", "finance/records.txt", False, "devuser reads
finance/records.txt")):
   rc, _ = run(["runuser", "-u", user, "--", "test", flag, os.path.join(root, path)])
   tests.append({"test": label, "expected": "allowed" if expect else "denied",
                 "actual": "allowed" if rc == 0 else "denied", "ok": (rc == 0) == expect})
checks.append(chk("Access", "Access-control tests behave as intended", all(t["ok"] for t
in tests),
                 f"{sum(t['ok'] for t in tests)}/{len(tests)} tests passed"))
return {"rows": rows, "files": nf, "dirs": nd, "acl": acl_n, "xattr": xat_n,
       "tests": tests, "checks": checks}
```

```
# ------------------------------------------------------------------ backup
def backups():
  g, base, bdir = "Backup", baseline(), Path(CFG["backup"])
  expect = {"company"} | {"company/" + e["path"] for e in base["entries"] if e["path"] !=
  "."}
  rows = []
  for tar in sorted(bdir.glob("*.tar"), key=lambda p: p.stat().st_mtime, reverse=True):
     st, side = tar.stat(), tar.with_name(tar.name + ".sha256")
     try:
         with tarfile.open(tar) as tf:
            content = {n.rstrip("/") for n in tf.getnames()} == expect
     except (tarfile.TarError, OSError):
         content = False
     if side.exists():
         sha = side.read_text().split()[0] == sha256_of(tar)
         sha_txt = "OK" if sha else "FAILED"
     else:
         sha, sha_txt = None, "no .sha256 file"
     rows.append({"name": tar.name, "size": human(st.st_size),
                  "time": datetime.fromtimestamp(st.st_mtime).strftime("%Y-%m-%d %H:%M"),
                  "mode": format(stat.S_IMODE(st.st_mode), "o"), "content": content,
                  "checksum": sha_txt, "ok": content and sha is not False})
  pre = next((r for r in rows if "pre-migration" in r["name"]), None)
  sched = [r for r in rows if "pre-migration" not in r["name"]]
  rc, cron = run(["crontab", "-l"])
  cron_line = next((ln for ln in cron.splitlines()
                    if CFG["script"] in ln and not ln.startswith("#")), "")
  st = os.stat(CFG["script"]) if os.path.exists(CFG["script"]) else None
  script_ok = bool(st and st.st_uid == 0 and st.st_gid == 0 and stat.S_IMODE(st.st_mode) ==
  0o750)
  private = all(int(r["mode"], 8) & 0o007 == 0 for r in rows)
  checks = [
     chk(g, "Pre-migration archive on the backup volume matches the source tree",
         pre and pre["content"], f"{pre['name']} ({pre['size']})" if pre else "archive not
  found"),
     chk(g, "Latest scheduled archive passes its SHA-256 check",
         sched and sched[0]["checksum"] == "OK",
         f"{sched[0]['name']}: {sched[0]['checksum']}" if sched else "no scheduled archive
  yet"),
     chk(g, "Backup archives are not world-readable", rows and private,
         "mode 600" if private else "too open"),
     chk(g, "Backup script is root-owned with mode 0750", script_ok, CFG["script"]),
     chk(g, "Cron entry for the backup script exists", cron_line, cron_line or "not
  scheduled"),
  ]
  return {"rows": rows, "cron": cron_line, "checks": checks}
# ---------------------------------------------------------------- security
def security():
  g, svc = "Security", []
  for s in ("sshd", "cronie", "firewalld"):
     rc, out = run(["systemctl", "is-active", s])
     svc.append({"service": s, "state": out or "unknown", "ok": out == "active"})
  rc, fw = run(["firewall-cmd", "--list-services"])
  checks = [
     chk(g, "sshd, cronie and firewalld are active", all(s["ok"] for s in svc),
         ", ".join(f"{s['service']}={s['state']}" for s in svc)),
     chk(g, "Firewall is active and allows SSH", "ssh" in fw.split(), f"services: {fw or
  'none'}"),
  ]
  return {"services": svc, "firewall": fw, "checks": checks}
```

```
# ---------------------------------------------------------------------- glue
def logs(n=12):
    try:
        return Path(CFG["log"]).read_text().splitlines()[-n:]
    except OSError:
        return []

def summary():
    s, i, m, b, x = storage(), identities(), migration(), backups(), security()
    checks = s["checks"] + i["checks"] + m["checks"] + b["checks"] + x["checks"]
    passed = sum(c["ok"] for c in checks)
    return {"generated": now().strftime("%Y-%m-%d %H:%M:%S"), "host": hostname(),
            "passed": passed, "total": len(checks),
            "status": "VERIFIED" if passed == len(checks) else "ATTENTION",
            "storage": s, "identities": i, "migration": m, "backup": b, "security": x,
            "checks": checks}

def report_text():
    d = summary()
    out = ["LINUX SERVER MIGRATION - VERIFICATION REPORT", "=" * 44,
           f"Generated : {d['generated']}", f"Host : {d['host']}",
           f"Data path : {CFG['data']}", f"Baseline : {CFG['baseline']}"]
    group = None
    for c in d["checks"]:
        if c["group"] != group:
            group = c["group"]
            out += ["", group.upper()]
        out.append(f" [{'PASS' if c['ok'] else 'FAIL'}] {c['name']} - {c['detail']}")
    verdict = "MIGRATION VERIFIED" if d["status"] == "VERIFIED" else "ATTENTION REQUIRED"
    out += ["", f"RESULT: {d['passed']}/{d['total']} checks passed - {verdict}"]
    text = "\n".join(out) + "\n"
    Path(CFG["report"]).parent.mkdir(exist_ok=True)
    Path(CFG["report"]).write_text(text)
    return text

@app.route("/") def index():
    return render_template("index.html")

@app.route("/api/summary") def api_summary():
    return jsonify(summary())

@app.route("/api/storage") def api_storage():
    return jsonify(storage())

@app.route("/api/identities") def api_identities():
    return jsonify(identities())

@app.route("/api/migration") def api_migration():
    return jsonify(migration())

@app.route("/api/backup") def api_backup():
    return jsonify(backups())
```

```
@app.route("/api/security") def api_security():
  return jsonify(security())
@app.route("/api/logs") def api_logs():
  return jsonify({"logs": logs()})
@app.route("/api/report") def api_report():
  return jsonify({"report": report_text(), "saved_to": CFG["report"]})
if __name__ == "__main__":
  if os.geteuid() != 0:
     raise SystemExit("Run with sudo: the dashboard must read root-only data and backups.")
  app.run(host="127.0.0.1", port=5000, debug=False)
```

E.3 Frontend — ~/migration-dashboard/backend/templates/index.html

```
<!DOCTYPE html> <html lang="en"> <head> <meta charset="utf-8"> <meta name="viewport" content="width=device-width, initial-scale=1"> <title>Linux Server Migration Dashboard</title> <style> * { box-sizing: border-box; } body { margin: 0; font-family: Arial, Helvetica, sans-serif; background: #f1f5f9; color: #0f172a; } header { background: #172554; color: #fff; text-align: center; padding: 22px 12px; } header h1 { margin: 0 0 6px; font-size: 26px; } header p { margin: 0; font-size: 13px; color: #cbd5e1; } main { max-width: 1120px; margin: 0 auto; padding: 22px 16px 40px; } .cards { display: grid; grid-template-columns: repeat(4, 1fr); gap: 16px; margin-bottom: 18px; } .card, section { background: #fff; border-radius: 10px; box-shadow: 0 1px 4px rgba(15,23,42,.12); } .card { padding: 16px 18px; } .card h3 { margin: 0 0 10px; font-size: 13px; } .card .big { font-size: 28px; font-weight: bold; color: #2563eb; } .card .big.ok { color: #16a34a; } .card .big.bad { color: #dc2626; } .card small { display: block; margin-top: 8px; font-size: 11px; color: #64748b; } section { padding: 18px 20px; margin-bottom: 18px; } section h2 { margin: 0 0 12px; font-size: 18px; } table { width: 100%; border-collapse: collapse; font-size: 12.5px; } th { text-align: left; background: #f1f5f9; padding: 7px 9px; border-bottom: 2px solid #cbd5e1; } td { padding: 6px 9px; border-bottom: 1px solid #e2e8f0; } td.mono, pre { font-family: "DejaVu Sans Mono", monospace; font-size: 11.5px; } .pill { display: inline-block; padding: 2px 9px; border-radius: 10px; font-size: 11px;
```

```
font-weight: bold; } .pill.ok { background: #dcfce7; color: #166534; } .pill.bad { background: #fee2e2; color: #991b1b; } button { background: #2563eb; color: #fff; border: 0; border-radius: 6px; padding: 8px 14px;
      font-size: 12px; cursor: pointer; margin: 0 8px 12px 0; }
pre { background: #f8fafc; border: 1px solid #e2e8f0; padding: 12px; margin: 0;
   overflow-x: auto; line-height: 1.5; }
.note { font-size: 11.5px; color: #64748b; margin-top: 8px; } @media (max-width: 800px) { .cards { grid-template-columns: 1fr 1fr; } } </style> </head> <body> <header> <h1>Linux Server Migration Dashboard</h1> <p>Source 192.168.56.10 &rarr; Destination 192.168.56.20 &middot;
  storage, identities, data integrity, backup and security</p>
</header> <main> <div class="cards">
  <div class="card"><h3>Migration Status</h3><div class="big" id="c-status">...</div>
    <small id="c-status-s"></small></div>
  <div class="card"><h3>Checks Passed</h3><div class="big" id="c-checks">...</div>
    <small id="c-checks-s"></small></div>
  <div class="card"><h3>Data Volume</h3><div class="big" id="c-data">...</div>
    <small id="c-data-s"></small></div>
  <div class="card"><h3>Backup Volume</h3><div class="big" id="c-bak">...</div>
    <small id="c-bak-s"></small></div>
</div> <section><h2>Storage Layers (Destination)</h2><table id="t-storage"></table></section> <section>
  <h2>User and Group Identities</h2><table id="t-ident"></table>
  <p class="note">Numeric UID:GID on the source baseline versus the live destination.</p>
</section> <section><h2>Data Verification</h2><table id="t-data"></table>
  <p class="note" id="n-data"></p></section>
<section><h2>Access-Control Tests</h2><table id="t-access"></table></section> <section><h2>Backup and Recovery</h2><table id="t-backup"></table>
  <p class="note" id="n-backup"></p></section>
<section><h2>Services and Firewall</h2><table id="t-sec"></table></section> <section><h2>Acceptance Checklist</h2><table id="t-check"></table></section> <section>
  <h2>Backup Logs</h2>
  <button onclick="loadLogs()">Refresh Logs</button>
  <pre id="logs">Loading...</pre>
```

```
</section> <section>

  <h2>Verification Report</h2>
  <button onclick="loadSummary()">Refresh All Checks</button>
  <button onclick="loadReport()">Generate Report</button>
  <pre id="report">Press "Generate Report" to save reports/migration_report.txt.</pre>

</section> </main> <script> function td(text, cls) { const c =
document.createElement("td"); c.textContent = text; if (cls) c.className =
cls; return c; } function pill(ok, yes, no) { const c =
document.createElement("td"), s = document.createElement("span");
s.className = "pill " + (ok ? "ok" : "bad"); s.textContent = ok ? (yes ||
"MATCH") : (no || "DIFFERENT"); c.appendChild(s); return c; } function
fill(id, head, rows) { const t = document.getElementById(id);
t.replaceChildren(); const hr = document.createElement("tr");
head.forEach(h => {

  const th = document.createElement("th");
  th.textContent = h;
  hr.appendChild(th);

}); t.appendChild(hr); rows.forEach(cells => {

  const r = document.createElement("tr");
  cells.forEach(c => r.appendChild(c));
  t.appendChild(r);

}); } function size(b) { if (b >= 1073741824) return (b /
1073741824).toFixed(1) + " GiB"; if (b >= 1024) return Math.round(b /
1024) + " KiB"; return b + " B"; } function set(id, text, cls) { const e =
document.getElementById(id); e.textContent = text; if (cls) e.className =
"big " + cls; } function pct(p) { return p < 1 ? "<1%" : Math.round(p) +
"%"; } async function loadSummary() { const d = await (await
fetch("/api/summary")).json(); const good = d.status === "VERIFIED", cls =
good ? "ok" : "bad"; set("c-status", d.status, cls); set("c-status-s",
"Checked " + d.generated + " on " + d.host); set("c-checks", d.passed + "
/ " + d.total, cls); set("c-checks-s", "Baseline comparison with the
source server"); const m = d.storage.mounts.lv_data, b =
d.storage.mounts.lv_backup; set("c-data", pct(m.pct)); set("c-data-s",
"Used: " + size(m.used) + " of " + size(m.size) + " | " + m.mount);
set("c-bak", pct(b.pct)); set("c-bak-s", "Used: " + size(b.used) + " of "
+ size(b.size) + " | " + b.mount); fill("t-storage", ["Layer", "Name",
"Details"],

   d.storage.layers.map(l => [td(l.layer), td(l.name, "mono"), td(l.detail)]));
```

```
fill("t-ident", ["Type", "Name", "Source UID:GID", "Destination UID:GID",
"Result"],
   d.identities.rows.map(r => [td(r.kind), td(r.name), td(r.source, "mono"),
                          td(r.dest, "mono"), pill(r.ok, "IDENTICAL")]));
fill("t-data", ["Path", "Type", "Owner:Group", "Mode", "SHA-256 (first
12)", "Result"],
   d.migration.rows.map(r => [td(r.path, "mono"), td(r.type), td(r.owner, "mono"),
                         td(r.mode, "mono"), td(r.sha, "mono"),
                         pill(r.ok, "MATCH", r.note || "DIFFERENT")]));
set("n-data", d.migration.files + " files, " + d.migration.dirs +
  " directories | extended ACL entries: " + d.migration.acl +
  " | files with extended attributes: " + d.migration.xattr);
fill("t-access", ["Test", "Expected", "Actual", "Result"],
   d.migration.tests.map(t => [td(t.test), td(t.expected), td(t.actual),
                          pill(t.ok, "PASS", "FAIL")]));
fill("t-backup", ["Archive", "Size", "Created", "Mode", "Content vs
source", "SHA-256 file"],
   d.backup.rows.map(r => [td(r.name, "mono"), td(r.size), td(r.time), td(r.mode,
"mono"),
                       pill(r.content, "MATCH", "DIFFERENT"), td(r.checksum)]));
set("n-backup", "Cron: " + (d.backup.cron || "not scheduled")); const svc
= d.security.services.map(s =>
  [td(s.service, "mono"), td(s.state), pill(s.ok, "ACTIVE", "NOT ACTIVE")]);
const     ssh     =     d.security.firewall.split("     ").includes("ssh");
svc.push([td("firewall services", "mono"), td(d.security.firewall),
       pill(ssh, "SSH ALLOWED", "SSH BLOCKED")]);
fill("t-sec", ["Item", "State", "Result"], svc); fill("t-check", ["Area",
"Check", "Detail", "Result"],
   d.checks.map(c => [td(c.group), td(c.name), td(c.detail), pill(c.ok, "PASS",
"FAIL")]));
} async function loadLogs() { const d = await (await
fetch("/api/logs")).json(); document.getElementById("logs").textContent =
  d.logs.length ? d.logs.join("\n") : "No backup log available.";
} async function loadReport() { const d = await (await
fetch("/api/report")).json();
```

```
document.getElementById("report").textContent = d.report + "\nSaved to: "
+ d.saved_to; } loadSummary(); loadLogs(); setInterval(loadSummary, 30000);
</script> </body> </html>
```

E.4 Baseline Script — ~/migration-dashboard/scripts/make_source_baseline.py

```
#!/usr/bin/env python3 """Run on the SOURCE server (as root) BEFORE the
final synchronisation. Records identities, ownership, modes and SHA-256
checksums of /srv/company so the destination dashboard can compare against
them.
```

```
  sudo python3 make_source_baseline.py > source_baseline.json
```

```
""" import grp import hashlib import json import os import pwd import socket
import stat import subprocess from datetime import datetime ROOT =
"/srv/company" USERS = ["devuser", "financeuser"] GROUPS = ["developers",
"finance"]

def sha256(path):
```

```
  h = hashlib.sha256()
  with open(path, "rb") as fh:
     for block in iter(lambda: fh.read(1 << 20), b""):
         h.update(block)
  return h.hexdigest()
```

```
def count(cmd, marker=None):
```

```
  out = subprocess.run(cmd, capture_output=True, text=True).stdout
  lines = out.splitlines()
  return len([l for l in lines if l.startswith(marker)]) if marker else \
     len([l for l in lines if l and not l.startswith("#")])
```

```
entries = [] paths = ["."] for dp, dns, fns in os.walk(ROOT):
```

```
  paths += [os.path.relpath(os.path.join(dp, n), ROOT) for n in dns + fns]
```

```
for rel in sorted(set(paths)):
```

```
  full = os.path.join(ROOT, rel)
  st = os.lstat(full)
  e = {"path": rel, "type": "dir" if stat.S_ISDIR(st.st_mode) else "file",
      "uid": st.st_uid, "gid": st.st_gid, "mode": format(stat.S_IMODE(st.st_mode), "o")}
  if e["type"] == "file":
     e["size"] = st.st_size
     e["sha256"] = sha256(full)
  entries.append(e)
```

```
baseline = {
```

```
  "generated": datetime.now().strftime("%Y-%m-%d %H:%M:%S"),
  "host": socket.gethostname(),
  "data_root": ROOT,
  "identities": {
     "users": {u: {"uid": pwd.getpwnam(u).pw_uid, "gid": pwd.getpwnam(u).pw_gid} for u in
  USERS},
     "groups": {g: grp.getgrnam(g).gr_gid for g in GROUPS},
  },
  "entries": entries,
  "acl_entries": count(["getfacl", "-R", "-s", "-p", ROOT]),
  "xattr_entries": count(["getfattr", "-R", "-d", "-m", "-", ROOT], "# file"),
} print(json.dumps(baseline, indent=2))
```

E.5 Sample Baseline — ~/migration-dashboard/baseline/source_baseline.json

The file is generated by the script above; the content below is the baseline of the sample data set. Values such as the host name and time differ in each run.

```
{ "generated": "2026-10-15 11:58:40", "host": "source-server", "data_root":
"/srv/company", "identities": {
  "users": {
    "devuser": {
     "uid": 1001,
     "gid": 1001
    },
    "financeuser": {
     "uid": 1002,
     "gid": 1002
    }
  },
  "groups": {
    "developers": 1001,
    "finance": 1002
  }
}, "entries": [
  {
    "path": ".",
    "type": "dir",
    "uid": 0,
    "gid": 0,
    "mode": "755"
  },
  {
    "path": "development",
    "type": "dir",
    "uid": 1001,
    "gid": 1001,
    "mode": "2770"
  },
  {
    "path": "development/config.txt",
    "type": "file",
    "uid": 1001,
    "gid": 1001,
    "mode": "640",
    "size": 33,
```

```
    "sha256": "cf3debbc84344f997f8dc83dc37b2dbf458ceb205246cb9eefe6045bec2c720b"
  },
  {
    "path": "finance",
    "type": "dir",
    "uid": 1002,
    "gid": 1002,
    "mode": "2770"
  },
  {
    "path": "finance/records.txt",
    "type": "file",
    "uid": 1002,
    "gid": 1002,
    "mode": "640",
    "size": 26,
    "sha256": "ad6e39141719a12a93d3e180abff6fc3ca01ac8cbc5a5602d1f576c8a4e7a2e3"
  },
  {
    "path": "shared",
    "type": "dir",
    "uid": 0,
    "gid": 1001,
    "mode": "2775"
  },
  {
    "path": "shared/readme.txt",
    "type": "file",
    "uid": 0,
    "gid": 1001,
    "mode": "644",
    "size": 32,
    "sha256": "dbe54134ce0490b6e54ce9be17f091dbadfb1b4de669d1142595eaabd042a184"
  }
], "acl_entries": 0, "xattr_entries": 0 }
```